How Many Runs Does an RL Post-Training Comparison Need? Forecasting Run-to-Run Spread from a Single Run

Mehmet Demir Güven[†]
ETH Zürich[*]
dgueven@student.ethz.ch

Abstract

Reinforcement learning post-training is compared from single runs, and how many runs a comparison needs is set by a quantity nobody measures: how much a rerun would move the score. The usual recourse is to fix the run count in advance, or to spend runs estimating the spread before the comparison can begin, because after one run per arm there is no empirical spread to put in a stopping rule.

We forecast it instead. Training noise is not accumulated: a perturbation entering at one update is reweighted by every update that follows, so the covariance of a run about its mean obeys $S_{t+1} = A_t S_t A_t^\top + Q_t$. Carrying the gradient of the reported metric backwards along one stored trajectory turns that recursion into the across-seed variance of the metric, as a sum of scalars with no covariance matrix formed. On enumerable policies, where every input is exact, and frozen before the remaining seeds existed, it has a median error of 1.10× over 192 settings of at most 25 updates.

That is a statement about runs that barely move: those settings gain at most 0.036 in pass rate. Raised until runs learn, the same tier breaks the forecast with every input exact, by more than 3× in 44.1% of 354 settings and by orders of magnitude once a policy collapses. Linearising the model instead of the run survives it there: seeds simulated in the tangent model at the starting policy, with sampling and saturation kept exact and no run trained, are within 1.5× in 74.9% of those settings, about what ten real seeds achieve.

Used to size comparisons whose ordering is clear, with every stopping threshold chosen on other tasks before it is applied, it reaches a 0.1% mistake rate in 3.14 runs, 5.6 to 8.2× fewer than a fixed budget measured below its cap and 2.12× fewer than estimating the deviation from spent seeds. A closed form that needs no run comes within 1.11× once its threshold is learned on other tasks; the stored run's scale replaces that learning. Near-ties are settled by no policy within the seeds we have. Corrupting the forecast on purpose puts the point where it stops paying at about 4.2× relative error.

On pretrained models accuracy is regime-dependent. Across 4 pre-specified regimes on Qwen2.5-0.5B the frozen forecast misses by factors of 1.66, 2.13, 2.80, 1.24, one of them inside a criterion declared before the sweep ran and all of them inside that break-even. A corpus whose reward groups go unanimous is missed by 25× although its measured seed spread is the largest we report, and most of its runs turn out to answer one constant whatever they are asked; Qwen2.5-7B under Adam on the same task is short by 3.1 to 17×. We report where it fails as well as where it works.

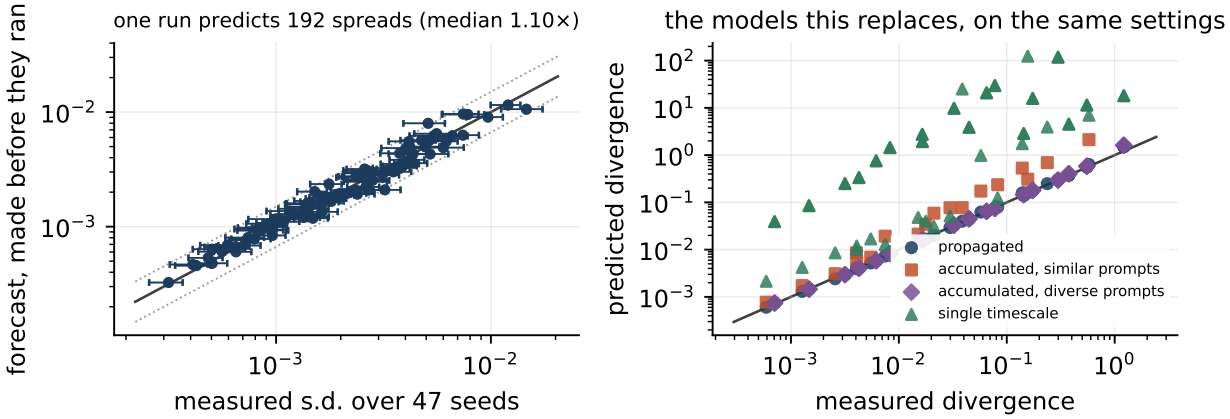


Figure 1: Left: the across-seed standard deviation of the reported pass rate, forecast from a single frozen run, against the value realised over 47 independent runs trained afterwards. Bars are bootstrap intervals over runs; the solid line is equality and the dotted lines a factor of 1.5. **Right:** predicted against measured policy divergence for the models this one replaces. Propagating the covariance tracks it everywhere. Treating noise as accumulating tracks it too where the prompts in a pool are unlike each other, and fails by a factor of two where they are alike, which is where the objective supplies the most curvature. Compressing the filter to a single timescale fails by two orders of magnitude throughout.

*Department of Computer Science, ETH Zürich, 8092 Zurich, Switzerland.

†ETH Zürich neither endorsed, supervised nor funded this work; the author was an undergraduate in its BSc Computer Science programme during this research, hence the affiliation.

1 Introduction

Reinforcement learning with verifiable rewards is now the standard way to turn a pretrained model into a reasoning one [DeepSeek-AI et al., 2025], and a paper reports that it moved a benchmark by two points. Whether that is a result depends on a number nobody has: how much the same recipe would have moved it with a different seed. The problem is documented [Henderson et al., 2017, Hochlehnert et al., 2025] and the standard instrument is replication, which at current RL compute is close to unaffordable. So the number usually goes unmeasured, and single-run improvements are read as though the training procedure were deterministic.

This paper asks what a single run can be made to say about that number. The answer has two halves. Where the comparison can be made exactly, a run does contain enough information to forecast its own spread, and we derive the estimator that extracts it and test it prospectively. Carried to a pretrained model the recommended estimator underpredicts by a factor of 25 and the transported one by 16, for a reason the same framework identifies: the runs stop being small perturbations of each other. Both halves are reported, because a method that is presented without its boundary is not a method.

Getting there means first correcting the model of where the spread comes from. The obvious picture is that gradient noise perturbs each update, perturbations accumulate, two runs execute a random walk away from each other, and reproducibility decays with run length. That picture is implicit in how run-to-run variance is discussed, and it is what a noisy gradient suggests. It is wrong. Noise from one update does not survive to the end of training unchanged: it is passed through every update that follows, which reweights it. What reaches the end is a convolution of injected noise with the sensitivity of the remaining trajectory, an object that obeys a recursion and that can be carried against the gradient of the reported metric instead of being formed.

Contributions.

- **Comparisons sized from one run** (Section 7). The forecast is measured by what it is for. Two recipes are ordered by training seeds until the lead clears z standard errors, a rule that needs a deviation and therefore cannot start until one has been estimated. Supplying it from the first run of each arm, with every threshold chosen on other tasks before it is applied, settles 1020 comparisons at a 0.1% mistake rate in 3.14 runs, against 14 for an uncapped fixed budget and 2.12× as many for a deviation estimated from spent seeds, a rule that cannot stop at one seed per arm however large the lead. Read in sample, as is usual, the saving over the capped fixed budget is 9.57× $[7.24, 10.56]$. The closed form (25), which needs no run, comes within 1.11× $[0.97, 1.28]$ once its threshold is learned on other tasks; what the stored run adds is an absolute scale, which lets the textbook threshold be used with no calibration. Corrupting the forecast deliberately locates the break-even against a fixed budget at 4.2× relative error, and a pure scale error is free only where the threshold can be tuned on comparisons with known outcomes.
- **An error bar from one run** (Theorem 2 and Proposition 1). Carrying the metric gradient backwards along a single stored trajectory gives the across-seed variance of the reported score as a sum of scalars, one per update, with no covariance matrix formed. The transpose the backward pass would need is free, because the mean update of any count-based estimator is the gradient of a scalar potential $\mathbb{E}_x[\Lambda(p_x)]$ and its Jacobian is therefore symmetric. For RLOO the potential is the pass rate itself. In its cheap form, which an ablation shows loses nothing at these run lengths, it costs one extra pass per update, against N full runs for replication. We test it prospectively, training one run and freezing it and recording the forecast before the remaining seeds were trained. On enumerable policies, where every input to the forecast is exact and runs are at most 25 updates long, its median absolute error against the realised spread is 1.10× across 192 settings spanning two policy shapes, two prompt-pool diversities and a 46× range of spreads. Its worst is 1.56×, and it falls inside the measured 95% interval in 162 of them.
- **A finite-time covariance law, and what its operator really does** (Theorem 1, Section 6.3). $S_{t+1} = A_t S_t A_t^\top + Q_t$ predicts measured seed divergence to 3% at the median over 54 settings, against 3.8× at worst for accumulation and two orders of magnitude for a single-timescale approximation. Whether the filter matters is itself a measurement: accumulation is wrong by a factor of two on homogeneous prompt pools and by three percent on heterogeneous ones, which is what the recursion predicts, since a pool of similar prompts is where the objective's curvature is consistent. Measuring the spectrum of A_t says why: it is not a contraction. The spectral radius exceeds one throughout, by up to 2% early and decaying as the policy improves; only 31–51% of directions contract, a share that grows during the run; and the contracting timescales span tens of updates to effectively unbounded. Divergence stays bounded

because expansion is slow relative to run length, not because every direction shrinks.

- **An honest accounting of what the backward pass buys.** Holding the adjoint at ∇M instead of carrying it back changes the forecast by a median of 2% and the median error not at all, so the Hessian-vector products, two of the three passes per update, are unnecessary over runs of this length. The recursion explains why: most of the spectrum of A_t sits within 10^{-4} of one and ∇M points into that part of it. We recommend the cheap estimator and say what would make the full one necessary.

- **The filtering is caused by learning.** Replacing rewards with coin flips, which leaves the rollouts, the batch composition and the step sizes untouched, turns a divergence trace that ends at 0.14 of where it started into one that grows by a factor of 90.5 over eighty updates, a gap of 671× at the end of the run. The learning condition also moves *further* per update, so what brings those runs back together is not smaller steps.

- **Where the uncertainty comes from.** Restricting the injected covariance to one of its terms attributes the forecast by origin. The share owed to which responses were sampled rather than which prompts were drawn falls from 93% at $G = 2$ to 46% at $G = 16$, so the majority changes hands over the range we measure; with prompts drawn independently the number of prompts scales both terms equally and does not move it.

- **On a real model it underpredicts, and we say why** (Section 6.9). Run end to end on Qwen2.5-0.5B, the recommended cheap estimator is 25× short of the measured spread and the transported one 16×, and on Qwen2.5-7B under Adam it is short by between 3.1 and 17. Accuracy is regime-dependent rather than uniformly poor: across 4 pre-specified regimes the errors are $1.66, 2.13, 2.80, 1.24$, one of them inside a criterion fixed before the sweep and all of them inside the break-even above (Section 6.10). On the task the large misses use, most runs, the one the forecast is made from included, learn to answer one constant whatever they are asked; which constant a run falls into is most of the spread, and a linearisation about one run that has already collapsed has no way to represent it. What survives is narrower than the magnitude: the share of prompts whose group is not unanimous falls from about a half to under 5% by the fifth update, which is measured rather than modelled, so the run stops injecting seed noise there. The failed forecast's own kernel puts 90% of its predicted variance before that point, but whether the actual seed variance is distributed the same way we did not test.

- **The boundary reproduced where everything is exact, and an estimator that crosses it** (Sections 3.5 and 6.5). The enumerable settings above gain at most 0.036 in pass rate. Raising the step size until runs gain more breaks the forecast with every input exact: it is more than 3× off in 44.1% of 354 settings and short by up to 1715× once a policy has collapsed, so the failures on pretrained models are failures of linear response about one run and need nothing else. Linearising the model instead of the run gives a surrogate in which seeds can be simulated with the sampling, the estimator and the saturation kept exact and no pass through the model; it is exact for a softmax over one decision (Proposition 2). Made before any run is trained, its spread is within 1.5× of the deviation over 500 real seeds in 74.9% of those settings, against 73.7% for the deviation over ten real seeds and 44.4% for the forecast. Two grids were run with their predictions recorded first; four of the six recorded parts failed, three on one clause, and we report which. On a network the surrogate is untested.

- **A metric can go blind before the seeds agree.** At 7B on a task the base model already mostly passes, twenty updates collapse the response distribution: every held-out prompt scores exactly 0 or 1, two seeds report the same number to the last decimal, and their policies still differ by up to 8×10^{-2} in KL. A zero measured spread on a saturated benchmark is not evidence of a reproducible procedure.

- **The machinery that makes the injected noise measurable**: an exact finite-G mean and covariance for the count-based advantage estimators under binary rewards (Propositions 3 and 4), a decomposition of the gradient noise into between-prompt and within-prompt terms, and an estimator that recovers both from the microbatch gradients a trainer already accumulates. The same decomposition fixes the compute-optimal group size (Theorem 3), which Section 8 develops as an application.

- **An explicit account of what does not hold.** A single effective timescale fails by two orders of magnitude. A predicted square-root relation between policy divergence and outcome spread is not supported by our data and is withdrawn. We looked for a short memory horizon, a long run whose spread is generated only near its end, and found the opposite: in 254 of 288 settings more

of the final variance was injected in the first half of the run than the second, because the injected covariance is largest before the policy has learned anything. Lifting the recursion to Adam's optimiser state works where the moments are exact, at 1.10× against the 24.6× of ignoring that state, and the same lift on a 7B model under Adam leaves a forecast short by between 3.1 and 17. The largest model we measure is 7B with 10 seeds, which bounds what any of this establishes about frontier-scale RLVR.

Every experiment here runs on one laptop, which sets the ceiling on the last point and is stated plainly in Section 10 rather than left for a reader to infer.

2 Setup

A policy $\pi_\theta(y \mid x)$ generates a response y to a prompt $x \sim \mathcal{D}$, and a verifier returns a binary reward $r(x, y) \in \{0, 1\}$. The objective is the expected pass rate

$$J(\theta) = \mathbb{E}_x \mathbb{E}_{y \sim \pi_\theta(\cdot|x)} \big[r(x, y) \big] = \mathbb{E}_x \big[p(x) \big], \qquad (1)$$

with $p(x) := \Pr[\, r = 1 \mid x \,]$ the pass rate on prompt x. Write $\boldsymbol{s}(y) := \nabla_\theta \log \pi_\theta(y \mid x)$ for the score, so that $\mathbb{E}_y[\boldsymbol{s}] = 0$, and let $u_r := \mathbb{E}[\boldsymbol{s} \mid r]$ and $S_r := \mathrm{Cov}(\boldsymbol{s} \mid r)$ be the mean and covariance of the score conditional on the reward. The zero-mean property of the score gives the identity used throughout,

$$p\, u_1 + (1 - p)\, u_0 = 0 \qquad \Longrightarrow \qquad u_0 = -\tfrac{p}{1-p}\, u_1, \ (2)$$

and the per-prompt gradient is $h(x) := \mathbb{E}_y[r\, \boldsymbol{s}] = p\, u_1$. Because the reward is binary, every quantity in this paper depends on the policy only through the four per-prompt objects (p, u_1, S_0, S_1). That reduction is what makes an exact finite-G theory possible.

Batches. A batch consists of P prompts drawn independently from $\mathcal{D}$ and G responses drawn independently from the policy for each prompt. We call G the *group size* and $R = PG$ the *rollouts per step*. Hardware fixes R; how to split it is the question.

The estimator family. Let $k = \sum_j r_j$ be the number of successes in a group. We consider any advantage of the form

$$A_j \;=\; w(k, r_j), \qquad (3)$$

that is, any rule in which a response's advantage depends only on its own reward and on how many of its group-mates succeeded. This covers the estimators in current use, each of which is fully described by a $(G + 1) \times 2$ table (Table 1):

estimator	$w(k, r)$
RLOO [Ahmadian et al., 2024]	$r - (k - r)/(G - 1)$
GRPO, mean baseline [Liu et al., 2025]	$r - k/G$
GRPO, standardised [Shao et al., 2024]	$(r - \bar{r})/\sqrt{\bar{r}(1 - \bar{r})}$
fixed external baseline	$r - \hat{p}$

Table 1: The estimators in current use, each a $(G+1) \times 2$ table. Standardised GRPO is defined to be zero when $k \in \{0, G\}$, where the group has no spread; $\bar{r} = k/G$.

The per-prompt estimator and the batch estimator are

$$z \;:=\; \frac{1}{G} \sum_{j=1}^{G} w(k, r_j)\, \boldsymbol{s}(y_j), \qquad \hat{\boldsymbol{g}} \;:=\; \frac{1}{P} \sum_{i=1}^{P} z_i. \quad (4)$$

Token-level reshaping of the advantage, length normalisation and clipping fall outside (3); Section 10 returns to them.

How every interval in this paper is computed. Seed experiments produce pairwise quantities: n runs give $\binom{n}{2}$ distances, and it is tempting to treat those as $\binom{n}{2}$ observations. They are not. The independent units are the n runs, and the distances built from them are dependent by construction, since every run appears in $n - 1$ of the pairs. Treating the pairs as independent understates the standard error by roughly a factor of two at the seed counts anyone actually trains. Every interval reported here is a bootstrap that resamples *runs* and rebuilds the pairwise quantity from the resampled set, and the difference this makes is stated wherever it changes a conclusion. It changes one: at six seeds it turns an apparent peak in the divergence trace of a pretrained model into a shape that is not resolved.

3 From one run to the spread of its result

Fix a policy θ_0 and start two runs from it. They begin from the same parameters, the same optimiser state, the same data distribution and the same recipe, and independently draw every stochastic quantity the training seed controls: which prompts each batch contains and which responses are sampled for them. That is the comparison a practitioner makes when they change a seed and rerun, and it is the estimand throughout, so the two terms of Section 2 both contribute: Σ_{b} carries the uncertainty in which prompts were drawn and Σ_{w} the uncertainty in which responses were sampled. Holding the prompt schedule fixed and varying only the rollouts would condition the first away, and we do not do that anywhere. The question this section answers is how far apart the numbers two such runs report end up, using only one of them.

3.1 Noise is filtered, not accumulated

Take one run and write $\delta_t := \theta_t - \bar{\theta}_t$ for its departure from the mean trajectory. With updates $\theta \leftarrow \theta + \eta\hat{\boldsymbol{g}}$ and $\mathcal{J}_t := \partial\bar{\boldsymbol{g}}/\partial\theta$ the Jacobian of the mean update,

$$\delta_{t+1} = A_t\delta_t + \eta\,\xi_t, \qquad A_t := I + \eta\mathcal{J}_t, \tag{5}$$

to first order in δ, where ξ_t is the batch noise at update t, with $\mathbb{E}[\xi_t \mid \mathcal{F}_t] = 0$ and $\mathrm{Cov}(\xi_t) = \Sigma_t/P$.

Two independent runs enter only afterwards. Writing $\Delta_t := \theta_t^A - \theta_t^B = \delta_t^A - \delta_t^B$ for a pair whose noise is drawn independently, $\mathrm{Cov}(\Delta_t) = 2\,\mathrm{Cov}(\delta_t)$: the factor of two belongs to the difference of two runs and not to the covariance of one, and keeping the two objects apart is what makes it unambiguous. The batch noise Σ_t is left abstract here and made explicit in Section 4, where it is computed exactly at finite group size; nothing in this section depends on which estimator produced it.

If the objective were flat, $\mathcal{J} = 0$, this would be a random walk: divergence would accumulate, $\mathbb{E}\|\Delta_t\|^2 \propto t$, and a longer run would be strictly less reproducible than a short one. That is the implicit model behind treating run-to-run variance as something to be measured by brute force. But two RLVR runs are climbing the same objective, and A_t reweights a perturbation by the local curvature rather than passing it through. We will call this filtering rather than contraction, because Section 6.3 measures the spectrum of A_t and finds it straddling one: a minority of directions contract, the leading direction expands slowly, and the contracting timescales differ among themselves by more than an order of magnitude.

The consequence is that noise from update t is not added to the final divergence. It is passed through every subsequent update first.

Theorem 1 (Finite-time covariance)**.** *Let $S_t := \mathrm{Cov}(\delta_t)$ denote the covariance of one run about its mean trajectory, with $S_0 = 0$ when runs start from the same policy. Under the linearisation* (5)*,*

$$S_{t+1} = A_tS_tA_t^\top + Q_t, \qquad Q_t := \eta^2\Sigma_t/P, \tag{6}$$

and therefore

$$S_T = \sum_{t=0}^{T-1} \Phi_{T,t+1}\,Q_t\,\Phi_{T,t+1}^\top, \qquad \Phi_{T,s} := \prod_{u=s}^{T-1} A_u, \tag{7}$$

the product taken in order of decreasing u. Two independent seeds satisfy $\mathrm{Cov}(\theta_T^A - \theta_T^B) = 2S_T$, so their expected policy divergence is

$$\mathbb{E}\big[\mathrm{KL}(\pi^A \,\|\, \pi^B)\big] \;\approx\; \mathrm{tr}\big(F_T\,S_T\big). \tag{8}$$

Proof. Taking the covariance of (5) for a single run about its mean leaves a cross term $\mathbb{E}[\delta_t\xi_t^\top]$. It vanishes without assuming ξ_t and δ_t are independent, which they are not: the batch noise is drawn at θ_t, so its law depends on a parameter that earlier noise produced. What holds is that it is a martingale difference with respect to the filtration $\mathcal{F}_t$ generated by the run up to t, $\mathbb{E}[\xi_t \mid \mathcal{F}_t] = 0$, and δ_t is $\mathcal{F}_t$-measurable, so $\mathbb{E}[\delta_t\xi_t^\top] = \mathbb{E}\big[\delta_t\,\mathbb{E}[\xi_t^\top \mid \mathcal{F}_t]\big] = 0$. That gives (6); unrolling it from $S_0 = 0$ gives (7). Two independent runs have independent noise, so the covariance of their difference is twice that of one about the mean. Expanding $\mathrm{KL}(\pi^A\|\pi^B) \approx \frac{1}{2}\Delta^\top F\Delta$ and taking expectations gives $\frac{1}{2}\,\mathrm{tr}(F \cdot 2S_T)$. □

Equation (7) is the object this section is about, and it is worth reading as a convolution: what survives to the end is the noise injected at each update, weighted by how strongly the remaining trajectory preserves a perturbation entering there. Write

$$k_t \;:=\; \mathrm{tr}\big(F_T\,\Phi_{T,t+1}Q_t\Phi_{T,t+1}^\top\big), \qquad \mathbb{E}[\mathrm{KL}] = \sum_t k_t, \tag{9}$$

for the contribution of update t. Two runs are not equally sensitive to noise at every point in training, and k_t says where the sensitivity is.

3.2 Why a single timescale is not enough

An earlier version of this work replaced the spectrum of A_t by one effective eigenvalue, giving $\mathbb{E}[\mathrm{KL}_\infty] = \tau_c D_{\mathrm{noise}}$ for a persistence time τ_c. That approximation is the natural first move and it fails badly: Section 6.2 measures it at a median error of two orders of magnitude, against 3% for (7). The reason is visible in (7). A_t has a spectrum, fast directions equilibrate within a few updates while slow ones set the final level, and a single fitted exponential is pinned by the early rise and blind to what dominates the answer. We report the scalar law only as the comparator it turned out to be.

3.3 What the recursion predicts

Divergence stops growing when, and because, the run is learning. $\mathcal{J}_t$ is the curvature of the estimator potential Ψ, and it is what makes A_t anything other than the identity. With no learning signal there is no potential to be curved: $\mathcal{J}_t \to 0$, $A_t \to I$, and (5) degenerates to a random walk in which every update's noise survives to the end unaltered. Note that this does not require $\mathcal{J}_t$ to be negative definite, and Section 6.3 finds that it is not; what it requires is that the directions carrying the variance are ones the objective bends. The claim is testable directly by removing the learning signal while holding the sampling protocol fixed, which Section 6.2 does.

The level is set by batch composition. $Q_t \propto \Sigma_t/P$, so doubling the prompt batch halves the injected *covariance* and quadrupling it halves the corresponding standard-deviation scale. Nothing in (7) depends on the number of updates once the sum has converged.

A memory horizon, if the filter is sharp enough to make one. If $\Phi_{T,t+1}$ decays with $T-t$, the sum in (7) is dominated by its last terms and the final $h \ll T$ updates account for nearly all of the spread; reproducing a result would then mean reproducing its end. Where the filter is close to the identity, k_t is flat and every update counts the same. Section 6.2 asks which of these holds and finds the second, over every setting we can reach.

3.4 Carrying it to the number that gets reported

Equation (8) is a statement about policies. What a paper reports is a score: pass rate, exact match, win rate, mean reward. Write $M(\theta)$ for that score as a function of the parameters. Linearising it about the endpoint the run reached,

$$\mathrm{Var}_{\text{seed}}\big[M(\theta_T)\big] \;\approx\; \nabla M(\theta_T)^\top S_T\, \nabla M(\theta_T), \qquad (10)$$

which needs S_T, a matrix nobody can form for a real model. It is not needed. Substituting (7) turns the quadratic form into a sum of scalars.

Theorem 2 (Adjoint form of the reported spread)**.** *Define $b_T := \nabla M(\theta_T)$ and $b_s := A_s^\top b_{s+1}$ for $s = T-1, \dots, 0$. Then, under the assumptions of Theorem 1,*

$$\begin{aligned} V_T^{lin} \;:=\; \nabla M^\top S_T \nabla M \;&=\; \sum_{t=0}^{T-1} b_{t+1}^\top Q_t\, b_{t+1} \\ &=\; \frac{\eta^2}{P} \sum_{t=0}^{T-1} \mathrm{Var}_i\big(b_{t+1}^\top z_{t,i}\big), \end{aligned} \qquad (11)$$

where $z_{t,i}$ is the per-prompt estimator of Section 2 at θ_t and the variance is taken over prompts. The equality is exact for V_T^{lin}, the variance of the metric linearised at $\bar\theta_T$; the quantity a practitioner cares about is $\mathrm{Var}_{seed}[M(\theta_T)]$, and the two agree to the order at which (10) *holds. Keeping them apart matters here because the whole question of Section 6.11 is when that agreement fails. Since $\hat{\boldsymbol{g}}_t = P^{-1}\sum_i z_{t,i}$, the same term is $\eta^2\,\mathrm{Var}(b_{t+1}^\top \hat{\boldsymbol{g}}_t)$; the factor P belongs with the per-prompt form and not with the batch one, and we use the per-prompt form throughout because it is what Algorithm 1 accumulates.*

Proof. $b_s = \Phi_{T,s}^\top \nabla M$ by induction. Substituting (7) into (10) gives $\sum_t (\Phi_{T,t+1}^\top \nabla M)^\top Q_t (\Phi_{T,t+1}^\top \nabla M)$. The second equality is $Q_t = \eta^2 \Sigma_t / P$ together with $b^\top \Sigma_t b = \mathrm{Var}_i(b^\top z_{t,i})$, since $\Sigma_t = \Sigma_{\mathrm{b}} + \Sigma_{\mathrm{w}}$ is the covariance of one prompt's contribution rather than of the batch mean. □

Algorithm 1 The forecast, as a trainer would run it

```
train as usual, storing θ_t and η_t for each update
b ← ∇M(θ_T)   ▷ one rollout and backward pass on the eval set
v ← 0
for t = T − 1 down to 0 do
    draw a batch at θ_t; let ĝ_{t,i} be its per-prompt gradients
    v ← v + (η_t^2 / P) · Var_i(b^⊤ ĝ_{t,i})          ▷ one scalar
    if transport is needed then
        δ ← ε b / ‖b‖
        b ← b + (η_t / 2) · (‖b‖ / ε) (ḡ(θ_t + δ) − ḡ(θ_t − δ))   ▷ common random numbers
    end if
end for
return √v   ▷ forecast s.d. of M across seeds; √(2v) is the s.d. of the signed difference between two runs
```

Lines 7–10 are what the recursion asks for and what Section 6.1 finds costs three times as much for a 2% correction over runs of this length. Omitting them gives (12).

Every term of (11) is the variance of a batch gradient projected onto a single direction: one scalar per update, which a trainer can accumulate alongside the update it is already making. What the backward pass costs depends on $A_s^\top$, and here the estimator family pays for itself.

Proposition 1 (The mean update is a gradient field)**.** *For any count-based estimator, $\bar{\boldsymbol{g}}(\theta) = \nabla_\theta\, \Psi(\theta)$ with $\Psi(\theta) = \mathbb{E}_x[\Lambda(p_x)]$ and $\Lambda' = \lambda$. Hence $\mathcal{J} = \nabla^2 \Psi$ is symmetric and $A_s^\top = A_s$.*

Proof. By Proposition 3, proved in Section 4, the per-prompt mean is $\lambda(p_x, G)\, \nabla_\theta p_x$, and λ depends on θ only through p_x. So $\lambda(p_x)\nabla p_x = \nabla \Lambda(p_x)$, and averaging over x commutes with the gradient. □

We call Ψ the *estimator potential*. It is not the objective J of Section 2 unless $\lambda \equiv 1$; the gap between them is exactly the difficulty reweighting that Section 4 makes explicit. The backward pass is therefore a Hessian-vector product, which finite differences deliver from two gradient evaluations and reverse mode from one double backward. Neither $\mathcal{J}$ nor S_T is ever formed. For RLOO, where $\lambda \equiv 1$, the potential is the pass rate itself: the mean update ascends exactly the quantity being reported.

What a practitioner would do with this. Three things during the run, none of which changes the run: store θ_t and η_t at each update, accumulate the scalar

$\widehat{\mathrm{Var}}_i(b^\top \hat{g}_{t,i})$ once b is known, and log the fraction of prompts whose group is not unanimous. Then one rollout and backward pass on the evaluation set for $\nabla M(\theta_T)$, and the backward sweep of Algorithm 1. The extra cost is two gradient evaluations per stored update and one vector of parameter size held in memory.

And one thing before believing the answer: check that the runs it describes are small perturbations of each other. The pairwise residual of Section 6.4 measures that directly on a pair of runs already in hand, and Section 6.9 is what happens when nobody checks, on a real task whose runs separate by which constant answer they collapse to, where the forecast is short by a factor of 25 and no amount of care in computing it would have helped.

What it costs, plainly. As written, the backward pass makes three passes per stored update: one batch for the injected term, and two for the Hessian-vector product. That is about three times the compute of the training run itself, not a rounding error, and we would rather state it than let a reader discover it.

It is also usually unnecessary. Section 6.1 ablates the transport, holding b at $\nabla M(\theta_T)$ instead of carrying it back, and finds the forecast changes by a median of 2% and the median error not at all, over runs of ten and twenty-five updates. The reason is in the spectrum: most of A_t sits within 10^{-4} of one and ∇M points into that part of it, so $\Phi_{T,t}$ is close to the identity along the one direction the forecast needs. What we recommend, then, is the cheap estimator,

$$\mathrm{Var}_{\text{seed}}\big[M(\theta_T)\big] \;\approx\; \sum_t \frac{\eta_t^2}{P}\,\widehat{\mathrm{Var}}_i\big(\nabla M(\theta_T)^\top \hat{g}_{t,i}\big), \quad (12)$$

one extra pass per update and no Hessian-vector products, with the full recursion held in reserve for runs long enough, or curvature strong enough, that a per-update discrepancy of a percent compounds. Knowing which regime one is in is what the recursion is for, and the spectrum of A_t is what says it.

Equation (12) is (11) with $\Phi_{T,t} \approx I$ along ∇M; it is not a different method but the same one, evaluated where the operator is measured to be close to the identity.

The comparison that matters is not against zero. Estimating the same quantity by replication costs N full runs and returns an N-seed estimate whose own interval is wide. At six seeds it is wide enough to leave the shape of a divergence trace unresolved (Section 6.2). One extra pass per update is about the cost of a second run, and it is available on a run that has already finished.

The number that comes out is the seed component only. A reported score also carries the randomness of the evaluation set, which is the binomial term the eval literature already estimates [Miller, 2024]. The two are independent by construction, one being over training seeds at a fixed evaluation and the other over evaluation draws at a fixed model, so they add in quadrature, and the practical recommendation is to report both terms rather than either alone. On a saturated benchmark the second is small and the first is invisible, which is exactly the situation Section 6.13 runs into.

Two consequences worth stating plainly. First, this is a *forecast*: (11) reads only the states one run visited, so it can be computed before any second run exists, which is how Section 6.2 tests it. Second, restricting Σ_t to one of its two terms attributes the answer by origin, since Σ_b is which prompts were drawn and Σ_w is which responses were sampled. The two shares are exact and sum to one.

3.5 Linearising the model instead of the run

Everything above linearises the training dynamics about one trajectory. A different linearisation is available, and it fails differently. Fix reference parameters θ_0, the point runs start from, and write $\boldsymbol{s}_0(x,y) := \nabla_\theta \log \pi_{\theta_0}(y \mid x)$ for the score there. The *tangent policy* is

$$\pi'_\vartheta(y \mid x) \;=\; \frac{\pi_{\theta_0}(y \mid x)\,\exp\big(\boldsymbol{s}_0(x,y)^\top \vartheta\big)}{Z_x(\vartheta)}, \qquad \pi'_0 = \pi_{\theta_0}, \quad (13)$$

an exponential family whose sufficient statistics are the frozen scores. Its own score is $\boldsymbol{s}_0(x,y) - \mathbb{E}_{\pi'_\vartheta}[\boldsymbol{s}_0(x,\cdot)]$, so an update of ϑ by any estimator of (3) needs the sampled responses, their rewards and inner products between frozen scores, and no pass through the model. A seed is then *simulated* rather than trained: draw the batch, draw each group from π'_ϑ, apply the estimator's table, update ϑ. Which prompts are drawn, which responses are sampled, the estimator's weights, the normalisation, and the fact that a unanimous group contributes nothing are all kept exactly. What is given up is the movement of the scores.

Proposition 2 (When a simulated seed is a real one)**.** *Suppose that for every prompt the log-probabilities are affine in the parameters up to a normaliser,* $\log \pi_\theta(y \mid x) = \log \pi_{\theta_0}(y \mid x) + \boldsymbol{s}_0(x,y)^\top(\theta-\theta_0) - c_x(\theta)$*. Then for any estimator of* (3)*, the run* $\theta_{t+1} = \theta_t + \eta\hat{\boldsymbol{g}}_t$ *started at* θ_0 *and the tangent run started at* $\vartheta_0 = 0$ *have the same law:* $(\theta_t - \theta_0)_{t\le T} \overset{d}{=} (\vartheta_t)_{t\le T}$*. In particular* $M(\theta_T)$ *has the distribution of its tangent counterpart, not only its variance.*

Proof. Normalisation forces $c_x(\theta) = \log Z_x(\theta-\theta_0)$, so $\pi_\theta = \pi'_{\theta-\theta_0}$, and differentiating gives $\nabla_\theta \log \pi_\theta(y \mid x) = \boldsymbol{s}_0(x,y) - \mathbb{E}_{\pi_\theta}[\boldsymbol{s}_0(x,\cdot)]$, which is the tangent score at $\vartheta = \theta - \theta_0$. The two runs are therefore Markov

chains with the same starting point, the same sampling distribution in every state and the same update map, hence the same transition kernel. □

A softmax over a single decision satisfies the hypothesis. An autoregressive policy does not, because its normaliser is applied token by token and depends on the prefix, and neither does a network. There the scores move: the tangent run is exact in its first update, where the frozen scores are the real ones, and departs as they drift. Equation (13) is the lazy or tangent-kernel approximation [Jacot et al., 2018, Chizat et al., 2019, Lee et al., 2019] applied to the log-probability of each response, and prompt-based fine-tuning of language models is often well described by it [Malladi et al., 2023]. What is different here is the use it is put to.

Theorem 2 is exact in the model and first-order in the distance between runs. It needs one run, and it inherits that run's blind spot: when the run saturates, $\nabla M(\theta_T) \to 0$ and the forecast vanishes with it, wherever the other seeds went. Equation (13) is first-order in the model and exact in the distance between runs. Simulated seeds may end arbitrarily far apart, and a run that has collapsed onto one response is a state of the surrogate like any other. It needs no run at all. Its cost is one backward pass per prompt and candidate response to obtain $\boldsymbol{s}_0$, after which the Gram matrix of those scores is all a simulated update touches. The two approximations fail in different places, and Section 6.5 measures both on the same settings.

3.6 Where the argument stops

Everything above linearises the update around the shared trajectory, which requires the two runs to stay close enough for $\bar{\boldsymbol{g}}$ to be locally linear between them, and requires $\eta\|\mathcal{J}_t\| \ll 1$. Both fail when the step grows relative to the curvature, and RLVR supplies a concrete way for that to happen: a controller holding drift at a fixed target must raise η as the signal decays, and $\mathbb{E}_x[p(1-p)] \to 0$ as a task is solved takes the signal with it. Section 6.2 shows runs crossing out of the regime exactly this way.

A second boundary is not a failure of the linearisation but a property of the run, and the kernel makes it visible. A count-based advantage is identically zero on a group whose responses all earn the same reward, so once a policy has become deterministic on the corpus, $\Sigma_t = 0$ and *no noise enters at all*. The forecast then reports that every later update contributes nothing, which is correct: two seeds from that point on are the same run. Whether this is good news depends on why the policy stopped sampling. Section 6.13 shows an RLVR run reaching it in a single update at a step size a practitioner might well choose, and the diagnostic that separates it from an estimation failure is the fraction of prompts whose group is not unanimous, which costs nothing to log.

4 Supplying Σ_t: what the estimators compute, and how noisy they are

Section 3 left two objects abstract: the batch covariance Σ_t that (6) injects, and the difficulty weight λ that Proposition 1 integrates into a potential. This section computes both, exactly, at the group sizes actually used. Everything the forecast needs about the estimator is here.

All results in this section are exact at finite G, and all are verified against brute-force enumeration in Section 6. Define, for $m \sim \text{Binomial}(G-1, p)$ and $m' \sim \text{Binomial}(G-2, p)$,

$$\begin{aligned} W_r &:= \mathbb{E}_m\big[w(r+m, r)\big], \\ Q_r &:= \mathbb{E}_m\big[w(r+m, r)^2\big], \\ C_{ab} &:= \mathbb{E}_{m'}\big[w(a+b+m', a)\, w(a+b+m', b)\big]. \end{aligned} \tag{14}$$

4.1 The estimator zoo is a reweighting of prompt difficulty

Proposition 3 (Exact expectation)**.** *For any estimator of the form* (3) *and any binary reward,*

$$\mathbb{E}[z \mid x] = \lambda(p, G)\, h(x), \quad \lambda := W_1(p, G) - W_0(p, G). \tag{15}$$

Each of the following is exact for every p and every G. RLOO has $W_1 = 1-p$ and $W_0 = -p$, hence $\lambda \equiv 1$: it is unbiased. GRPO with a mean baseline has $\lambda = (G-1)/G$, independent of p: it is RLOO multiplied by a constant. Standardised GRPO has a $\lambda(p, G)$ with no closed form but an exact binomial sum.

So none of the estimators in Table 1 points in the wrong direction *on any individual prompt*: each computes $h(x)$ scaled by a positive weight, for every $p \in (0, 1)$ and every G. That is a property of those weight tables and not of the family, since an arbitrary $w(k, r)$ can plainly give $\lambda < 0$; the estimators in use happen to lie in the subset where $W_1 > W_0$ throughout. This does not carry over to the aggregate. The family optimises the reweighted field $\mathbb{E}_x[\lambda(p(x), G)\, h(x)]$, which need not be collinear with $\mathbb{E}_x[h(x)]$ once prompts disagree, so a p-dependent λ can rotate the direction the run actually follows. Standardisation is therefore not *merely* a variance-reduction transformation: under binary rewards it also changes the expected gradient, upweighting prompts whose pass rate is near 0 or 1 relative to prompts of intermediate difficulty. Whether

it additionally reduces variance is a separate question that Proposition 3 does not settle either way.

A constant λ is invisible: rescaling an estimator by c rescales the optimal step size by $1/c$ and leaves every trajectory unchanged. Only the *dependence of λ on p* can alter what a run converges to. This is the sense in which most of the zoo is one estimator.

4.2 Exact covariance at finite group size

Proposition 4 (Exact per-prompt covariance)**.** *With* $U := u_1 u_1^\top$*,*

$$\mathrm{Cov}(z \mid x) = \frac{1}{G}\Big[p\,Q_1 S_1 + (1-p)\,Q_0 S_0\Big] + c_U(p,G)\,U, \tag{16}$$

$$c_U = \frac{1}{G}\Big(p\,Q_1 + \frac{p^2 Q_0}{1-p}\Big) + p^2\Big[\tfrac{G-1}{G}\big(C_{11} - 2C_{10} + C_{00}\big) - \lambda^2\Big]. \tag{17}$$

4.3 Two levels of noise

Prompts are drawn independently, so with $\bar{\boldsymbol{g}} := \mathbb{E}_x[\lambda h]$ the batch estimator has

$$\mathrm{Cov}(\hat{\boldsymbol{g}}) \;=\; \frac{1}{P}\Big[\Sigma_{\mathrm{b}} + \Sigma_{\mathrm{w}}\Big], \tag{18}$$

$$\Sigma_{\mathrm{b}} := \mathrm{Cov}_x\big(\lambda(p(x),G)\,h(x)\big),$$
$$\Sigma_{\mathrm{w}} := \mathbb{E}_x\big[\,\mathrm{Cov}(z \mid x)\big].$$

Σ_{b} is the irreducible variance of sampling prompts and does not shrink with G; Σ_{w} is the variance of sampling responses and does. Pretraining has one level of sampling and therefore one noise scale. RLVR has two, and the split of a fixed rollout budget between them is a free choice that is not determined by the noise scale alone. Concurrent work approaches the same split from a different direction: Zhou et al. [2026] show the GRPO policy gradient is a U-statistic, characterise its mean squared error at finite sample, and derive a scaling law for the group size from that error. Our route is the two-level covariance itself, which is what lets the same measurement serve the seed-uncertainty forecast and the allocation question together, and what carries the prefill economics of Section 8; the group-size result should be read as one consequence of that decomposition rather than as the first principled treatment of it.

Corollary 1 (The reward histogram predicts the response-level noise)**.** *Exactly, from Proposition 4,*

$$\mathrm{tr}\,\Sigma_{\mathrm{w}} \;=\; \mathbb{E}_x\Big[\tfrac{1}{G}\big(pQ_1\,\mathrm{tr}\,S_1 + (1-p)Q_0\,\mathrm{tr}\,S_0\big) \;+\; c_U\|u_1\|^2\Big]. \tag{19}$$

For RLOO, $pQ_1 + (1-p)Q_0 = p(1-p)\,G/(G-1)$ exactly, for every G. If in addition the conditional score covariances are comparable across the two reward classes, $\mathrm{tr}\,S_0 \approx \mathrm{tr}\,S_1 \approx \sigma_s^2$, and the outer-product term is small against the first, then

$$\mathrm{tr}\,\Sigma_{\mathrm{w}} \;\approx\; \frac{\sigma_s^2}{G-1}\,\mathbb{E}_x\big[p(x)\,(1-p(x))\big]. \tag{20}$$

The response-level noise is then, up to one slowly varying scalar, the mean Bernoulli variance of the pass-rate distribution: a quantity every RLVR trainer already logs, at no cost.

Both conditions are approximations rather than identities, and we measure both rather than assert them. The outer-product term dropped in (20) carries between 5.6% and 9.6% of $\mathrm{tr}\,\Sigma_{\mathrm{w}}$ over the pools and group sizes used here, falling as G grows, so (20) runs that much low; the exact form (19) is what the implementation evaluates and what every number in Section 6 uses. The comparability of $\mathrm{tr}\,S_0$ and $\mathrm{tr}\,S_1$ is a statement about conditional score scales, not about the shape of either covariance, and Section 6 reports how far it holds across difficulty buckets.

5 Measuring the two noise scales from a single batch

Both τ_b and τ_w are recoverable from gradients a trainer already computes. Index the microbatches by prompt block and by rollout sub-group and accumulate them separately:

$$g_{ab} := \text{mean gradient over block } a \text{ using sub-group } b, \tag{21}$$

for $a = 1 \ldots K$ and $b \in \{1, 2\}$, each computed with its own sub-group's advantages over P/K prompts and $G/2$ responses. Blocks hold disjoint prompts, and given a prompt the two sub-groups are independent, so every term below is an inner product of independent quantities:

$$\bar{\boldsymbol{g}}^\top M \bar{\boldsymbol{g}} = \mathbb{E}\big[g_{a1}^\top M\, g_{a'2}\big]_{a \neq a'}, \tag{22}$$
$$\mathrm{tr}(M\Sigma_{\mathrm{b}}) = \tfrac{P}{K}\Big(\mathbb{E}\big[g_{a1}^\top M\, g_{a2}\big] - \bar{\boldsymbol{g}}^\top M \bar{\boldsymbol{g}}\Big),$$
$$\mathrm{tr}(M\Sigma_{\mathrm{w}}) = \tfrac{P}{2K}\,\mathbb{E}\big[\|g_{a1} - g_{a2}\|_M^2\big].$$

The first pairs different prompts with different responses, the second the same prompts with different responses, and the third cancels the prompt-level term inside the difference.

The cost is $2K$ gradient buffers rather than one, no extra responses, and no extra backward work beyond tagging each microbatch – which any trainer using gradient accumulation is already positioned to do. Raising K averages the block statistics over $K(K-1)$ ordered

pairs rather than one and shrinks the P/K multiplier in front of the noisiest term, so the estimate improves with the number of microbatches the trainer already uses. $M = I$ gives the practical form; $M = F$ is obtained from measured KL rather than by materialising the Fisher.

Getting this right requires handling three biases, each easy to walk into.

The subtractive form is badly conditioned. Estimating the total variance and subtracting the within-prompt part differences two large, highly correlated quantities. On homogeneous pools it returns a negative τ_b and an infinite $G^\star$; where it returns something finite, it compresses the answer toward the middle of the range. Measured against exactly known values over pools whose noise ratio spans a factor of 26, (22) recovers $G^\star$ to a median error of 1.3% and a worst case of 6.1%, while the subtractive form reports 6.1 where the truth is 9.4 and 6.9 where the truth is 15.2.

The probe measures a half-sized estimator. One calibration batch supplies two independent estimates per prompt by splitting the group in two, so the advantages inside each half are computed among $G/2$ responses and what the probe measures is the $G/2$ estimator, not the one the run trains with. It is rescaled to full G by the $1/(G-1)$ law that Corollary 1 implies. That extrapolation is worth checking, and it is only safe for some estimators. Against the exact $\operatorname{tr}\Sigma_{\mathrm{w}}$ at full G it is accurate to +2.9% at $G = 4$ and +0.4% at $G = 32$ for RLOO, whose within-prompt term follows the law almost exactly. For the mean-baseline and standardised GRPO variants the same extrapolation is low by 54% and 61% at $G = 4$, and still by 25% and 29% at $G = 8$, because their weight tables make the within-prompt term depend on G in a way the law does not capture. Every measurement reported here uses RLOO. A practitioner probing a GRPO variant should draw two full groups per prompt on the calibration batch instead, at twice the rollout cost on those batches only.

Sampling with replacement needs no correction. Two blocks of a batch drawn with replacement are independent draws from the prompt distribution, so the different-block inner product is unbiased for $\bar{\boldsymbol{g}}^\top M \bar{\boldsymbol{g}}$ whether or not a prompt happens to appear twice. A coincidence between two independent indices does not make them dependent. Measured against the exact pool value the estimator returns 1.001 and 1.006 of it on pools of 160 and 32 prompts. An earlier version of this work claimed the opposite, that a repeated prompt biases τ_b down; it does not, and what we had measured was the bias of the other scheme.

A finite corpus makes blocks anticorrelated. Drawing *without* replacement from a corpus of N is what needs correcting: two blocks hold distinct prompts, so

$$\mathbb{E}\big[g_{a1}^\top M\, g_{a'2}\big]_{a\neq a'} \;=\; \bar{\boldsymbol{g}}^\top M \bar{\boldsymbol{g}} \;-\; \frac{\operatorname{tr}(M\Sigma_{\mathrm{b}})}{N-1}, \tag{23}$$

which is exact. Combining it with the same-block expectation gives $W - A = N\,\operatorname{tr}(M\Sigma_{\mathrm{b}})/\big(n(N-1)\big)$ for blocks of n prompts, so the raw estimator $n(W-A)$ overstates τ_b by $N/(N-1)$ and the unbiased form carries the extra factor $(N-1)/N$. On a corpus of 32 that is a 3% correction, and the uncorrected estimator returns 1.031 of the exact value where the corrected one returns 1.00. The signal is then recovered as $A + \tau_b/(N-1)$, which matters whenever τ_b is large relative to it, that is whenever the run is far below its critical batch size. Uncorrected, the signal estimate on one pool came out negative. The same factor appears in the theory: with a finite corpus the between-prompt variance enters $\operatorname{Cov}(\hat{\boldsymbol{g}})$ multiplied by $1-(P-1)/(N-1)$. It cannot simply be substituted into the unbounded optimum, because the rollout budget fixes $R = PG$, so $P = R/G$ moves with the very quantity being chosen and the factor is a function of it. Minimising $G\,B_{\mathrm{crit}}$ at fixed R gives

$$\begin{aligned} &\frac{\partial}{\partial G}\left[\frac{(GN-R)\,\tau_b}{N-1} + \frac{G\,\tau_w}{G-1}\right] = 0 \\ &\qquad\Longrightarrow\quad G^\star \;=\; 1 + \sqrt{\frac{(N-1)\,\tau_w}{N\,\tau_b}}, \end{aligned} \tag{24}$$

subject to $P \le N$, that is $G \ge R/N$, and in practice minimised over the group sizes the budget admits. A finite corpus therefore lowers the optimum, by the factor $\sqrt{(N-1)/N}$, rather than raising it: prompts drawn without replacement carry slightly less independent information than the unbounded count suggests, so slightly fewer of them are worth buying. The effect is also small: the factor $\sqrt{(N-1)/N}$ is under half a percent at $N = 128$. What was not small is the error it replaces. Evaluating the correction once at the reference split $P = R/8$ divides τ_b by 0.756 rather than multiplying it by 0.996, and reports $G^\star = 3.38$ where the fixed-budget optimum is 3.06. Table 16 carries the corrected values.

What the forecast needs is simpler than this. Equations (22) recover two traces of Σ_t against a metric. Theorem 2 needs less: a single scalar $b^\top \Sigma_t b$ per update, which is the sample variance across prompts of the per-prompt gradient projected onto b. A trainer accumulating microbatches already holds those per-prompt gradients for the length of one step; projecting each onto a fixed vector and taking their variance

costs one dot product per microbatch and no extra memory beyond b itself. The machinery above is what the batch-allocation results of Section 8 need, and is more than the error bar requires.

A closed form with no run at all. Take (12) with $\Phi_{T,t} \approx I$ and a constant step, replace the projection $\mathrm{Var}_i(\nabla M^\top \hat{g}_{t,i})$ by the trace of the injected covariance, and the run drops out entirely:

$$\sigma \;\propto\; \eta\sqrt{T/P}\,\sqrt{\tau_b + \tau_w/(G-1)}. \tag{25}$$

We call this the *frozen* forecast, because it needs only the run length, the batch shape and the two noise traces, and is therefore available before any training happens.

It is written as a proportionality on purpose. The constant it is missing carries $\|\nabla M\|$, which is exactly the part that requires a trained model, and on the grid of Section 6.1 (25) overstates the measured deviation by a factor of about 84. That factor is not quite constant: it varies by 4.8× across the grid, leaving a relative error of 1.26× at the median, and 2.43× at worst, once one common scale is removed. So (25) is useless for reporting an error bar, and by the argument of Section 7 entirely serviceable for sizing one, because a stopping threshold absorbs a constant and nothing absorbs the part that varies. That distinction is not a technicality here: it is why the cheapest predictor in this paper is also the one we would recommend deploying first, and why we separate the two uses rather than reporting a single accuracy number.

Preconditioners. An update is $\eta\,\Pi\,\hat{\boldsymbol{g}}$ for a preconditioner Π, and Adam is scale invariant, so both the drift model and the estimators must be read in the metric the update actually moves in. In practice this means instrumenting the preconditioned gradient: the results carry over with $F \mapsto \Pi F \Pi$ under the standard assumption that Π varies slowly relative to batch-to-batch fluctuation.

Controlling drift. Since $D \propto \eta^2$, a multiplicative controller

$$\eta_{t+1} \;=\; \eta_t \cdot \mathrm{clip}\Big(\big(D^*/D_t\big)^{1/2},\; c^{-1},\; c\Big) \tag{26}$$

corrects a mis-set step in a single update when the drift estimate is clean. Measured over runs on transformers, it holds the median drift at its target with no detectable bias in log space. Exponential smoothing of D_t, the obvious refinement, makes it worse: the lag biases the realised drift low by roughly a factor of two.

grouped by	settings	median	worst	inside 95%
all settings	192	1.10×	1.56×	162/192
policy 3^3	96	1.10×	1.56×	82/96
policy 4^2	96	1.10×	1.38×	80/96
$T = 10$	96	1.10×	1.34×	86/96
$T = 25$	96	1.11×	1.56×	76/96
$P = 8$	96	1.12×	1.56×	78/96
$P = 32$	96	1.08×	1.53×	84/96
$\eta = 0.5$	96	1.10×	1.29×	92/96
$\eta = 1.5$	96	1.13×	1.56×	70/96

Table 2: Forecast against realised spread. The error is $|\log(\text{predicted}/\text{measured})|$ exponentiated; the last column counts settings where the forecast falls inside the bootstrap interval of the measured standard deviation over 47 runs, which is consistency with that interval rather than the coverage of an interval the forecast itself produces.

6 Experiments

Four levels of evidence, in decreasing order of what can be computed and increasing order of what a practitioner would recognise. Policies small enough to enumerate, where the objective, the Fisher information, the drift and both noise terms are computed rather than estimated, and where the forecast can be checked against the quantity it predicts rather than against an estimate of it. Transformers trained from scratch, where a population of independent runs executes as one batched kernel so that sweeps carry standard errors rather than anecdotes. A pretrained model of half a billion parameters. And Qwen2.5-7B, eight seeds, which is what this machine holds. Everything ran on a single Apple M4 laptop with 24 GB of memory.

6.1 Forecasting the spread of a reported number

This is the experiment the paper is for, and its protocol is prospective by construction. In each of 192 settings one run is trained and frozen; the states it passed through are carried backwards through (11) to produce a forecast of the across-seed standard deviation of the final pass rate; the forecast is recorded; and only then are 47 further seeds trained and their spread measured, without the anchor. Nothing about the outcome is available when the prediction is made.

The enumerable policy is used because there the reported number is the exact pass rate rather than an estimate of it, so the measured spread is the spread of the quantity itself and not of a finite evaluation set. Settings vary two policy shapes (a vocabulary of three over three positions, and of four over two), two prompt-pool diversities, three difficulty bands, the run length T, the prompt count P and the step size η, which between them move the realised spread over a factor of 47.

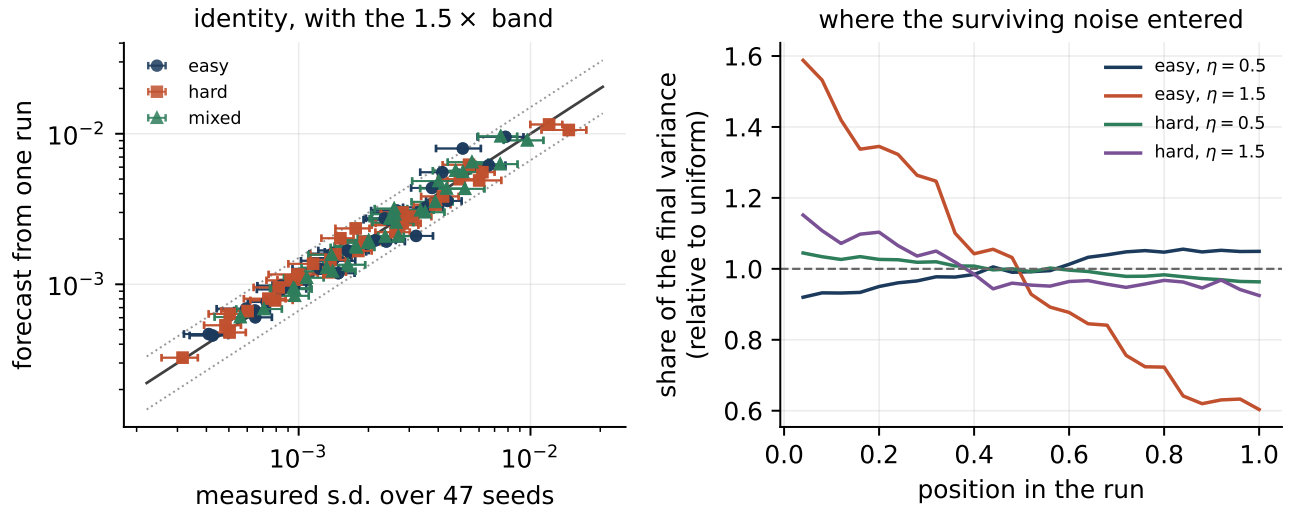


Figure 2: Left: the forecast made from one run against the standard deviation realised over 47. Horizontal bars are bootstrap intervals over runs. The solid line is equality and the dotted lines are a factor of 1.5. **Right:** the share of the final variance injected at each point in the run, normalised so that a flat line at 1 is a uniform kernel. It is nearly flat, which is the answer to the memory-horizon question and not the one we expected.

Table 2 reports a median absolute error of 1.10× and a worst case of 1.56×, and the forecast lands inside the measured 95% interval in 162 of 192 settings. The grid uses a pool of 24 prompts; the same 192 settings on a pool of 16, run as a replicate, give 1.10×, 1.50× and 154 of 192. That last count says the point forecast is consistent with the interval a finite number of seeds puts on the measured spread. It is not the coverage of an interval the forecast itself produces, which is a quantity this method does not yet report. Two qualifications belong next to the headline. The measured deviation is an estimate from 47 seeds, and a forecast equal to the true deviation would itself score 1.07× against it; that sampling error accounts for about 51% of the variance of the log error in Table 2. And the runs of this grid gain a median of 0.009 in pass rate and at most 0.036, so it tests the forecast where a run barely moves. Section 6.5 tests it where runs learn, and there it does not hold. Two things in the breakdown are worth noting. The two policy shapes agree with each other to within a hundredth at the median (Table 2), so the result is not a property of one tabular family. And accuracy is flat in run length at the median but that consistency is not: the forecast lands inside the measured interval far more often at $\eta = 0.5$ than at $\eta = 1.5$ (Table 2), which is the linearisation showing up exactly where Section 6.4 says it should.

To make the deliverable concrete: one setting in the middle of the grid, at twenty-five updates, thirty-two prompts, groups of eight and mixed difficulty, reports a pass rate of 0.502. The forecast made from that single run is a standard deviation of 1.11×10^{-3}; the value realised over 47 independent runs trained after it is 1.08×10^{-3}, with a bootstrap interval of $[8.6 \times 10^{-4}, 1.3 \times 10^{-3}]$. What a reader would be handed is 0.502 ± 0.002 from the seeds, before the evaluation's own term is added.

	median		worst	
settings	carried	held	carried	held
all 192	1.10	1.11	1.56	1.75
$T = 10$	1.10	1.10	1.34	1.29
$T = 25$	1.11	1.11	1.56	1.75
$\eta = 0.5$	1.10	1.09	1.29	1.32
$\eta = 1.5$	1.13	1.14	1.56	1.75

Table 3: Forecast error with the adjoint carried back against held at ∇M. Carrying it changes the forecast by a median of 2% and buys only the worst case, and only where the run is long and the step large.

What the forecast costs is two gradient evaluations per update for the backward pass and one scalar per update for the injected term. It never forms S_T, and by Proposition 1 it never needs a transpose.

What the backward pass is worth. The expensive part of Algorithm 1 is carrying the adjoint back, two gradient evaluations per update. The ablation that says whether it earns them is to hold b at $\nabla M(\theta_T)$ throughout, keeping the injected term and discarding everything the trajectory does to it.

It does not, at these horizons (Table 3). The two agree to a median of 2% in the forecast itself (held over carried, 0.72 to 1.05), the median error is 1.10× carried against 1.11× held, and they land inside the measured interval in 162 and 158 settings. What carrying it back buys is the tail: at $T = 25$ and $\eta = 1.5$ the worst case is 1.56× carried against 1.75× held.

This is not a defect in the theory; it is the theory

estimator	settings	median	worst	inside 95%
RLOO	192	1.11×	1.58×	162/192
GRPO, mean baseline	12	1.10×	1.27×	10/12
GRPO, standardised	12	1.10×	1.28×	8/12

Table 4: The forecast under each estimator in the family. The RLOO row is the full grid; the two GRPO rows are a common sub-grid of twelve settings at matched difficulty bands, run lengths and group sizes. Those sub-grids predate storing per-seed scores, so the anchor run is counted among the seeds here, for RLOO too, which is why its row differs slightly from Table 2.

being informative about its own cost. Section 6.3 measures most of the spectrum of A_t sitting within 10^{-4} of one, and ∇M points into the slow part of it, so over tens of updates $\Phi_{T,t}$ is close to the identity along that one direction. The practical reading is that a run of this length should use the cheap estimator, which costs one extra pass per update, needs no Hessian-vector products and does not depend on the noisy directional derivative Section 6.9 reports. The recursion is what tells you when to stop: when the spectrum of A_t moves away from one, or the run is long enough that a 1% per-update discrepancy compounds.

The whole estimator family, not one of them. The grid above uses RLOO. Repeating a common sub-grid under the mean-baseline and standardised GRPO estimators gives the same picture (Table 4). This matters more than a robustness check usually would, because Proposition 1 is what makes the backward pass affordable and it is a claim about the whole count-based family: if it held only for the estimator whose difficulty weight is constant, the result would be a curiosity rather than a method.

Where the uncertainty comes from. Restricting Σ_t to Σ_b or to Σ_w splits the same forecast by origin. The two shares are exact and sum to one, and across 288 settings the rollout share has a median of 78%: over the grid as a whole, which responses were sampled matters more than which prompts were drawn. It is monotone in group size, running 93% at $G = 2$, 82% at $G = 4$, 65% at $G = 8$ and 46% at $G = 16$, so the majority changes hands somewhere between groups of eight and sixteen rather than rollout sampling dominating throughout. With prompts drawn independently, Σ_b and Σ_w are both divided by P and the mix is independent of the number of prompts: adding prompts buys a smaller error bar, only larger groups change what the error bar is made of. That equality is a property of independent sampling. Drawing distinct prompts from a corpus of N multiplies Σ_b alone by $1-(P-1)/(N-1)$, so a run that covers much of its corpus every step does shift the mix towards rollout sampling as P grows.

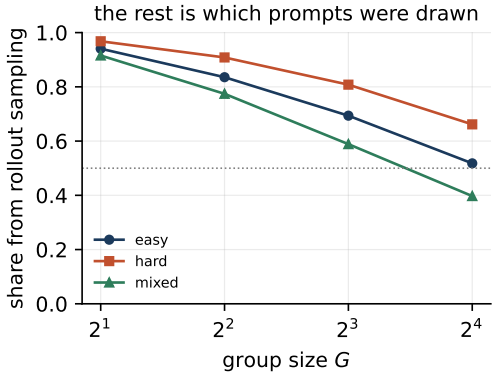


Figure 3: The share of the forecast variance attributable to rollout sampling rather than prompt selection, against group size. Each point is the median over run lengths and prompt counts, which do not move it.

The uncertainty is made early, not late. If the filter were sharp, the kernel k_t of (9) would concentrate near the end of the run and reproducing a result would mean reproducing only its tail. We looked for that and found the opposite. Over 288 settings, the number of final updates holding 95% of the forecast variance is a median of 1.00 of the run against 0.95 for an exactly uniform kernel, and it does not move when the step size is raised sixteen-fold or when the run is started from a policy already trained for 1200 updates.

Where the kernel is not flat it tilts the other way. In 254 of 288 settings more of the final variance was injected in the first half of the run than in the second, with a median first-half share of 55% (Figure 2, right). The mechanism follows from the two measurements either side of it: with $\Phi_{T,t}$ close to the identity along ∇M, $k_t \propto \eta_t^2 \Sigma_t / P$, and Σ_t is largest early because $\mathbb{E}_x[p(1-p)]$ is largest before the policy has learned anything. Early noise is both bigger and no more forgotten than late noise.

The practical reading is the reverse of the one we went looking for. Reproducing a run means reproducing all of it, and if anything the beginning matters more, which is also where a larger batch would buy the most.

6.2 Why the forecast rests on filtering, not accumulation

The recursion predicts divergence; a single timescale does not. The enumerable policy admits three things at once: the exact linearised recursion (6), the exact Fisher information, and as many independent stochastic trajectories as we care to run. That allows candidate models of seed divergence to be scored against ground truth rather than against each other. Over 54 settings spanning prompt diversity, group size, batch size and step size, we compare the propagated co-

model	all	$d = 0.15$	$d = 1$	$d = 2.5$	worst
propagated, (7)	1.03	1.04	1.02	1.02	1.2
accumulation	1.06	2.06	1.03	1.03	3.8
single timescale	57.93	3.24	88.88	88.88	791.5

Table 5: Predicted against measured divergence between independent runs in the enumerable policy, with 400 trajectories per setting. Error is the ratio of predicted to measured, summarised as exp of the median absolute log ratio. d is the diversity of the prompt pool: low d means the prompts are alike, which is where the objective supplies the most curvature.

variance against two alternatives: accumulation with no contraction, and the single-timescale law an earlier version of this work relied on.

The propagated recursion is accurate to 3% at the median and 1.2× at worst, everywhere in the grid. The interesting column is not the first one.

Accumulation, the model that says noise is added rather than filtered, is wrong by a factor of 2.06 on homogeneous prompt pools and by 3% on heterogeneous ones, with a worst case of 3.8×. That is not a caveat on the result; it is the result. Filtering is done by the curvature of the estimator potential, and a pool of similar prompts is where that curvature is consistent enough to matter. Where the prompts disagree with each other the mean field is nearly flat, A_t is nearly the identity, and a run genuinely does accumulate its noise. A reader who trains on a narrow task distribution is in the first column; a reader who trains on a broad one is in the second, and for them the simpler model is fine.

The single-timescale law is wrong by two orders of magnitude wherever there is a spectrum to compress, and its one respectable column (3.24× at $d = 0.15$) is the case where contraction is strong enough that one rate almost describes it. Section 6.3 measures the spectrum directly and says why the general case is hopeless for it.

The filtering requires a learning signal. If divergence stops growing because the shared objective pulls two runs together, then removing the objective should restore accumulation. We ran a population of transformers twice from identical warmed parameters: once normally, and once with every reward replaced by an independent coin flip. The architecture, the initial parameters, the prompt draws, the rollout count, the group size, the step size and the estimator are all held fixed; what the coin destroys is the association between a response and whether it was correct.

We are careful about what that controls for. Replacing the rewards changes which responses receive positive advantage, so it changes Σ_t as well as the mean field, and the two conditions do not have identical injected covariance. What is held fixed is the

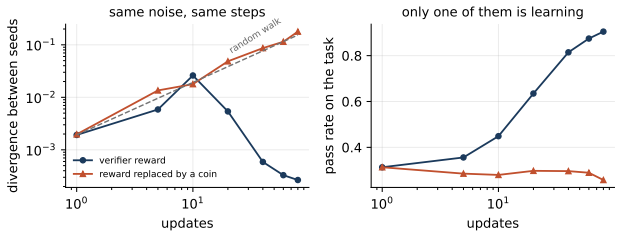


Figure 4: The same run with and without a learning signal. Left, divergence between seeds against the slope a random walk requires; right, pass rate on the actual task, which is what the coin condition is not optimising.

	learning		signal removed	
updates	diverg.	pass	diverg.	pass
1	1.92×10^{-3}	0.313	1.97×10^{-3}	0.313
5	5.87×10^{-3}	0.356	1.35×10^{-2}	0.285
10	2.61×10^{-2}	0.449	1.78×10^{-2}	0.280
20	5.39×10^{-3}	0.635	4.85×10^{-2}	0.298
40	5.88×10^{-4}	0.814	8.65×10^{-2}	0.296
60	3.29×10^{-4}	0.875	1.14×10^{-1}	0.289
80	2.66×10^{-4}	0.905	1.78×10^{-1}	0.258

Table 6: Divergence between runs that differ only in rollout randomness, with and without a learning signal. Same sampling protocol, same batches, same step sizes, with the association between a response and whether it was correct removed; the injected covariance is not held identical. The pass rate column is the verifier's in both conditions, so the coin column shows what the run is actually achieving while it optimises noise.

sampling protocol and the batch composition, and the realised step sizes are comparable rather than identical. The experiment therefore shows that destroying the reward-response association restores accumulation, which is strong evidence that the mean dynamics do the filtering. It is not the stronger statement that the filtering persists at exactly matched noise; establishing that needs a null whose updates are drawn as $(\hat{\boldsymbol{g}}^{(1)} - \hat{\boldsymbol{g}}^{(2)})/\sqrt{2}$, which has zero mean and the covariance of the gradient itself, and we have not run it.

The two conditions separate completely. With a signal, divergence rises for ten updates, turns over, and ends at 0.14 of where it started while the pass rate climbs from 0.31 to 0.90. Without one it grows by a factor of 90.5 over the same 80 updates and the pass rate does not move. At the end of the run the two differ by a factor of 671.

The control that matters is the step size. Mean realised drift per update is 6.10×10^{-3} in the learning condition against 5.17×10^{-3} in the null: the runs that end up closer together are the ones that moved *further* per update. Whatever brings them back is not smaller steps.

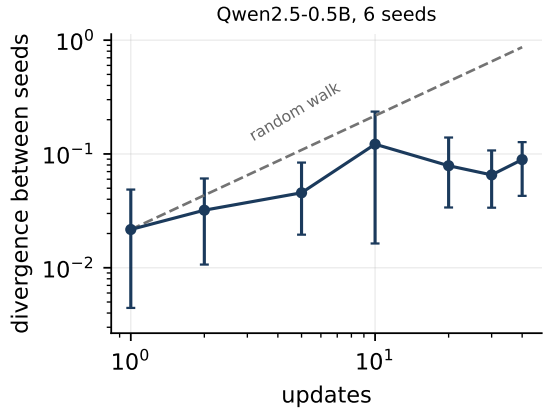


Figure 5: Divergence between six runs of a pretrained model differing only in rollout randomness. Intervals are 95% bootstrap intervals resampled over runs rather than over pairs, since the fifteen pairwise distances are built from six runs. They are wide enough that the shape of the trace is not resolved; what is resolved is that the trace is not the dashed line.

On a pretrained model. The transformers above are small and their task is synthetic. We repeated the measurement on Qwen2.5-0.5B-Instruct with LoRA adapters, six runs differing only in rollout randomness on exactly verifiable short-answer problems; because LoRA adapters start at zero the runs begin from an identical policy by construction, and every checkpoint is scored on one fixed batch of sequences.

Divergence reaches 1.22×10^{-1} at ten updates and is 8.9×10^{-2} at forty, with a tail slope of $+0.14$ against the $+1$ accumulation requires. Two caveats belong with that number. The fifteen pairwise distances come from six runs and are not independent, so intervals are resampled over runs; doing so roughly doubles the standard error against treating the pairs as independent. And once that is done the intervals at ten and forty updates overlap heavily, so the apparent peak-and-fall is not resolved at six seeds. What the experiment supports is the weaker statement that divergence does not accumulate on a pretrained model either. The released record is a rescoring of the stored checkpoints and does not carry the runs' training pass rates, so we quote none.

Bounded, not proven stationary. Across the settings in which a run is still learning, the log-log slope of divergence against update count runs from -1.15 to $+0.26$, against the $+1$ that accumulation requires. That rejects accumulation. It does not establish stationarity in the strict sense: $\mathcal{J}_t$, Σ_t and η_t all move during training, so the level that (7) converges to is itself slowly varying, and we describe the divergence as bounded rather than stationary throughout.

update	0	25	50	100	200	400	600
easy, $10^3(\rho-1)$	19.9	18.6	17.6	7.8	0.8	0.2	0.1
contracting, %	33	36	38	38	49	41	41
hard, $10^3(\rho-1)$	10.9	10.5	10.1	9.4	1.2	0.2	0.1
contracting, %	31	33	36	36	44	49	49
mixed, $10^3(\rho-1)$	11.6	10.0	10.9	10.3	0.9	0.1	0.1
contracting, %	36	38	44	41	49	46	51

Table 7: The spectrum of the transfer operator along a run of 600 updates at $\eta = 1$, reported as $10^3(\rho - 1)$ for the spectral radius ρ. The operator expands in its leading direction throughout, and contracts in a minority of directions that grows as the run proceeds.

How the level scales. Compared at matched pass rate rather than matched step count, a log-log fit over 9 settings gives an exponent of -0.61 in the batch, 95% CI $[-1.42, +0.24]$, and $+0.59$ in the drift target, 95% CI $[+0.27, +1.04]$, with $R^2 = 0.70$. Both intervals contain the values $-1/2$ and $+1/2$ that follow from $Q_t \propto \eta^2 \Sigma / P$, and the batch interval also contains zero. Nine settings are too few to pin a two-dimensional scaling law, and we report the intervals rather than the point estimates for that reason.

Why the route through policy divergence was abandoned. An earlier version of this work predicted that the spread in final task performance would follow $\sqrt{\mathrm{KL}_\infty}$. It does not, in our data: the fitted exponent is $+0.10$ with 95% CI $[-0.03, +0.45]$ against a predicted $+0.5$, and the interval contains zero. Policy divergence and outcome spread are positively associated across our settings, but these experiments do not establish the predicted relation, and we withdraw it.

The reason is structural rather than statistical, and Section 6.13 shows it in its extreme form. KL weights every direction the policy can move in by the Fisher information; a task metric weights them by ∇M, which is a single direction and can be nearly orthogonal to where the variance is, or, on a saturated evaluation, zero. No scalar function of KL can recover that. Theorem 2 takes the direct route instead, and Section 6.1 is what it delivers.

6.3 The filter is not a contraction

It is convenient to say that the updates following a perturbation shrink it. Measuring the spectrum of $A_t = I + \eta \mathcal{J}_t$ along a run says something more careful, and the more careful statement is the one the paper should make.

Three things are visible in Table 7 and none of them is a uniform contraction.

The spectral radius exceeds one everywhere we look, by 1–2% early and decaying towards 1.0001 as the policy improves. There are directions in which a perturbation grows. Over a run of twenty-five updates

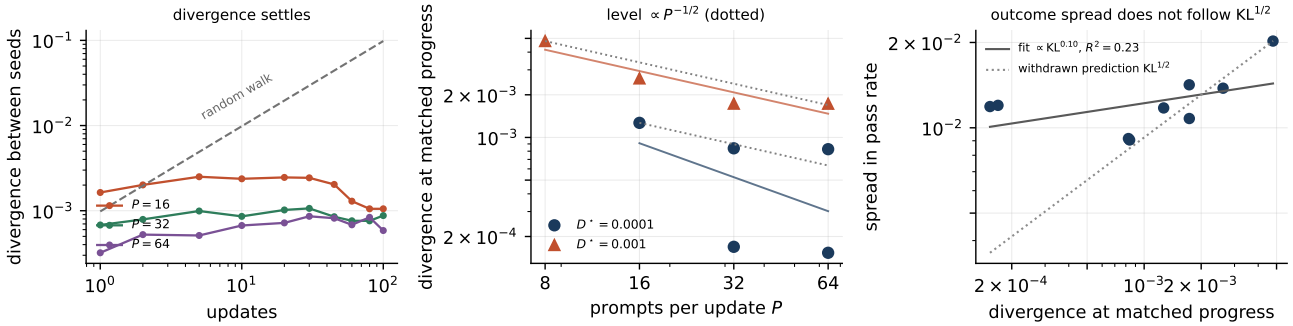


Figure 6: Left: divergence between seeds against update count on transformers trained from scratch, with the slope a random walk would require. It settles instead. **Centre:** the level against the number of prompts, compared at matched pass rate rather than matched step count, one line per drift target. A joint fit gives exponents of -0.61 in the batch and $+0.59$ in the drift target against the $-1/2$ and $+1/2$ that $Q_t \propto \eta^2\Sigma/P$ requires; both intervals contain those values and the first also contains zero. **Right:** the spread in pass rate against that divergence. Its fitted exponent is $+0.10$ against the $1/2$ an earlier version predicted, a relation we withdraw.

that growth is a factor of $1.0199^{25} \approx 1.6$, which is why divergence stays bounded over the horizons anyone trains and why it would not stay bounded forever: what makes the spread finite here is that expansion is slow relative to run length, not that every direction shrinks.

The share of directions that contract at all is between 31% and 51%, and it rises as the run proceeds, from 36% to 51% on the mixed pool. The filter changes shape during training. This is the non-stationarity that Theorem 1 handles by construction, since it carries A_t rather than an average of it.

Among the directions that do contract, the fastest has an eigenvalue of 0.979 and the slowest is 1.0000 to four places. That is a range of decay timescales from roughly fifty updates to effectively unbounded, in the same operator, at the same moment. It is the direct explanation of two results reported above: why replacing the spectrum by one effective timescale is wrong by two orders of magnitude, and why the per-update kernel has no horizon: the slow end of the spectrum has not forgotten anything by the end of an eighty-update run.

We therefore say that noise is *filtered* rather than contracted, and mean by it that (7) reweights each update's contribution by a spectrum straddling one, most of which is very close to it.

6.4 How far the linearisation can be pushed

Everything above linearises the mean update field between the two runs. That assumption is checkable without knowing the measured spread, though it needs a pair of runs rather than one. For a pair of runs at θ^A and θ^B with $\Delta = \theta^B - \theta^A$, define the residual

$$\varepsilon \;:=\; \frac{\big\| \bar{\boldsymbol{g}}(\theta^B) - \bar{\boldsymbol{g}}(\theta^A) - \mathcal{J}(\theta^A)\Delta \big\|}{\big\| \bar{\boldsymbol{g}}(\theta^B) - \bar{\boldsymbol{g}}(\theta^A) \big\|}, \qquad (27)$$

zero where the field is linear between them. We measure its median over the updates of one pair of runs in each of 36 settings and report the forecast error against it.

The residual behaves as it should: raising the step size eightfold raises it from 0.028 to 0.165. The forecast does not follow. Its median error is 1.11× where the residual is below 5%, 1.18× between 5 and 12%, 1.08× between 12 and 25%, and 1.18× above that, with a rank correlation of 0.17 between residual and error across the grid. What does track the residual is the tail: the worst case is 1.21× below 5% and 1.57× above it.

Two things follow. The derivation asks for more than the result needs, since a 16% departure from linearity costs nothing at the median, so the honest statement of the assumption is that it bounds the worst case rather than setting the accuracy. And ε is worth logging anyway, because it is the quantity that separates the regime where the forecast has a worst case of 1.21× from the regime where it has one of 1.57×, and computing it costs one extra gradient evaluation per checkpoint once a second run exists. Making it available before that second run is what Section 10 lists first among the things we would build next.

Both statements are about step sizes up to 8 times the smallest, at which these runs still gain only a few hundredths in pass rate. The next subsection goes further, and there the first of them stops being true.

6.5 When the run learns

The settings behind Figure 2 have something in common that the sweep above did not push far enough to

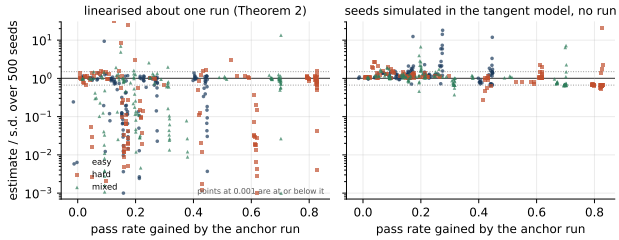


Figure 7: Estimate over measured deviation against the pass rate the anchor run gained, on 384 enumerable settings with exact inputs, each graded against 500 seeds trained afterwards. **Left:** linear response about one run. It holds while the run barely moves and falls short by orders of magnitude once the policy saturates. **Right:** seeds simulated in the tangent model at the starting policy, with no run trained. Dotted lines are a factor of 1.5.

expose: their runs barely learn. The run each forecast was made from gains a median of 0.009 in pass rate and at most 0.036 (0.069 on the replicate grid). The pretrained runs of Section 6.10 gain between 0.03 and 0.28 in twelve updates. The tier on which the forecast is validated and the tier on which it fails therefore differ in how far the policy moves as well as in everything else, and nothing above separates the two.

So we raised the step size on the enumerable tier until its runs moved that far and further, with every input still exact: 384 settings on an environment used nowhere else, two shapes, two pool diversities, three bands, two group sizes, 8 and 32 prompts, 12 and 50 updates, step sizes from 1.5 to 60. Runs gain up to 0.83. Three estimates of the across-seed deviation are made in each setting before the seeds they are graded against exist: the forecast of Theorem 2 from one anchor run; the same with the adjoint held at ∇M; and the spread of 2000 seeds simulated in the tangent model of Section 3.5 at the starting policy, which uses no run. A setting is *learning* if its anchor gains at least 0.05 in pass rate; 354 are. The grid was run twice, on two environments, and each time the settings, the thresholds and the predictions were committed first (`docs/predictions.md`). We report the second run, graded against 500 seeds per setting, and then what the first, graded against 47, got wrong.

Linear response about one run fails there, with exact inputs. On the learning settings the forecast of Theorem 2 is within 1.5× of the measured deviation in 44.4% and more than 3× off in 44.1%, with a median error of 1.80× and a worst case of 1715×. Of the 156 settings beyond 3× it is too small in 149 and too large in 7. The mechanism is the one Section 6.9 found on a pretrained model. A policy that answers every prompt of this tier has a best single sequence, large steps drive a run onto one, and which one differs by seed. At a run that has arrived, $\nabla M(\theta_T)$ is zero and so is the forecast, while the realised spread is the choice of sequence. Holding the adjoint fixed is no better (median 2.72×). No estimate enters any of this. The shortfalls on pretrained models therefore need neither estimation noise, nor Adam, nor anything specific to a network: linearising about one run that has saturated is sufficient to produce them, and the 1.10× of Section 6.1 is a statement about runs that gain at most 0.036.

Simulated seeds survive it. The spread of seeds simulated in the tangent model has a median error of 1.27× on the same settings, is within 1.5× in 74.9% and beyond 3× in 4.8% (17 settings, 16 of them too large), with a worst case of 56×. Where runs barely move it costs nothing: on the 15 settings gaining less than 0.02 its median error is 1.04× against 1.04× for the forecast. It was made before any run was trained.

A useful scale for those numbers is what replication buys. Take the deviation over the first n of the 500 seeds as an estimate of the deviation over all of them. Ten seeds are within 1.5× in 73.7% of the learning settings and beyond 3× in 11.3%; five seeds in 55.6% and 18.4%; three in 44.9% and 28.5%. On this tier, then, simulating seeds from the starting policy is about as accurate as training ten, and is badly wrong less than half as often; the forecast from one run sits with three seeds. Forty-seven seeds do better than either, at 91.5%.

What we predicted, and what failed. Of the six parts recorded across the two grids, two held and four failed, three of those on a single clause. On the first grid we predicted a median error of at least 2× for the forecast on learning settings and found 1.59×: it fails in its tail, with 30% of settings beyond 5×, and not at its median. We predicted no simulated spread beyond 3× and found 25 of 354. And on the second grid we predicted at least 75% of simulated spreads within 1.5× and found 265 of 354, one setting short.

The second of those is why there is a second grid. All 25 misses were settings where the simulated seeds spread *more* than 47 real ones that nearly agreed. Training 1500 real seeds in each showed that in 23 of them a few runs in a hundred, or in a thousand, end somewhere else, that those runs carry most of the variance, and that 47 seeds had not contained one. The simulated seeds had. That was found by inspecting failures, so we recorded a test of it before running the second grid: that the first 47 of 500 seeds would understate the deviation by more than half in at least a tenth of learning settings. They do so in 5.1%, by up to 92×. The rare outcomes are real and rarer than the settings we had picked suggested, and that prediction failed too.

What this does not show. The enumerable tier favours the tangent model in a way a network will not. A tabular tree policy and its tangent model span the same family of distributions, so the only error in a simulated seed is that the scores are frozen where the reference was taken; a transformer adds a second error, in the model itself, that this tier cannot see. Its settled runs are also not all right: in 16 settings the simulated seeds spread over outcomes that 500 real seeds do not contain. The test that matters is the same comparison on a pretrained model, against seeds already on disk. It is recorded in `docs/predictions.md` with its thresholds, and it had not been run when this version was written.

6.6 The exact moments the forecast is built on

Every term the forecast injects comes from Propositions 3 and 4, so those are checked first. The policy is a tabular autoregressive model over a small vocabulary, so its distribution over sequences, its score at every sequence, and the conditional moments (p, u_1, S_0, S_1) are available exactly. Propositions 3 and 4 were checked against brute-force enumeration over all $|\mathcal{Y}|^G$ group outcomes for five estimators and $G \in \{2, 3, 4\}$: agreement to 10^{-12}. The scaled within-prompt term $(G-1)\,\mathrm{tr}\,\Sigma_w$, which the theory treats as G-independent, varies by 6% over $G = 2$ to 32.

6.7 Recovering the noise terms from microbatches

The forecast needs one scalar per update, but the batch-allocation results of Section 8 need the two traces separately, and the same estimator supplies both. Figure 8 compares the group size recovered by the estimator of Section 5 against the exactly known value, over pools whose noise ratio spans a factor of 26 and whose $G^\star$ runs from 3.8 to 15.2. The median error in $G^\star$ is 1.3% and the worst is 6.1%, using 2500 batches of 32 prompts split into $K = 8$ blocks; the individual traces are recovered to 2.1% (τ_w) and 3.2% (τ_b) at the median.

6.8 When the optimiser carries state

Adam's update depends on two moment estimates as well as the parameters, so a gradient perturbation outlives the step that produced it and θ alone is no longer a closed state. The recursion lifts: with $z = (\theta, m, v)$ and Φ_z the mean Adam map, the transfer operator is $\partial\Phi_z/\partial z$ and the injection is $B\,\Sigma_t\,B^\top$ with $B = \partial\Phi_z/\partial\hat{g}$, the sensitivity of one update to the gradient it was handed. Both are taken by central differences on the exact mean map, so nothing about the derivation has to be trusted.

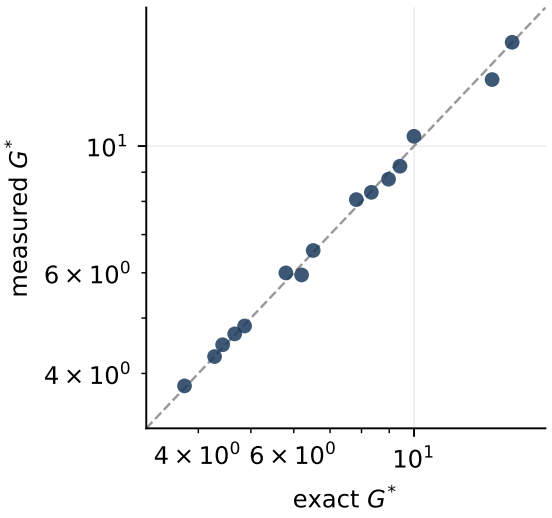


Figure 8: Measured against exact optimal group size across prompt pools.

model	median	worst	live signal only
lifted, $z = (\theta, m, v)$	1.10×	1.8×	1.13×
second moment frozen	1.21×	1.6×	1.23×
θ block only	24.60×	73.7×	27.31×
accumulation	24.22×	73.8×	26.89×

Table 8: Predicted against measured seed divergence under Adam, over 24 settings. The parameter-only row uses the same operators restricted to their θ block, so it isolates the cost of pretending the optimiser state does not carry a perturbation forward.

One caveat is structural. Adam's map is singular at $v = 0$: the derivative of $1/(\sqrt{v} + \epsilon)$ diverges there, and a recursion started at initialisation is not linearisable at all. The sweep above therefore warms the moments on mean gradients for five updates before propagating, and every seed shares the warmed state, so the covariance still starts at zero. That is a choice the pretrained runs of Section 6.12 did not make: they use MLX's Adam from a zero state and without bias correction, where the first update is close to $\eta\,\mathrm{sign}(\hat{g})\sqrt{(1-\beta_2)^{-1}}(1-\beta_1)$ and its derivative with respect to the gradient nearly vanishes. Rerunning this sweep in that regime (`--warmup 0 --no-bias-correction`) takes the lifted forecast from a median error of 1.10× to 2.40×, with a worst case of 549× in either direction; starting cold but with bias correction gives 1.34× and a worst of 2.90×. The lift is validated only for an optimiser whose state has been warmed.

The optimiser state is where almost all of the difficulty lives. Ignoring it underpredicts by 24.6× at the median, which is barely better than assuming noise

accumulates without any filtering at all (24.2×): a recursion on θ alone cannot see that a perturbation to one gradient is still moving the parameters twenty updates later through m and v. Carrying the full state brings that to 1.10× at the median and 1.80× at worst, and freezing the second moment at its mean trajectory rather than propagating it costs little, 1.21×.

What matters more is that the lift no longer degrades where the metric saturates. Splitting by pool, the median error is 1.06× on the hard band, 1.13× on the intermediate band where the learning signal is live, and 1.10× on the easy band where the pass rate reaches 0.74 and $\mathbb{E}_x[p(1-p)]$ is collapsing. The worst case on that easy band is 1.8×.

An earlier version of this work reported the opposite, a median of 8.0× on the easy band and a worst case two orders of magnitude out, and attributed it to Adam's normalisation being a nonlinearity that a linear recursion cannot contain. That diagnosis was wrong, and the cause was ours. Adam is handed a stochastic gradient, so the mean of its second-moment update is $\beta_2 v + (1-\beta_2)\mathbb{E}[\hat{g}^2]$ with $\mathbb{E}[\hat{g}^2] = \bar{\boldsymbol{g}}^2 + \operatorname{diag}\Sigma$. Squaring the mean gradient instead, which is what we did, discards the gradient variance. Along the mean trajectories of this grid that variance is between 1.3 and 16.5 times the term that was kept (the median over updates in each setting), and at least 5.1 times at eight prompts per update, so v was far too small along the whole mean trajectory and every step taken from it was correspondingly too large. The error was largest exactly where $\bar{\boldsymbol{g}} \to 0$ and the variance is all that remains, which is why it looked like a saturation effect. Correcting the mean map removes it.

The practical statement is therefore stronger than we could previously make: the recursion survives an optimiser that carries state, at the same accuracy it has under plain ascent, provided the mean map is the mean of the stochastic map and not the map evaluated at the mean gradient.

6.9 The backward pass on a pretrained model

We ran the forecast end to end on Qwen2.5-0.5B-Instruct with LoRA adapters, twelve updates of plain gradient ascent, which is the update the recursion is written for, on a task the base model passes at 0.16, and afterwards trained seven more seeds and measured what they did. The backward pass is the one in Algorithm 1, with the mean update field estimated from fresh rollouts under common random numbers.

Two diagnostics came back before the comparison did, and both matter more than the comparison.

The first is that the run stops injecting almost immediately. The fraction of prompts whose group is not unanimous is between 0.50 and 0.75 over the first four

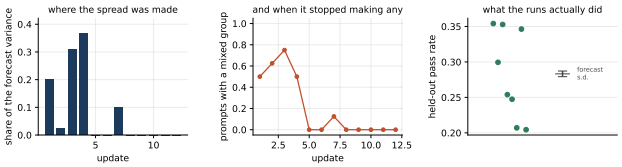


Figure 9: The forecast on a pretrained model. **Left:** each update's share of it. **Right:** the fraction of prompts still sampling more than one answer, which is what makes the left panel end where it does.

quantity	value
held-out pass rate, base	0.163
held-out pass rate, after 8 runs	0.283
observed s.d. across runs	0.0636
evaluation (binomial) part	0.0077
seed part, by subtraction	0.0631
its nominal 95% interval	[0.0350, 0.0735]
forecast from one run	0.0040
held at ∇M	0.0025
held, Σ from 64 prompts	0.0026
agreement of two Jb estimates	0.12
updates with a mixed group	5 of 12
share of it made in the first half	90%

Table 9: The forecast on Qwen2.5-0.5B against the spread it was made before seeing. The evaluation's own binomial noise is subtracted to recover the seed term. The interval is a percentile bootstrap at its nominal level, and Appendix C simulates its actual coverage at this design as 68% to 88%: it is narrower than it claims, so it should be read as indicative rather than as a calibrated 95% interval.

updates and zero in 7 of the remaining 8 (the other is 0.125), so $\Sigma_t \approx 0$ from update five onwards and the forecast is, in effect, made from four updates out of twelve (Figure 9). This is the sampling collapse of Section 3.4 happening at a step size we chose to be conservative, on a task chosen to be hard for the model.

The second is that the directional derivative is not reliably estimated. Taking Jb twice from independent draws of prompts gives two vectors whose cosine has a median of 0.12 and 0.21 in the two launches that stored it: mostly noise at this batch size. On its own that would sink the method. It does not, because Section 6.1 has already established that the transport is worth about 2%: what we report is the cheap estimator of (12) in all but name, and the quantity that is badly estimated is one the forecast barely uses.

What the comparison says. The eight runs realise a spread of 0.0636, of which 0.0077 is the evaluation's own binomial noise, leaving a seed term of 0.0631 with an interval of [0.0350, 0.0735] (Table 9). The forecast a practitioner would run is the cheap form of (12), which is what Section 6.1 recommends and what the noisy di-

rectional derivative leaves available. It is 0.0025, short by a factor of 25.

Taking the transport anyway gives 0.0040, and repeating the experiment says how far either number can be trusted. A launch made before the adapter initialisation was seeded started from a different draw and otherwise the same configuration. It gave a transported forecast of 0.0225 against 0.0040, a factor of 5.7, and a cheap one of 0.0056 against 0.0025, a factor of 2.2. Neither form is stable under a change of initialisation, and the transported one is far less so. The transport is worth 2% where it can be computed exactly; here it is estimated from vectors whose cosine with an independent estimate of themselves has a median of 0.12 and 0.21 in the two launches that stored it, and compounding twelve such corrections moves $\|b\|$ by an amount that does not reproduce. On a real model the transport is not a small correction badly estimated, it is unusable, and dropping it leaves a quantity that still moves by a factor of two between initialisations.

An explanation that would have been comfortable is ruled out by the same table. The injected term is estimated from a batch of eight prompts, and once most groups are unanimous that sample is mostly zeros, so it could be biased low. Re-estimating it from sixty-four prompts, from the same initialisation and changing nothing else, moves the forecast from 0.0025 to 0.0026, about five percent where a factor of 25 is needed. Measured on those sixty-four prompts the share of prompts whose group is not unanimous still reaches zero at the fifth update and never exceeds five percent afterwards, so the collapse is not a small-batch artefact either. The shortfall is not an estimation problem.

The reason for the shortfall is visible without any statistics. The eight held-out scores are not scattered around a centre. Three of them sit at 0.346, 0.353 and 0.354, within 0.008 of each other, and the remaining five are spread from 0.204 to 0.299 (Figure 9, right). A linearised model predicts the spread of a unimodal perturbation about one trajectory, and this is not that. Whether it is exactly two modes we would not claim from eight runs, and the grouping is not identical between reruns; what is stable across reruns is a tight cluster near 0.35 sitting apart from everything else. A standard deviation is a poor summary of such an outcome for anyone, forecast or not.

What the runs actually learned. The held-out prompts can be regenerated from the seed the experiment used, so each prompt's correct count is known, and that settles what the grouping is (`experiments/s16_saturation`). The run the forecast was made from answers "1" to every prompt: it passes exactly the 17 of 48 prompts whose count is one, which is the score of 0.354 the three clustered runs share. Of the 8 runs, 5 answer a single constant whatever they are asked ("1", "2" or "4"), and which constant a run falls into explains 94% of the across-seed variance of its score. Prompts sharing an answer move together across seeds (correlation 0.90) and against the others (-0.33). Two earlier launches of the same configuration look the same, with 6 and 5 of eight runs constant and the label explaining 99% and 81%, and in every launch the anchor answers "1". At 7B under Adam on the same task, 4 of 10 runs answer a constant and the label explains 40%.

On this task and budget, then, RLVR mostly teaches the model to guess a count. A constant answer gives every response to a prompt the same reward, so the count-based advantage is zero and a run that reaches one cannot leave it. The seed spread is which constant a run is absorbed into, a discrete outcome that a linearisation about one run, already absorbed, has no term for. That also qualifies everything else measured on this task: much of the held-out gain is guessing, and the forecasts made here, which differ by a factor of 681 between two repeats from the same 7B run, describe runs in a degenerate regime.

Collapse does not explain the support sweep. The four corpora of Section 6.10 were rerun with every run's answers stored, two at a time, and the reruns reproduce the original held-out scores run for run. At live probability 0.85, where the forecast held, 12 of 12 later runs answer "1" on most prompts, as the anchor does on 38 of 64; the collapse is there but shared, and it carries almost none of the spread. That suggested that collapse defeats the forecast only when the answer a run is absorbed into depends on its seed. Before the second rerun finished we recorded a prediction with a refutation criterion (`docs/predictions.md`): the band at 0.60, where the forecast missed by 2.80×, would have more than 8% of prompts on which two runs commit firmly to different answers. It has 3 of 64, against 5, and the prediction is refuted. It should not have been made in that form. The forecast at 0.60 is too large, and seed-dependent collapse adds spread, so it could only ever have made a forecast too small. Looked at after the result, that band hardly collapses at all: 0 of its runs give one answer to more than half of the prompts, and every run agrees firmly on 1 prompt against 26 at 0.85. That suggested a second pattern, again recorded before its runs: the forecast is too large where runs are not absorbed into shared answers. Band 0.35, where the forecast is 2.13 times the measured spread, would have at most 13 prompts on which every run agrees firmly, and band 0.15, where it is 0.60 times, at least 14. The first part holds with 1 and the second fails with 2; across the four bands the counts are $2, 1, 1, 26$. Shared answers mark the one corpus where a single answer is right often enough to be rewarded, not the forecasts that are too large, and

the corpora's construction made that knowable before the run. Collapse accounts for the shortfall on the counting corpus and for none of the sweep's errors; the overpredictions in its two middle bands remain unexplained.

Whether the outcome is decided early, which we cannot establish. It is tempting to read the grouping off the start of each run, and the data invite it: the pass rate a run achieves on its first three batches correlates with its final held-out score at $r = +0.44$ here, against -0.20 for its last three. The bootstrap interval over runs is $[-0.19, +0.96]$, the one other launch that stored its training traces gives $+0.65$, and at seven billion parameters the same correlation is $+0.11$, which is to say absent. We report the numbers and draw nothing from them. Eight runs of one configuration cannot support a claim of this shape, and we came close to making one.

What the measurements either side of it do support is narrower. The per-update kernel puts 90% of the forecast variance in the first half of the run, and Section 6.3 finds the leading eigenvalue of A_t above one, so variance is injected early and then amplified. Whether that is what decides which group a run ends in is the step we cannot take, and it is the one that would have made the observation useful. The mechanism the paper describes is visible on a real model; the magnitude the linearisation assigns to it is not right, because the amplification carries the runs out of the regime where a linear model applies.

Whether a better-behaved step size exists. Not obviously, at this scale. A smaller step keeps the runs closer and the linearisation valid, but then the metric barely moves and the seed term drops below what eight runs can resolve at all (Appendix C: below about 0.01 the subtraction returns nothing usable). The window where the runs both learn and stay in the linear regime may simply be empty for a half-billion-parameter model on a task it finds hard, which is a statement about the experiment we could afford rather than about the method.

6.10 What the failing runs had in common

Sections 6.9 and 6.12 report the forecast short by 25× and by three to seventeen, and read that as the runs leaving the regime a local model describes. Both runs share something we did not control for. A count-based advantage is identically zero on a unanimous group, so a prompt contributes to the seed spread only when its group contains both a success and a failure. For a prompt the policy passes with probability p and a group of G,

$$\ell(p, G) \;=\; 1 - p^G - (1-p)^G \tag{28}$$

is the chance of that happening, and the batch size a run reports is not the batch size its estimator receives: with mean live probability $\bar{\ell}$ the expected number of prompts carrying any noise at all is $P\bar{\ell}$. That is a count rather than a statistical effective sample size, which would additionally depend on how unequally the live prompts contribute, but it is the quantity that says whether a batch has anything in it to measure. On the corpus those runs used, $\bar{\ell}$ fell to about a tenth within five updates, so a nominal eight prompts were carrying about one.

Restoring the support moves three things at once. We selected prompts the base policy is genuinely uncertain about, by measuring $p(x)$ on a survey pool and keeping those away from both ends, and repeated the prospective test of Section 6.9 unchanged in every other respect. The live share holds at 0.45 through training rather than collapsing. The forecast is computed from one anchor run, frozen to disk with the accept-or-refuse decision before any replica exists, and evaluated against 12 independently trained seeds that exclude the anchor. It is 0.0604 where the measured seed term is 0.0310, a ratio of 2.0, and two disjoint half-sample estimates of the injected term agree to 1.09. On the saturating corpus the forecast was 25× short at this size, and at 7B its two half-samples disagreed by 5.5×. The learning is not sacrificed for it: the pass rate goes from 0.271 to 0.648, a larger improvement than any other real-model run here.

The three quantities moving together is what the claim rests on, rather than the ratio alone. We do not conclude that the earlier failures were an artefact and the method works; 2.0 is still outside the factor of 1.5 we would want, and this is a half-billion-parameter model. What the evidence supports is narrower and, we think, more useful: the severity of the failure is strongly regime-dependent, and maintaining stochastic support moves the system a long way towards the regime the theory describes. A degenerate measurement regime, not model scale, accounts for most of the gap we previously reported.

A sweep across four regimes, and what it falsifies. The comparison above changes the corpus, and with it the pass-rate distribution, the score geometry and both noise terms together, so it implicates the measurement regime without isolating it. We therefore built 4 corpora of 255 prompts each at pre-specified mean live probabilities of 0.15, 0.35, 0.60 and 0.85, holding the model, optimiser, budget, update count, batch and group size, evaluation set and forecast procedure (the cheap form of (12)) fixed. The corpora

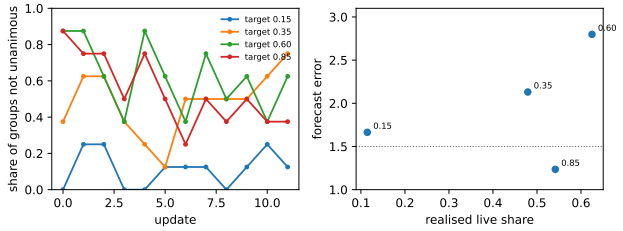


Figure 10: Left: the share of groups that are not unanimous, along each band's anchor run. Support is endogenous: the band that learns fastest spends its own, while the lowest band stays low throughout, so a mean over the run is not the treatment we set. **Right:** forecast error against realised support, with the declared 1.5× criterion dotted. There is no trend, which is the point.

and the launch script naming all four conditions were written before any of them ran.

The forecast errors are $1.66, 2.13, 2.80, 1.24$, in that order. They are not monotone in support, and the middle bands are the worst. They also do not point the same way: as ratios of forecast to measured spread they are $0.60, 2.13, 2.80, 0.81$, so the forecast is too small in the lowest and highest bands and too large in the two middle ones. Half-sample disagreement stays between 1.01 and 1.30 in every band, including the one whose live share averaged 0.11. We therefore withdraw the reading that sparse stochastic support is the dominant determinant of either estimator stability or forecast error: it is not, on this model. Taken with the cross-fitting result below, two candidate explanations of the original discrepancy are eliminated. The large curvature residuals remain consistent with a breakdown of the local linear approximation, but these experiments do not attribute the remaining error to it uniquely, and we do not claim they do.

Two features of the sweep are worth recording rather than explaining. Support is endogenous: the realised shares are $0.11, 0.48, 0.62, 0.54$, so the two highest bands reverse, and Figure 10 shows why. The band that learns fastest consumes its own support, falling from 0.88 to 0.38 along its anchor run, while the lowest band simply stays low throughout. A mean over a run is not the treatment we intended to apply, and a corpus that sustains a low share is not the same object as one that starts high and collapses.

One of the four is accurate, and the diagnostic refuses it. In the 0.85 band the frozen forecast is 0.0205 against a measured seed term of 0.0253, nominal interval $[0.0105,\ 0.0337]$, on a run whose pass rate rose by 0.275. That is a multiplicative error of 1.24, inside the 1.5× band we declared before the sweep, and the point estimate sits comfortably within the interval on the measured spread. It is the first prospective forecast in this paper that is quantitatively accurate on a pretrained model.

We report it as one of 4 pre-specified conditions rather than as a validation, and the other three are in the same table. The interval at twelve seeds is wide, so the claim is that the forecast is within a quarter of the measured spread multiplicatively, not that the error is exactly 1.24. And the diagnostic of Section 6.11 refuses this run too, at a residual of 8.0: a practitioner following the rule would have been told not to trust a forecast that was in fact good. The estimator can be accurate on a real model; what we cannot yet do is say in advance when it will be.

Effective prompt difficulty is policy-relative. That raises the obvious question of whether the same corpus can be built at 7B, and the answer in this task suite is no, for a reason worth reporting. We examined 24 task families in all, of which 18 have matched surveys at both scales. The families that carry support at half a billion are solved at seven: `add_two` reaches 1.000 and `sort_digits` 0.987. Families impossible at half a billion move into the band at seven: `sort_words` goes from 0.000 to the best family we measured. The informative band does not shrink with scale so much as move, and a fixed suite carries support only for the policies it happens to suit.

Support is hard to manufacture where answers are all-or-nothing. We then built a parametric ladder to place a corpus deliberately, varying the number of words to sort, numbers to order and terms to add. Only sorting words degrades gradually. Ordering numbers passes at 0.990 with six and 0.000 with eight, with nothing usable between: one rung takes it from solved to impossible. On tasks whose answer is exactly right or exactly wrong the difficulty gradient is a cliff rather than a slope, so stochastic support at a chosen level is not merely scarce at scale but structurally hard to construct. Across three surveys and every rung we could build, the eligible pool at 7B reached 192 prompts against the 240 our criterion required, a criterion fixed and hashed into the repository before the survey was extended, and we did not run the experiment. We record the criterion, its hash and the surveys rather than relaxing it after the fact. A task with genuinely graded difficulty, which short exact-match problems are not, is what this experiment needs.

6.11 A run deciding whether to trust its own forecast

Everything above rests on the mean update field being close to linear over the distance seed noise moves a run. Section 6.4 measures how far that holds, but between two runs, which diagnoses a pair once both exist and cannot certify a forecast made from one. The

quantity this section measures needs no replica. At a stored state, draw two independent batch gradients, take their difference as a direction, and probe the mean field at $\pm r$ along it where r is how far one update of that noise moves the parameters:

$$C \;=\; \frac{\left\| \bar{\boldsymbol{g}}(\theta + rd) + \bar{\boldsymbol{g}}(\theta - rd) - 2\bar{\boldsymbol{g}}(\theta) \right\|}{\left\| \bar{\boldsymbol{g}}(\theta + rd) - \bar{\boldsymbol{g}}(\theta - rd) \right\|}, \qquad (29)$$

zero on an affine field and growing as it bends. Three evaluations of the mean update per direction, and nothing that depends on knowing the answer.

It detects failure without predicting it. Over the 192 settings of Section 6.1, C correlates only weakly with the size of the forecast error, at a rank correlation of 0.27. As a detector of the tail it is better than chance and no better than modest: the area under the ROC curve for flagging a setting whose forecast is wrong by more than 1.2× is 0.70, over 36 such settings, and for more than 1.3× it is 0.78 over 12. On the replicate grid at a pool of 16 prompts the same two figures are 0.76 and 0.85, so the ranking is real but its strength moves with the sample.

It does not bound the worst case. Accepting only the settings whose residual falls below 0.0300, which is 80% of them, leaves the worst forecast error at 1.56×, exactly what accepting everything gives: the single worst setting sits at the 75th percentile of the residual, inside the accepted set. The replicate grid behaves the same way (1.50× against 1.50×). An earlier version of this paper calibrated the rule on a 96-setting grid, where refusing a fifth did cap the tail, and said so; that did not survive doubling the grid, and we withdraw it. On a grid generated from a different environment seed, with the threshold held fixed, the rule accepts 82% and refuses the worst setting there, at 2.13×, but still accepts one at 1.58×. We fix the threshold here, on the headline grid, and do not touch it again.

Applied unchanged, it refuses both pretrained runs. On Qwen2.5-0.5B the residual is 2.0, and on Qwen2.5-7B under Adam it is 160.5, against a threshold of 0.0300 and a grid whose whole range is 0.0044 to 0.0488. Both are refused, by factors of about seventy and six thousand. Those are the two runs whose forecasts Sections 6.9 and 6.12 report as short by 25× and by between 3.1 and 17, so the rule declines exactly the numbers that should not be believed, having been given no information about them.

It also refuses the forecast that worked, and that is the more informative case. The best of the four pre-specified regimes of Section 6.10, whose forecast is within 1.24× of the measured spread and whose half-sample estimates agree to 1.23, scores 8.0 and is refused as well. So the rule declines the best real-model forecast in this paper along with the two worst.

That settles what the two earlier refusals could not. Every real-model residual we have measured lies between 2 and 160, while the grid the threshold was calibrated on spans 0.0044 to 0.0488: the two scales barely overlap, and a threshold from one does not transfer to the other. On a tabular policy the second difference in (29) measures curvature; on a pretrained model $\bar{\boldsymbol{g}}$ is itself estimated from sampled rollouts and a second difference amplifies that estimation noise, so the same statistic is measuring curvature plus a Monte Carlo floor we have not subtracted. We therefore claim only that the diagnostic ranks nonlinear regimes within the controlled testbed, where its threshold was set and where both classes exist. Its absolute scale does not transfer to pretrained models, its coverage there is zero, and a rule that refuses everything is safe and useless. Calibrating it would need a statistic normalised by its own estimation noise, and more than one accepted example to calibrate against; choosing a threshold from the four real runs whose answers we already know would be fitting to the test set.

Of the 4 checkpoints probed on each model, 2 and 3 had any noise left to probe along at all, the rest having gone unanimous, and a checkpoint with no noise is reported as carrying no measurement rather than as a linear field.

6.12 The forecast at seven billion, under the optimiser people use

Section 6.9 makes the forecast under plain ascent, which is the update the recursion is written for. A practitioner uses Adam. Section 6.8 shows the lift to $z = (\theta, m, v)$ holds where the moments are exact and warmed, so the remaining question is whether it survives on a pretrained model. One answer is already in hand: these runs start Adam cold and without bias correction, the regime in which the same lift fails on the exact tier (2.40× median, 549× worst; Section 6.8), so what follows tests the lift outside the conditions it was validated under. Under the cheap form the injected term enters through $B = \partial\Phi/\partial\hat{g}$, which for Adam is diagonal and analytic, so the forecast is the same backward pass with the metric direction scaled by the optimiser's own preconditioner at each update.

The measurement. 10 runs of Qwen2.5-7B at four-bit precision, twelve updates of Adam, take the held-out pass rate from 0.203 to 0.362. The spread across runs is 0.0893, of which 0.0051 is the evaluation's own binomial noise, leaving a seed term of 0.0892 with an

interval of [0.0398, 0.1193]. No bootstrap draw lands at zero, so unlike the 0.5B design this one resolves the quantity it is measuring. The spread is a little over half the improvement the runs achieved.

The forecast is short, by between 3.1 and 17. It reads 0.0207 against a measured 0.0892, a point ratio of 4.3. We do not report that ratio on its own, because the same run's two half-sample estimates of the injected term are 0.0052 and 0.0287, a factor of 5.5 apart, and the shortfall they imply runs from 3.1 to 17. The direction agrees with Section 6.9 and the size is now within the range a linearised model might plausibly be wrong by, rather than the two orders of magnitude we previously reported.

A number we published and then corrected. An earlier version of this section put that shortfall at 38 and read it as evidence that the runs had left the regime a local model can describe. The forecast behind it applied Adam's preconditioner to each update's injection but then discarded the adjoint, carrying nothing from one update to the next. The exact tier says what that costs: on the same grid, a recursion on θ alone, carrying no optimiser state, is wrong by a factor of 24.6 in variance where the dynamics are perfectly local, no better than summing the injections with no contraction at all (24.2), while the full lift is right to 1.10×. Almost all of the 38 was therefore ours. Carrying Adam's moments backwards on the identical trajectory and the identical prompt sample moves the forecast from 0.00237 to 0.0207, a factor of 8.7, and the conclusion the earlier number supported does not survive it.

What carrying the state costs, which is nothing. The transfer operator on $z = (\theta, m, v)$ has three blocks that require the Jacobian of the mean update, and all three sit in the column that differentiates with respect to θ. That Jacobian is the one quantity a real model cannot supply: Section 6.9 measures independent estimates of it agreeing to a median cosine of 0.12 and 0.21 in the two runs that stored it. Dropping those blocks leaves couplings that are diagonal and analytic, costing one extra vector per update and no rollouts at all. On the exact grid it costs nothing systematic: the median error is 1.07× without the blocks against 1.10× with them, and the worst 1.90× against 1.80×. The accuracy the lift buys comes from carrying the optimiser state and not from the curvature. That is convenient, since the curvature is the half we could not have had.

The estimate of the injected term is now the binding constraint. Carrying the state concentrates the kernel on the first two updates, where a perturbation has the longest to act through the moments, and those are estimated from the same sparse sample as everything else. The half-sample disagreement accordingly rises from 2.4× without the state to 5.5× with it. The accuracy problem and the estimation problem have swapped places: what limits this forecast is no longer the recursion but how well 32 prompts pin down a variance when roughly a tenth of groups contribute anything at all.

The diagnostic still costs more than it should. That backward pass took 141 minutes against the 260 that training all 10 seeds costs. The cost argument is unchanged in kind by the correction, though the forecast it buys is now much closer to right. What the correction does change is where the remaining effort belongs: on estimating the injected term from the groups that actually carry it, rather than on the recursion, which is now the part that works.

6.13 Two ways a run stops being measurable

The largest model this machine holds is Qwen2.5-7B-Instruct at four-bit precision, trained with LoRA adapters and Adam. Running it turned up two ways for a reported number to stop carrying information, and they fail in opposite directions. In one the spread swamps the effect, so a single run cannot report the improvement it claims. In the other the metric stops resolving the runs at all, so the spread reads zero and a practitioner concludes their pipeline is reproducible. Both are visible in (11), which sends the spread of a reported number through ∇M: the first is that quantity being large next to what the run achieved, the second is ∇M vanishing on the evaluated distribution.

The spread swamped the effect. On counting a letter in a word, 8 runs of 20 updates take the held-out pass rate from 0.203 to 0.313 on average, an improvement of 0.110. The individual runs land between 0.208 and 0.460, a standard deviation of 0.0925 across seeds, which is 84% of the mean improvement (Table 10). The evaluation's own noise accounts for almost none of it.

The consequence is not that the training failed. On average it worked. It is that a single run of this recipe reports a number whose run-to-run uncertainty is nearly as large as the effect being claimed, so the ordinary practice of running once and comparing against a baseline cannot separate a real improvement from a lucky seed at this scale. This is exactly the case the forecast of Section 3.4 is meant for, and Section 6.12 supplies one for the same model under the same optimiser, twelve updates rather than twenty.

quantity	Qwen2.5-0.5B	Qwen2.5-7B
runs	8	8
updates	12	20
held-out pass rate, base	0.163	0.203
held-out pass rate, after	0.283	0.313
spread across runs	0.0636	0.0925
evaluation part	0.0077	0.0055
seed part	[0.0631, 0.0636]	[0.0923, 0.0925]
split at the largest gap	5 / 3	3 / 5

Table 10: The same measurement at two model sizes. The seed term is given as the bracket of Section 6.1 rather than by subtracting the binomial term, which is the conservative reading where a run's record does not establish that its seeds drew independent evaluations. Both brackets are narrow, so little turns on the choice here.

The metric went blind. On counting a letter in a word, a task the base model already passes at 0.31 on the prompts of the difficulty scan (Section 6.10), twenty updates take the held-out pass rate to 0.875 and drive every held-out prompt to a unanimous sample. All 32 of them return a measured rate $\hat{p}$ of exactly 0 or exactly 1: eight samples, one answer. Eight samples cannot distinguish $\hat{p} = 1$ from a true rate of, say, 0.9, which is unanimous 43% of the time, so what is established is that the evaluation has stopped separating the runs, not that the response distribution has collapsed to a point. Two runs differing only in rollout randomness report *the same number to the last decimal* and agree on 32 of 32 prompts individually, while their policies differ by between 1.8×10^{-5} and 7.7×10^{-2} in KL and that difference does not accumulate over the run.

This is worth being precise about, because the two readings are opposite. It is not that the seeds converged. It is that the metric stopped resolving them: the measured $\mathbb{E}_x[\hat{p}(1-\hat{p})]$ is zero on the evaluation set, so a score built from those samples has no variance to report, whatever the policies do. The two readings are separated by the KL between the policies, which is not zero, rather than by the scores, which are identical. The practical consequence is that a zero measured spread on a saturated benchmark is not evidence of a reproducible procedure, and the theoretical consequence is the one Section 3.4 is built on: the spread of a reported number is $\nabla M^\top S_T \nabla M$, and when ∇M is zero on the evaluated distribution no amount of S_T reaches it. It is also, in retrospect, why the route through $\sqrt{\mathrm{KL}}$ that an earlier version of this work took could not have worked in general.

6.14 The same benchmark on a pretrained model, and why it does not settle the question

The result above is measured where the forecast is validated exactly. To see whether it carries over, we trained 6 recipes on one corpus of Qwen2.5-0.5B at a matched rollout budget of 64 per update, crossing the two ways of splitting that budget, $(P, G) \in \{(16, 4), (8, 8)\}$, with three learning rates spanning a factor of four. Each recipe is one anchor run, which produces the forecast, and 16 validation seeds: 102 training runs.

The forecast itself holds up. Across the six recipes its median error against the measured spread is 1.61× and its worst 2.75×, so all six sit inside the 4.2× break-even. It overpredicts in five of the six. The across-recipe variation it is supposed to capture is real: the measured deviation spans 4.7× between the quietest and noisiest recipe, and the forecast spans 3.0× of that, correlating at 0.60 in logs. That is a weaker correlation than the 0.98 of the exact grid and a compressed range, but it is the right ordering from one run each.

The benchmark built on it is underpowered, and we report it as such. Only 4 of the 15 recipe pairs are ordered unambiguously at 2 standard errors; the median pair is separated by 1.5. The six recipes differ in mean by at most 0.025 while their per-recipe deviations run from 0.010 to 0.048, so most of these pairs are genuine near-ties at sixteen seeds. That fact is worth stating on its own, since it is the paper's premise showing up in the data: at this scale, most of the recipe choices a practitioner would actually weigh are not resolvable by the seed counts anyone runs.

What survives on four comparisons is one thing. A stopping rule of any kind beats a fixed budget, which needs 12.53 runs against the forecast's 8.95, a factor of 1.41 [1.28, 1.65]. Nothing else separates. The constant deviation costs 1.10× and the rule driven by spent seeds 0.95× [0.85, 1.15], both intervals straddling one, and the ordering among them changes with the seed cap and the target error. The clearest sign that this is resolution rather than result is that the measured deviation, which cannot be beaten by a forecast of it, comes out at 1.05× [0.95, 1.19]: nominally behind the forecast, which is not a real effect and marks the floor of what four comparisons can see.

So the 9.57× of Section 7 is not demonstrated at this scale and we do not claim it. What this experiment establishes is narrower: on this corpus and these six recipes the forecast is inside the break-even, and stopping early beats not stopping. Whether forecasting the

deviation beats estimating it cheaply is a question sixteen seeds on six recipes cannot answer, and answering it needs either more seeds per recipe or recipes chosen to be further apart, neither of which we could afford on one laptop.

7 What the forecast is for

A predicted standard deviation is not a result on its own. Nobody trains a model in order to learn how much its score would move under a different seed; they want to know whether one recipe beats another, and the deviation is the thing that says how many runs settle it. This section measures the forecast by that standard.

7.1 The decision

Two configurations are to be ordered. A practitioner trains each for a seed, looks at the lead, and either calls it or trains another seed of each. Stopping when the lead $\bar{D}_k$ after k seeds per arm clears z standard errors,

$$|\bar{D}_k| \;\geq\; z\,\frac{\sigma_D}{\sqrt{k}}, \qquad \sigma_D = \sqrt{\sigma_A^2 + \sigma_B^2}, \tag{30}$$

is the rule this is an instance of. It needs σ_D, and that is the difficulty: at $k = 1$ there is no empirical deviation to put there, because one seed per arm carries no information about spread. The choices that remain are to fix k in advance, which is what is normally done, or to spend seeds building a variance estimate before the comparison can begin. Section 3 supplies σ_A and σ_B from the first run of each arm, so (30) is available from $k = 1$.

Note what (30) does *not* require. Multiplying every σ by a common factor is undone by dividing z by the same factor, so a forecast that is uniformly off by a constant reaches the same operating points as one that is not, provided z is chosen on a range wide enough to allow the move. What matters is whether the forecast gets the *relative* deviation right, across configurations. We check both claims below.

7.2 Setup

The scores are the 48 per-seed scores already stored for each of the 192 configurations of Section 6.1; no new training is done, and every policy sees the same seeds in the same order, so the comparison is paired. We compare configurations that share a task.

The true ordering of a pair has to come from somewhere, and reading it from seeds a policy is also shown would correlate the policy's evidence with the answer it is graded against, flattering whichever policy looks at the most seeds. So 32 of the 48 seeds are reserved:

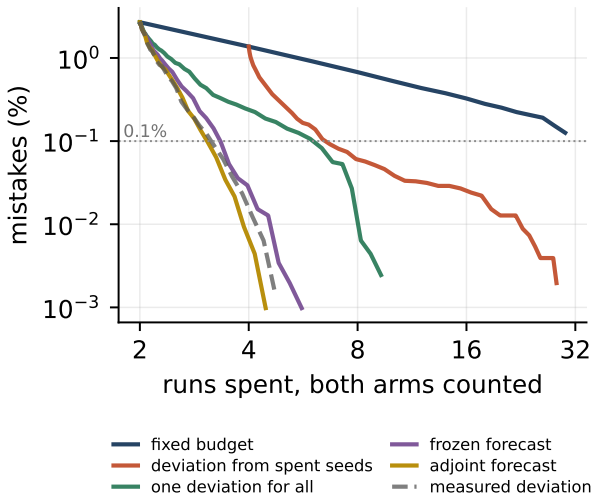


Figure 11: Mistakes against runs spent. A policy is a curve, not a point, since the stopping threshold trades one against the other, and what separates two policies is how far left a curve sits at a fixed height. At 0.1% the forecasts cross near 3.07 runs and a fixed budget near 30.4. The rule driven by spent seeds flattens on the left because it cannot stop before two seeds per arm however large the lead.

the ordering is fixed on those, and no policy may ever draw one. The 1020 pairs whose reserved-seed ordering is unambiguous at four standard errors are kept, so that a disagreement is a mistake rather than a coin landing the other way up, and the policies draw from the remaining 16. Each pair is replayed under 400 random orderings of those.

A policy is a curve rather than a point, since z trades runs against mistakes, and reading every curve at a common mistake rate is what makes two of them comparable. We report the runs each needs to hold its mistake rate at 0.1%, with both arms counted, so k seeds per arm is $2k$ runs.

Six policies are compared. *Fixed budget* spends a predetermined k and never stops early. *Deviation from the seeds* estimates σ_D from the seeds in hand, and cannot stop before $k = 2$. *Constant deviation* uses one number for every configuration, the median measured deviation, which is the sharpest form of the assumption that variance does not depend on the recipe. *Frozen forecast* is the predictor of Section 5 available before any run exists. *Adjoint forecast* is Theorem 2 run on the first seed of each arm. *Measured deviation* is given the realised across-seed deviation and is the ceiling.

7.3 Result

Table 11 is the outcome. The adjoint forecast reaches 3.07 runs. The ceiling, a rule handed the realised deviation, reaches 3.16: the two are the same policy to within resampling error, at $1.02\times$ $[0.99, 1.05]$, and we

policy	runs	vs. adjoint	95%
fixed budget, no stopping rule	30.41	9.57	[7.24, 10.56]
deviation from the seeds in hand	6.52	2.13	[1.98, 2.38]
one deviation for every recipe	6.04	1.93	[1.45, 2.28]
frozen forecast, before any run	3.34	1.09	[1.02, 1.14]
adjoint forecast, from one run	3.07	1.00	[1.00, 1.00]
measured deviation	3.16	1.02	[0.99, 1.05]

Table 11: Runs needed to hold the mistake rate at 0.1% over 1020 unambiguous comparisons, both arms counted, with 95% intervals from resampling the comparisons. The forecast is not better than knowing the deviation; it is not measurably worse, which is the point. Ordering follows forecast accuracy: the adjoint forecast is 1.11× off at the median here, the frozen predictor is coarser, and a constant deviation discards the variation entirely.

	target 1%		target 0.1%	
threshold set on other tasks	runs	mistakes	runs	mistake
fixed budget	6.00	0.91%	32.00	0.00%
one deviation for every recipe	2.47	0.98%	6.24	0.12%
deviation from spent seeds	4.05	1.02%	6.66	0.10%
closed form, no run	2.30	0.96%	3.48	0.08%
closed form, one scale from other tasks	2.29	0.98%	3.47	0.08%
adjoint forecast	2.23	0.98%	3.14	0.10%
measured deviation	2.22	0.95%	3.22	0.09%
textbook threshold, no tuning				
one deviation for every recipe	4.36	0.22%	5.77	0.11%
closed form, no run	32.00	0.00%	32.00	0.00%
closed form, one scale from other tasks	4.64	0.01%	6.23	0.00%
adjoint forecast	4.37	0.00%	5.96	0.00%
fixed budget, 24-seed cap, 3 splits	4.0–4.0		15.3–23.3	

Table 12: Runs per comparison and realised mistake rate over 1020 comparisons when no threshold is read off the comparisons being graded. Upper block: each policy's threshold is fitted on the other eleven tasks and applied to the twelfth. Lower block: the textbook threshold, with no tuning. At 0.1% the fixed budget sits at the 16-seed cap, so the last row extrapolates it without one from the normal approximation.

read that as the forecast being good enough to cost nothing here rather than as an improvement on knowing the answer. Ordinary fixed-budget practice needs 30.4 runs for the same reliability, 9.57× [7.24, 10.56] more. Spending seeds to estimate the deviation instead of forecasting it costs 2.13× [1.98, 2.38], and most of that penalty is structural rather than statistical: that rule cannot stop at one seed per arm however large the lead, so four runs are committed before any comparison can be decided.

The reserved ordering set matters to the size of the effect, and in the direction that argues for using it. Grading against all 48 seeds instead, so that a policy's own seeds are part of the answer, cuts the advantage over a fixed budget from 9.57× to 4.76× and over spent seeds from 2.13× to 1.65×. The overlap helps a policy in proportion to how many seeds it draws, so it favours the baselines that draw the most, and the headline numbers here are the ones without it.

The ordering of the six tracks how accurately each predicts the deviation, which is the result rather than a coincidence, and it is what makes the next question the operative one.

Note which predictor is doing the work. The closed form of Section 5 (called the frozen forecast in the tables, since it is fixed before any run): the step size, the run length and the two noise traces, with no stored trajectory, no backward pass and no adjoint. It reaches 3.34 runs, within 1.09× of the adjoint and still 9.1× better than a fixed budget. On the exact models those traces are computed in closed form; on a trained model they come from the microbatch gradients a trainer already accumulates (Section 5), which is close to free. So the bulk of the saving here does not require Theorem 2 at all, and the backward pass buys the last 1.09×. That is worth saying plainly, because it is the cheap half of this paper that a practitioner would deploy first.

7.4 Setting the threshold before the comparison

Every number above reads a policy at the threshold that meets the target on the comparisons it is then graded on. That is even-handed, since each policy gets the same in-sample choice of its one knob, but a practitioner cannot do it, and it is what makes a constant scale error look free. We therefore repeat the benchmark in the two ways a practitioner could run it (Table 12, `experiments/s13_sizing/heldout.py`). In the first, the threshold, or for a fixed budget the seed count, is the cheapest one meeting the target on the other eleven tasks, and is applied unchanged to the twelfth. In the second there is no tuning at all: the rule stops once the lead clears the one-sided normal quantile of the target, in units of the policy's own standard error. Intervals resample the twelve tasks rather than the comparisons, because comparisons inside a task share arms and are not independent.

Thresholds chosen on other tasks transfer: realised mistake rates land near the target, the largest overshoot being the single deviation for every recipe at 0.12% for a 0.1% target. At 0.1% the adjoint forecast needs 3.14 runs and a fixed budget 10.20× [9.59, 10.88] as many, but that budget is pinned at its cap. A normal approximation on the ordering seeds puts the uncapped budget at 14 runs, but those gaps are inflated by the selection of clear pairs, so we measured it instead: splitting the seeds 24/24, so that a fixed budget can stop below a cap of 24 per arm, it needs 15.3 to 23.3 runs over 3 truth/available splits, 5.6 to 8.2× the forecast's 2.75 to 2.86, where the approxima-

tion said 10. At 1% the saving is 2.69× over a fixed budget and 1.82× over a rule that estimates the deviation from spent seeds. Against the closed form it is 1.11× [0.97, 1.28] at 0.1% and 1.03× at 1% (1.10 to 1.11× over the 24/24 splits): once a threshold can be learned from comparisons on other tasks, the stored run adds little to a forecast that only has to rank. The closed form is fed the exact noise traces there. Estimating them instead with the split estimator of Section 5, averaged over one run's worth of batches at each configuration's own prompt count and group size (`experiments/s13_sizing/estimated.py`), puts the estimate within 1.12× of the exact closed form at the median, with 1 of 192 configurations estimating a spread that is not positive, and moves the held-out cost to 1.18× [1.00, 1.42] the forecast's runs at 0.1% and 1.04× at 1%. On this tier estimating the traces costs a little; on a pretrained model the same estimate would face the sparse groups the forecast does, which we have not measured. Two caveats on the strict target come from the design: the adjoint's mistakes there come from a handful of comparisons in half the tasks, and a call that runs to the cap is never wrong, because at four standard errors the sixteen available seeds always agree with the ordering seeds.

At the textbook threshold the uncalibrated closed form never stops, because its scale is off by a factor of 84. Given one scale factor learned on other tasks it decides in 4.64 runs with a realised mistake rate of 0.013% against a 1% target, and the adjoint forecast in 4.37 runs with 0.001%. What the stored run's scale buys, then, is that one factor: it lets the textbook threshold be used without comparisons on other tasks to learn it from. Both textbook rules spend about twice the tuned runs, and their low realised rates reflect how clear the selected pairs are as much as the forecast. A single deviation for every recipe costs 4.36 runs with 0.218% mistakes; it needs measured spreads from replicates of at least one recipe. When near-ties are kept, ordering pairs at two standard errors rather than four (1108 pairs), every policy runs into the cap at 0.1%, and at 1% every policy is left at a floor of about 1% mistakes on the comparisons whose available seeds contradict the ordering seeds. Within the seeds we have, no policy settles near-ties, and the result we quote is for comparisons whose ordering is clear.

7.5 The procedure, stated so it can be followed

Nothing above needs a change to the training loop. Written out as a practitioner would run it, for two recipes A and B to be ordered:

1. **During the first run of each arm**, tag the microbatches a trainer already accumulates by prompt block and rollout sub-group and accumulate (22), giving τ_{b} and τ_{w}. Store θ_t and η_t. Log the share of prompts whose reward group is not unanimous. None of this changes the run.

2. **Take a deviation.** One backward sweep of Algorithm 1 over the stored trajectory gives the adjoint forecast, whose absolute scale is what lets step 4 use the textbook threshold. The closed form (25) needs nothing further and ranks configurations almost as well, but its scale is off by a factor of 84, so use it only where the threshold can be set on comparisons with known outcomes from related tasks (Section 7.4).

3. **Complete it with the evaluation term.** The forecast covers the seed part. What is watched is a score on a finite prompt set, which carries its own binomial noise, and the stopping rule needs the deviation of the observable. Add the two in quadrature; the second is closed form and costs no runs.

4. **Run the rule.** After each seed of each arm, stop when (30) fires and report the sign of the lead. With the adjoint forecast, z at the one-sided normal quantile of the tolerated mistake rate realised far less than that rate on clear comparisons, and 1.103% for a 1% target once near-ties were included. That is an observation on one grid, not a coverage guarantee (Section 10).

5. **Check the break-even before trusting it.** The procedure beats a fixed budget while the forecast's *relative* error is under 4.2×, a figure measured with z read in sample. A systematic scale error does not count against it only where z is set on comparisons with known outcomes. Two conditions predict being outside it, and both are measured in step 1 rather than assumed: reward groups going unanimous, which drives the injected covariance to zero and is the failure mode of Section 6.9, and a metric that has saturated, where a zero measured spread is not evidence of anything.

The honest summary of step 5 is that we can tell a practitioner what the tolerance is and how to watch the two quantities that predict falling outside it, but not, from one run, whether they have.

7.6 The other decision, where it helps less

Sizing is not the only thing a deviation is used for. The other is choosing: run each of several configurations once, and keep the one with the most gain per unit of noise and rollout. Reporting only the decision that

flatters the forecast would misrepresent it, so we measure both on the same grid, with the seeds split three ways and never reused across roles. One pilot supplies the observed gain, a disjoint pool supplies whatever deviation a policy is entitled to, and a third reserved set scores the choice. That last separation matters: letting the scoring seeds also set the deviation makes an oracle look unbeatable, because its denominator then shares noise with the denominator it is graded against, and an earlier version of this experiment did exactly that.

Over 12 tasks, the share of the best achievable value the choice keeps is 66% with one deviation for every recipe, 74% with the frozen forecast, 75% with the adjoint forecast and 77% with the measured deviation. The forecast recovers 8.8 points [1.7, 15.6] of the 10.2 that a measured deviation recovers, a share of 0.86.

So the forecast is close to the ceiling in both decisions, but the ceilings differ. Choosing has only 10.2 points of headroom to compete for at all, because the observed gain of a single pilot run is noisy and that noise limits the choice whatever denominator is used. Sizing has no such competing term: the run count scales as $(\sigma/\mu)^2$, the deviation enters squared, and at one run per arm there is no alternative estimate of it. That asymmetry, rather than the quality of the forecast, is why the two numbers look so different, and it is the reason we lead with sizing.

7.7 How wrong the forecast may be

On a pretrained model the forecast is accurate only in some regimes (Section 6.10). Whether that matters depends on how much error the decision tolerates, so we corrupt the forecast on purpose and rerun the benchmark: a lognormal multiplicative error of controlled median magnitude, and separately a systematic scale error.

The scale error does nothing here. Over 100-fold changes in scale the runs needed are identical to the digits we print, a factor of 1.000, because z absorbs it exactly as (30) says. That holds because this benchmark reads z off the graded comparisons. With the textbook threshold a scale error is not absorbed, which is why the closed form never stops in Table 12: calibration is free only to someone who already has comparisons with known outcomes to set z on.

Relative error is what costs, and it costs less than we expected. At the 0.1% target the runs needed rise from 3.07 at the measured accuracy to 5.4 at a median error of 1.5×, 9.5 at 2×, 13.3 at 2.5× and 18.7 at 4×, and never reach the fixed budget's 30.4 anywhere in the range we corrupted, which extends to 12×. At a looser 1% target, where a fixed budget only needs 5.61 runs, the break-even is 4.2×.

That the tolerance widens as the reliability demanded goes up is not a paradox: a fixed budget scales badly with the target because it must carry every comparison to the same depth, while a stopping rule spends deeply only on the comparisons that need it, and a corrupted deviation degrades that allocation without destroying it. The practical reading is the looser number, since 4.2× is the point at which the condition binds first.

That break-even, measured with z read in sample, is the statement of scope. All 4 regimes of Section 6.10, whose errors are $1.66, 2.13, 2.80, 1.24$, sit inside it, though only one of them met the tighter 1.5× accuracy criterion declared before that sweep ran. The two are different bars and we keep them apart: one asks whether the forecast is right, the other whether it is useful.

What sits outside is the task on which most runs learn to answer one constant, missed by 25×, and Qwen2.5-7B under Adam on the same task, short by between 3.1 and 17. There the seed spread is which constant a run is absorbed into (Section 6.9), and a local model about one absorbed run does not see it. Where the forecast is short by that much, a forecast-driven stopping rule stops too early. The contribution is a decision procedure together with the condition under which it beats what is currently done, and a statement of what puts a run outside that condition.

8 Application: spending a rollout budget

The two numbers that made the injected noise measurable answer a second question, one with nothing to do with reproducibility: given a fixed number of rollouts per update, how should they be split between prompts and responses? It follows from the same decomposition, which is why it is here and not in a separate paper.

8.1 Why the usual route does not work here

The standard route to a critical batch size is a second-order model of the objective [McCandlish et al., 2018]. For RLVR it is the wrong model. The objective $J = \mathbb{E}[r]$ is not a log-likelihood, its Hessian is not the negative Fisher information, and along a policy gradient direction a log-linear policy keeps improving well past the step at which a quadratic model turns over; measured against exactly computed improvement below, it places its optimum at least four times too low. A model of how far the *policy* moves is accurate over the same range, and everything below is built on that instead.

8.2 Drift and efficiency

One update of size η displaces the policy by

$$D(\eta) := \mathbb{E}_x\Big[\mathrm{KL}\big(\pi_{t+1}(\cdot \mid x) \,\big\|\, \pi_t(\cdot \mid x)\big)\Big] = \tfrac{1}{2}\eta^2\big(\mathcal{G} + N/P\big) + O(\eta^3), \tag{31}$$

with F the Fisher information of the policy, $\mathcal{G} := \bar{\boldsymbol{g}}^\top F \bar{\boldsymbol{g}}$ and $N := \mathrm{tr}(F\Sigma_\mathrm{b}) + \mathrm{tr}(F\Sigma_\mathrm{w})$. In the enumerable testbed this predicts the true KL to within 4% over three orders of magnitude in η.

Drift splits into a part along the mean update, $\frac{1}{2}\eta^2\mathcal{G}$, and a diffusive part $\frac{1}{2}\eta^2 N/P$ that performs a random walk in function space. The fraction that is signal,

$$\rho := \frac{\mathcal{G}}{\mathcal{G} + N/P} = \frac{1}{1 + B_\mathrm{crit}/P}, \qquad B_\mathrm{crit} := \frac{N}{\mathcal{G}}, \tag{32}$$

is the RLVR analogue of the gradient noise scale, measured in prompts. It is a two-level quantity: a floor set by prompt diversity, plus a term that G divides down. A trainer can therefore read its own batch-size efficiency off the KL it already logs, without a sweep.

Assumption 1 (Regularity)**.** Throughout Section 8: (i) prompts are drawn independently and, given a prompt, responses are drawn independently from the current policy; (ii) the advantage rule belongs to the count-based family (3); (iii) $\tau_w := (G-1)\,\mathrm{tr}(F\Sigma_\mathrm{w})$ does not depend on G, which holds exactly for the Bernoulli term of Corollary 1 and is measured to hold within 6% over $G = 2$ to 32; (iv) η is small enough that the $O(\eta^3)$ term of (31) is negligible.

8.3 Progress is governed by the square root of the efficiency

Proposition 5 (Progress at a fixed drift budget)**.** *Under Assumption 1, let $a := \nabla J^\top \bar{\boldsymbol{g}}$ be the alignment of the estimator's mean with the true gradient. An update constrained to drift D improves the objective, to first order, by*

$$\mathbb{E}[\Delta J] = \sqrt{2D}\,\frac{a}{\sqrt{\mathcal{G}}}\,\sqrt{\rho}, \tag{33}$$

and a run of N_s such updates sharing a total drift budget D_tot accumulates first-order progress proportional to $\sqrt{N_\mathrm{s}\,\rho}$.

Proof. Solving (31) for η at drift D gives $\eta = \sqrt{2D/(\mathcal{G} + N/P)}$, and the first-order improvement of an update $\eta\hat{\boldsymbol{g}}$ is $\eta\,\mathbb{E}[\hat{\boldsymbol{g}}]^\top\nabla J = \eta a$. Substituting and using $\rho = \mathcal{G}/(\mathcal{G} + N/P)$ gives (33). Splitting D_tot equally over N_s updates sets $D = D_\mathrm{tot}/N_\mathrm{s}$ per update; since (33) is proportional to $\sqrt{D}$, the total is $N_\mathrm{s}\sqrt{2D_\mathrm{tot}/N_\mathrm{s}}\,(a/\sqrt{\mathcal{G}})\sqrt{\rho} \propto \sqrt{N_\mathrm{s}\rho}$. □

The estimator enters (33) only through $a/\sqrt{\mathcal{G}}$, the alignment of its mean with the true gradient in Fisher units, and the batch composition enters only through ρ. Because KL is quadratic in the step while progress is linear in it, many small steps travel further than one large step of the same total drift, which is why N_s appears under the square root.

8.4 The optimal split

Figure 12 is the result this subsection builds to. Hardware fixes the rollouts per step $R = PG$; the question is how to divide them. When a group shares its prompt's prefill, prefill is paid once per prompt and decode once per response, so a step costs $P(c_\mathrm{pre} + G\,c_\mathrm{dec})$. Write $\alpha := c_\mathrm{pre}/c_\mathrm{dec}$ for the prefill ratio and $\beta_b := \tau_b/\mathcal{G}$, $\beta_w := \tau_w/\mathcal{G}$.

Theorem 3 (Compute-optimal split)**.** *Under Assumption 1, maximising the progress of Proposition 5 per unit of compute at fixed R is equivalent to minimising*

$$f(G) \;=\; (\alpha + G)\Big(\frac{R}{G} + \beta_b + \frac{\beta_w}{G-1}\Big) \tag{34}$$

over $G \in (1, \infty)$. Then:

(i) f is strictly convex on $(1,\infty)$ and has a unique minimiser $G^\star$, which is the unique root of

$$\beta_b \;=\; \frac{\alpha R}{G^2} \;+\; \frac{(1+\alpha)\,\beta_w}{(G-1)^2}. \tag{35}$$

(ii) If rollouts cost the same however they are grouped ($\alpha = 0$), then

$$G^\star \;=\; 1 + \sqrt{\tau_w/\tau_b}, \tag{36}$$

exactly and independently of R.

(iii) For $\alpha > 0$, $G^\star$ is strictly increasing in both α and R, and

$$G^\star \;\to\; 1 + \sqrt{(1+\alpha)\,\tau_w/\tau_b} \quad \text{as} \quad \alpha R/G^2 \to 0. \tag{37}$$

(iv) The optimal integer group size is $\lfloor G^\star \rfloor$ or $\lceil G^\star \rceil$.

Proof. By Proposition 5, total progress under a compute budget C is proportional to $\sqrt{N_\mathrm{s}\rho}$ with $N_\mathrm{s} = C/\big(P(c_\mathrm{pre} + Gc_\mathrm{dec})\big)$, so $N_\mathrm{s}\rho \;\propto\; \big[(\alpha + G)(P + B_\mathrm{crit}(G))\big]^{-1}$ using (32). Substituting $P = R/G$ and $B_\mathrm{crit}(G) = \beta_b + \beta_w/(G-1)$ gives (34), and maximising $N_\mathrm{s}\rho$ is minimising f.

Expanding, $f(G) = \alpha R/G + G\beta_b + (1+\alpha)\beta_w/(G-1) + \mathrm{const}$, whence

$$f''(G) = \frac{2\alpha R}{G^3} + \frac{2(1+\alpha)\beta_w}{(G-1)^3} > 0 \qquad \text{on } (1,\infty),$$

so f is strictly convex. Since $f'(G) \to -\infty$ as $G \downarrow 1$ and $f'(G) \to \beta_b > 0$ as $G \to \infty$, f' has a unique zero, which is (35). This proves (i).

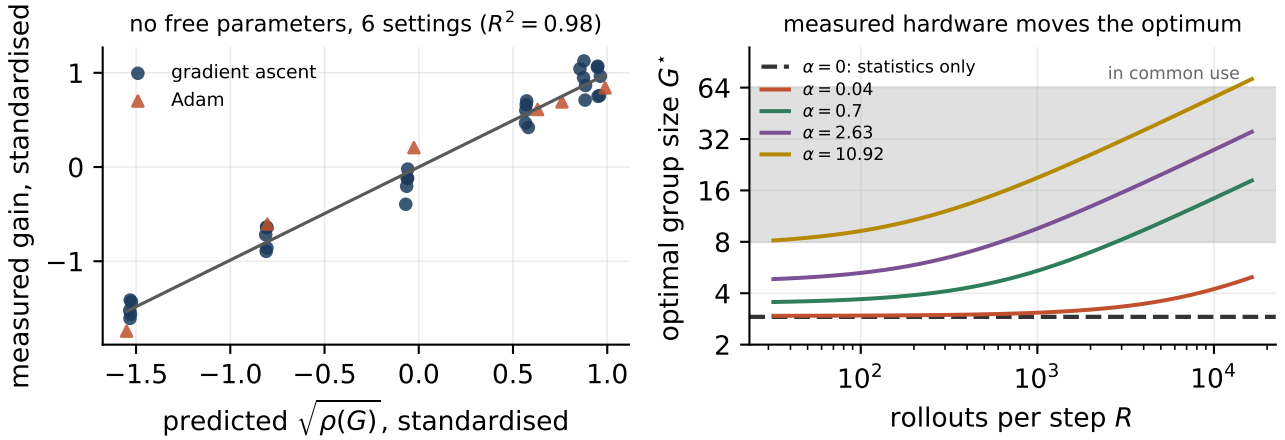


Figure 12: Left: the efficiency $\rho(G)$ predicted from two measured traces with no fitted parameters, against the gain actually obtained by training at that split. Settings differ in run length and task, so both axes are standardised within a setting; what is pooled is the shape. **Right:** the optimal group size from Theorem 3 at that measured noise ratio, against the rollout budget, for the prefill ratios α we measure on real hardware. Statistics alone say three; the hardware says the rest, and says something different for long prompts with short answers than for reasoning-length responses.

For $\alpha = 0$ the first term of (35) vanishes and the condition reduces to $\beta_b = \beta_w/(G-1)^2$, giving (36); R has dropped out, proving (ii). Writing (35) as $\Xi(G, \alpha, R) = 0$ with $\Xi := \beta_b - \alpha R/G^2 - (1+\alpha)\beta_w/(G-1)^2$, we have $\partial_G \Xi > 0$ at the root while $\partial_\alpha \Xi < 0$ and $\partial_R \Xi < 0$, so the implicit function theorem gives $\partial G^\star/\partial\alpha > 0$ and $\partial G^\star/\partial R > 0$. Dropping the $\alpha R/G^2$ term from (35) leaves $\beta_b = (1+\alpha)\beta_w/(G-1)^2$, which is (37). This proves (iii). Part (iv) follows from strict convexity, since a strictly convex function on an interval is minimised over the integers at one of the two integers bracketing its real minimiser. □

Part (ii) is the case in which the split is purely a statistical question, and there the answer is one plus the square root of the ratio of within-prompt to between-prompt gradient noise, with no dependence on how many rollouts the hardware affords. Both traces are measurable from a single batch (Section 5), so $G^\star$ is an observable rather than a hyperparameter.

Part (iii) is where the economics enter, and it is the reason published recipes are not obviously wrong. Prefill sharing pushes $G^\star$ up, and so does a larger rollout budget: a run with long prompts, short responses and a large batch can have an optimum in the tens even when the purely statistical optimum is three. Section 8.6 measures α on real hardware and finds it ranging from 0.04 to 10.9 depending only on the prompt-to-response length ratio, which moves $G^\star$ over an order of magnitude. The approximation (37) that we quote alongside is the small-batch limit and understates $G^\star$ badly when R is large; the root of (35) is what should be used, and costs one line of arithmetic.

The optimum moves during a run. Both τ_w and τ_b change as the policy improves. $\mathbb{E}_x[p(1-p)]$ falls once a task is largely solved, while prompt-to-prompt gradient heterogeneity falls as a model acquires a shared algorithm. Which of the two dominates is an empirical question, and the sweeps below find both directions in different settings.

8.5 The step size is a drift target

Equation (31) inverts to $\eta(D) = \sqrt{2D/(\mathcal{G} + N/P)}$, so a run can be specified by a target drift rather than a learning rate. Learning rate, clip range and KL coefficient are three controls acting on the single quantity D, which is why they interact and why none of them transfers between settings on its own. Drift is measured in nats per token and is a property of the policy's behaviour rather than of its parameterisation, which is the same reparameterisation-invariance argument that motivates the natural gradient [ichi Amari, 1998], used here to specify a step rather than to take one. Whether that makes it transfer is tested in Section 8.12, and the answer has a boundary.

The rest of this section is measurement: what the prefill ratio actually is on hardware, whether the predicted split matches the one that trains best, and where specifying a run by its drift stops working.

8.6 What the hardware is worth

Theorem 3 turns the split into a question about the serving stack as much as about the learning problem, through the prefill ratio $\alpha = c_{\text{pre}}/c_{\text{dec}}$. We measured both costs for a 0.5B model in bfloat16 on the machine

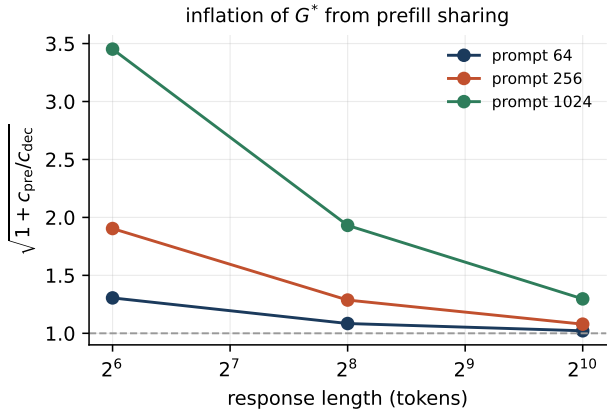


Figure 13: Measured prefill-to-decode economics on an Apple M4, as the group-size inflation implied in the small-batch limit. Long responses make prefill sharing almost worthless; short answers behind long prompts make it decisive.

everything else ran on. The ratio is governed almost entirely by the prompt-to-response length ratio, running from 0.04 when a short prompt is followed by a thousand tokens of response to 10.9 when a thousand-token prompt is followed by a short answer.

Figure 13 is what the hardware contributes, and Table 13 is the reconciliation the paper has been building towards. A purely statistical reading of the split says three; the settings in which published recipes were developed say otherwise, and the theorem says how much of that is justified. Long prompts with short answers at a large batch put the optimum in the tens, which is where the recipes are. Reasoning-length responses, where the response dwarfs the prompt, put it back at three to four, and there a group of sixteen is spending most of its budget on responses that a second prompt would have used better. The disagreement between a recipe and this analysis is therefore not evidence against either: it identifies the regime the recipe came from.

8.7 What transfers when the batch size changes

The sharpest consequence of the drift model is a statement about transfer. A run tuned by learning rate is tuned in a unit that depends on the batch through $\mathcal{G} + N/P$; a run tuned by drift is not. Table 14 sweeps both parameterisations at four batch sizes and reports what it costs to carry the value tuned at the smallest batch to the others.

Figure 14 shows both sweeps and the optima they imply. The single-step drift model accounts for part of the movement in η^* and not all of it: the coefficient $\mathcal{G} + N/P$ measured at the initial policy predicts a smaller shift than the one observed over a full run. The coefficient keeps moving as the policy improves, so a controller that re-reads it tracks the optimum while a learning rate fixed in advance cannot.

8.8 The drift model, and the efficiency curve on enumerable policies

The same enumerable policies that carried the forecast in Section 6 carry the allocation results, where what is being predicted is not a spread but a rate of progress.

The curvature model fails, the drift model does not. Taking a real update and computing the resulting improvement and KL exactly, the second-order model of the objective places its optimal step at least four times below the step that actually maximises improvement, which is where the sweep ended, because $J = \mathbb{E}[r]$ is not a log-likelihood and a log-linear policy keeps improving past the turnover of a quadratic model. The drift model (31) predicts the measured KL to between 2.6% and 3.9% across a thousandfold range of step sizes. The theory in Section 8 is built on the second of these, not the first.

Splitting a rollout budget. For each prompt pool the parameter-free efficiency curve $\rho(G)$ is compared against the improvement obtained by training at every split of a fixed budget, holding total rollouts, total drift and step count fixed. Table 15 groups pools by how much ρ varies across the splits on offer, and Figure 15 shows the four whose predictions discriminate most strongly.

8.9 Transformers trained from scratch

A population of independent transformers is warmed up on a verifiable running-sum task until each member reaches a common pass rate, the noise terms are measured at that frozen policy, and every member is then trained from the same starting point at each split of a fixed rollout budget. The corpus size is the knob: sampling without replacement from a corpus of N scales the between-prompt term by $1-(P-1)/(N-1)$, which moves $G^\star$ without touching the task. At a fixed rollout budget that factor depends on the group size being chosen, and (24) shows it *lowers* the optimum, by $\sqrt{(N-1)/N}$; the effect is under half a percent at the corpus sizes here, so what the sweep varies is mostly the measured noise terms themselves. Table 16 reports the predicted and measured optimum for each corpus, and Figure 16 the sweeps behind it.

8.10 Noise scales on a pretrained model

We repeat the measurement on Qwen2.5-0.5B-Instruct with LoRA adapters as the trainable parameters, over

$c_{\text{pre}}/c_{\text{dec}}$	$R = 64$	$R = 256$	$R = 1024$	$R = 8192$
0.00	2.9	2.9	2.9	2.9
0.04	3.0	3.0	3.1	4.0
0.70	3.6	4.0	5.5	13.1
2.63	5.0	6.2	9.7	25.1
10.92	8.7	11.5	19.2	51.0
small-batch limit (37) — $\alpha = 0.00$: 2.9, $\alpha = 0.04$: 2.9, $\alpha = 0.70$: 3.5, $\alpha = 2.63$: 4.6, $\alpha = 10.92$: 7.6				

Table 13: The optimum $G^\star$ from Theorem 3, at the noise ratio measured on our transformers, across measured prefill ratios and rollout budgets. Entries are the root of (35), which agrees with direct numerical minimisation of (34) to one part in 10^7. The last row shows the small-batch limit, which is what one would quote from (37) alone and which understates the optimum badly at large R.

prompts per step P	8	16	32	64
best step size η^*	0.0417	0.0786	0.1228	0.1615
best drift target D^*	5.06e-03	5.29e-03	5.15e-03	3.98e-03
cost of transferring η (%)	+0.0	-9.9	-17.9	-22.0
cost of transferring D^* (%)	+0.0	+0.0	+0.0	+0.0

Table 14: Transfer across batch size. The learning rate that is optimal at $P = 8$ loses a fifth of the achievable gain by $P = 64$; the drift target that is optimal at $P = 8$ remains optimal.

corpora of short exact-match tasks: counting a letter in a word, naming the k-th item of a list, adding two numbers, and a mixture of all of them. Measured optimal group sizes run from 2.7 to 6.2, the same range as the enumerable policies and the transformers trained from scratch.

Corollary 1 predicts that the within-prompt noise tracks the reward histogram at a fixed score scale, and difficulty buckets inside one family are where that assumption is tenable. We screen prompts with a larger sample than the measurement uses, so the bucket boundaries do not inherit the noise of the quantity under test, and report the pass rate re-measured during the probe rather than the one used to sort. Across buckets spanning pass rates from 0.25 to 0.30, the ratio $\tau_w/\mathbb{E}[p(1-p)]$ is constant to within 14% (Table 17).

Across *different* families the same ratio varies by a factor of 8.8 (Appendix E), and the reason is visible in the assumption: the constant the corollary predicts is the score scale σ_s^2, and a family whose answer is one number does not share a score scale with one whose answer is four. This bounds how the corollary should be used. It converts a reward histogram into a noise estimate within a task distribution, after one calibration measurement, and it does not transport that calibration to a task of a different shape.

8.11 Measuring the noise scales in Adam's metric

Everything above is under plain gradient ascent, where the update is the gradient and the noise can be read in the parameter metric. Real runs use Adam, which moves in a different metric and is scale invariant, so the theory only carries over if the noise is measured after preconditioning. Repeating the sweep under both optimisers gives Table 18.

The law survives, but measuring in Adam's metric required fixing two things, and both are consequences of the preconditioner being an estimated quantity rather than a fixed one.

First, a probe run at a frozen policy starts with an empty second-moment estimate, so there is no metric to measure in yet. Priming it – running the optimiser's state forward on real batches with a zero step size, so the policy does not move – gives the preconditioner a real run would have.

Second, and less obviously, dividing by $\sqrt{v}$ is unbounded. Coordinates that rarely receive gradient have a tiny second moment, which is harmless for the update, since Adam's update is bounded by construction, but ruinous for an inner product between two noisy gradients: those coordinates dominate it. Measured this way the estimator returned $\tau_b < 0$ and an inverted efficiency curve. Flooring the second moment at a fraction of its own mean before dividing fixes it, and the result is insensitive to where the floor is put: over three orders of magnitude in the floor, the measured $G^\star$ moves only from 3.51 to 3.15, against an observed optimum of 4. Without a floor the same measurement reports 11.9.

This is the one place where our quantities need care that the plain-gradient case does not. The working rule is to instrument the update rather than the gradient, and to floor the preconditioner before inverting it.

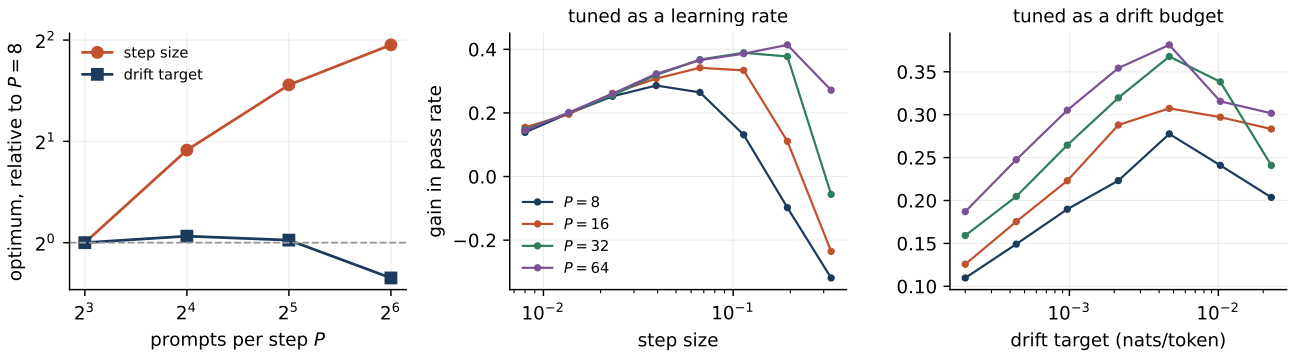


Figure 14: Left: the optimum of each parameterisation relative to its value at $P = 8$. Centre and right: the sweeps themselves. The learning-rate optimum moves; the drift optimum does not.

predicted spread $\max\rho/\min\rho$	pools	median R^2	median Spearman
1.00–1.15	8	0.670	+0.588
1.15–1.35	11	0.876	+0.833
1.35–2.00	6	0.971	+0.957
2.00–8.00	7	0.983	+0.978

Table 15: Agreement between the predicted efficiency and the measured gain in the enumerable testbed, as a function of how strongly the prediction discriminates between splits. The prediction has no free parameters. Pools where it predicts that the split hardly matters are the pools where the measured curves are flat to within their standard errors.

8.12 A prospective test, and where the drift target stops transferring

Everything so far tunes one quantity at a time on the setting it is measured on. The stronger test is prospective: fix a setting the method has never seen, let the probe choose G before any training happens, and check afterwards whether it chose well. We moved to a held-out setting with four times the rollout budget, eight times the corpus and a different seed. The probe reported $G^\star = 3.1$, so the recipe took $G = 4$; the comparison is against $G = 16$, which is what a default recipe would use.

The prospective choice was right. $G = 4$ beats $G = 16$ under both parameterisations. Under drift control, where the step size is held to the same behavioural budget in both arms and so cannot confound the comparison, it wins by 11.1% against a predicted $\sqrt{\rho_4/\rho_{16}} - 1 = 10\%$.

The drift target transfers across the group size exactly. Both group sizes put their optimum at the same drift target, 1.45×10^{-3}. Changing how the budget is split does not change how far the policy should move per update, which is what a behavioural unit ought to do.

It does not transfer across this change of budget. The tuning setting's optimum was 3.19×10^{-3}, a factor of 2.2 away, and carrying it over costs 20% of the achievable gain. Earlier the same target held to within 5% across a fourfold change in batch size and 21% across an eightfold one. A sixteenfold change in prompts per step, combined with a fourfold change in rollout budget, is past where a single number holds. We found this the hard way: packaging the two together and carrying both to the held-out setting lost to the default by 28.9% (paired $t = -3.01$ over 32 runs), and Table 19 is what attributes that loss to the drift target rather than to the group size.

A constant drift target is not the right schedule. At both group sizes a tuned constant learning rate beat the best constant drift, with its optimum at the edge of the grid. A fixed learning rate produces a drift that changes over a run as the gradient statistics change, and that this wins says the optimal drift trajectory is not flat. The unit appears to be right and the schedule is not, which is the most concrete thing this paper leaves open.

9 Related work

Run-to-run variance. That reinforcement learning results move under reseeding is old news in deep RL [Henderson et al., 2017] and has been documented again for language-model reasoning, where sampling parameters, prompt formatting and hardware all shift reported numbers [Hochlehnert et al., 2025]. Those papers establish the size of the problem by replication, and Colas et al. [2018] turn it into a power analysis that sizes the seed count from a pilot sample of seeds, which is the practice Section 7 replaces. Replication is not the only instrument: Heineman et al. [2025]

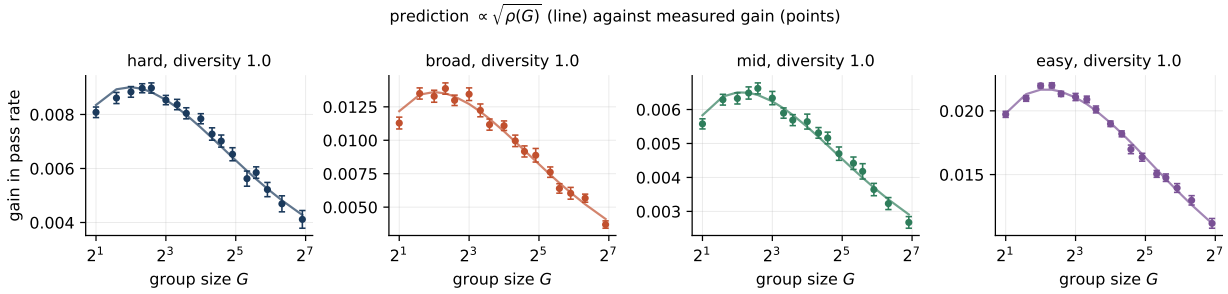


Figure 15: Measured gain against $\sqrt{\rho(G)}$ for the four pools whose predictions discriminate most strongly. Error bars are standard errors over seeds.

corpus N	τ_b	τ_w	predicted $G^\star$	best measured G	Spearman	R^2
128	32.1	137.1	3.06	2	+0.943	0.983
192	35.4	140.3	2.98	2	+0.943	0.954
512	37.6	146.8	2.97	2	+0.943	0.987
4096	37.7	141.8	2.94	4	+1.000	0.984
unbounded	37.3	140.0	2.94	4	+1.000	0.985

Table 16: Predicted and measured optimum on transformers trained from scratch. The sweep visits $G \in \{2, 4, 8, 16, 32, 64\}$, so a predicted optimum near three can only appear as a peak at two or four; those two are within one standard error of each other in four of the five corpora, and every corpus shows the predicted decline beyond $G = 8$. Spearman and R^2 are computed against the parameter-free prediction across the whole sweep.

estimate a benchmark's noise from the variability of scores across checkpoints of a single run, which serves choosing an evaluation, where ours is the across-seed spread of the training procedure. For replicated runs, Agarwal et al. [2021] prescribe how the resulting samples should be summarised, using interquartile means, stratified bootstrap intervals and performance profiles, and their prescription is what a practitioner should use once the runs exist. All of it presumes the runs exist. This paper supplies a model of where the variance comes from, so that a single run can be asked what its own spread would have been; the two are complementary, and where several runs are affordable their statistics remain the right ones.

Stochastic dynamics of training. Treating stochastic optimisation as a process whose covariance is set by the balance of gradient noise against curvature is standard [Mandt et al., 2017], and the Lyapunov recursion in Theorem 1 is not new as mathematics. What we add is the RLVR-specific measurement of Q_t through the two-level noise decomposition, the finite-time rather than stationary treatment, and the observation that the resulting filter, not the size of the noise, is what governs reproducibility here. What is not established, and what Section 6 tests, is whether it survives RLVR: the objective is non-stationary because the data distribution is the policy's own samples, the noise covariance depends on the pass-rate distribution and moves as the policy improves, and the reward is binary. None of that is in the setting the original analysis assumes, and the consequence for reproducibility, that the spread across seeds does not grow with the length of a run, does not appear to have been stated or measured.

Wang et al. [2026] arrive at a filtering picture of RLVR from the opposite side, reporting that weights and log-probabilities evolve close to linearly across model families and algorithms and attributing it to the noisy training signal acting as a low-pass filter along a low-dimensional drift. Their object is the trajectory of a single run; ours is the covariance between runs that differ only in seed, and the quantity we push it through is a reported metric rather than the parameters. The two are complementary, and their scales reach considerably further than ours: they report the linear regime across families and sizes we cannot train, which is worth weighing against the boundary Section 6.10 draws at half a billion parameters.

Entropy collapse. That RLVR drives policy entropy down sharply, often within the first updates, is documented and actively worked on [Cui et al., 2025]; the literature treats it as an optimisation pathology, since a policy that has stopped exploring has stopped learning. It has a second consequence that follows from (11) and does not appear to have been drawn. A count-based advantage is identically zero on a unanimous group, so a collapsed policy injects $\Sigma_t = 0$: the run stops accumulating seed uncertainty at all, and any

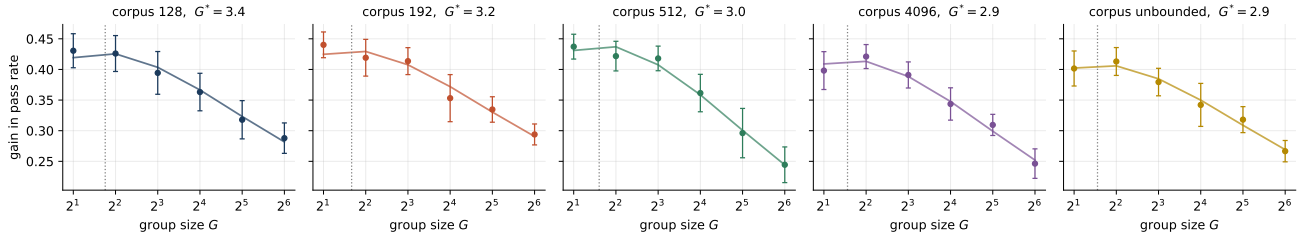


Figure 16: Gain against group size at a fixed rollout budget, with the parameter-free prediction overlaid and the predicted $G^\star$ marked.

bucket	screened p	measured p	$\mathbb{E}[p(1-p)]$	τ_w	$\tau_w/\mathbb{E}[p(1-p)]$, relative
0	0.160	0.253	0.163	1.15e+04	0.90
1	0.269	0.272	0.175	1.33e+04	0.96
2	0.380	0.304	0.185	1.66e+04	1.14

Table 17: Within-prompt noise across difficulty buckets of one task family. The gap between the screened and re-measured pass rates in the outer buckets is regression to the mean, which is why the screening sample is the larger of the two.

measurement of its reproducibility from that point forward is measuring nothing. Section 6.13 reports both halves of this on real models: a run that reaches it in a single update, and an evaluation that cannot see the difference between two runs that have.

Adjoint sensitivity. Carrying a scalar backwards through a trajectory to avoid forming a large intermediate is the adjoint method, standard in optimal control [Pontryagin et al., 1962] and reintroduced to machine learning for continuous-depth models [Chen et al., 2018]. Theorem 2 is that construction applied to a covariance rather than a loss: the backward variable is the metric gradient and what accumulates along the way is variance rather than gradient. The step that makes it cheap here is Proposition 1, which is specific to the estimators RLVR uses and removes the transpose an adjoint would otherwise need.

Linearised models and simulated training. That a network trained with small steps behaves like its linearisation at initialisation is the tangent-kernel or lazy regime [Jacot et al., 2018, Lee et al., 2019, Chizat et al., 2019], and Malladi et al. [2023] show that prompt-based fine-tuning of language models often sits close enough to it for the kernel to predict the fine-tuned model's accuracy. That literature uses the linearisation to predict where *one* run ends. Section 3.5 uses it to draw *many*: the object simulated is the stochastic policy-gradient run itself, with its sampling kept, and the quantity read off is the spread between seeds, which a deterministic kernel regression does not have. Closest in spirit is Simfluence [Guu et al., 2023], which fits a simulator of supervised fine-tuning runs from observed runs in order to attribute a loss trajectory to training examples; ours is derived from the scores at the starting point with no run observed, and its target is across-seed variance under reinforcement learning.

Uncertainty in a reported number. Practice for benchmark error bars is to treat the evaluation set as the only source of randomness and report a binomial or bootstrap interval over questions [Miller, 2024]. That is the right interval for a fixed model and the wrong one for a training procedure: it says nothing about what retraining would do. Bouthillier et al. [2021] make the case that the training pipeline's own randomness belongs in the interval and estimate it by replication. Theorem 2 estimates the same quantity without replicating, and the two components are separable rather than competing, and our real-model measurements subtract the binomial term explicitly to recover the seed term.

Batch size and gradient noise. Pretraining has a compute-allocation literature that tells a practitioner how to spend a budget before spending it [Kaplan et al., 2020]; RLVR does not, which is part of what Section 8 is for. The gradient noise scale predicts the batch size beyond which data parallelism stops buying speed, and does so from statistics measurable during training [McCandlish et al., 2018]; the empirical picture across tasks and optimisers is mapped in Shallue et al. [2018]. Both concern a single level of sampling. RLVR samples prompts and then samples responses within each prompt, and the resulting two-level noise scale, the split it implies, and the fact that the response-level term is readable off the reward histogram, have no counterpart in that literature. The variance decomposition itself is the classical two-stage sampling result [Cochran, 1977], applied here to policy

gain by group size	2	4	8	16	32	64	measured $G^\star$	R^2
gradient ascent	+0.409	+0.413	+0.390	+0.333	+0.300	+0.250	2.94	0.992
Adam	+0.519	+0.527	+0.514	+0.492	+0.447	+0.384	3.42	0.969

Table 18: The same sweep under gradient ascent and under Adam. The optimum is at $G = 4$ for both, and the predicted curve explains the measured one about as well. The measured $G^\star$ for Adam is taken in Adam's own metric, which takes some care – see below.

best achievable gain	$G = 4$	$G = 16$
tuning a learning rate	$+0.4947^*$	$+0.4780^*$
tuning a drift target	+0.4431	+0.3989
predicted efficiency ρ	0.815	0.674

Table 19: Best achievable gain at the held-out setting, crossing the group size the probe selected against the group size a default recipe would use, under both ways of setting the step. Starred entries sit at the edge of their grid and are lower bounds. The probe's choice wins under both.

gradients.

Published recipes. Open post-training recipes report the batch sizes, group sizes and learning rates they used [Lambert et al., 2024, DeepSeek-AI et al., 2025] without a stated principle for choosing them. Those settings were arrived at by tuning, so they are a record of which regimes practitioners actually train in. Section 8 uses them that way, to ask which prefill ratio and rollout budget would make a group of sixteen optimal, rather than as a test the theory could pass or fail.

Hyperparameter transfer. μP makes learning rates transfer across width by fixing what a step does to the network's functions rather than to its parameters [Yang and Hu, 2021, Yang et al., 2022]. This paper takes the same stance one level up: fix what a step does to the policy's *behaviour*, measured as drift in nats per token, and the recipe transfers across batch size. We find no comparable prescription for RL post-training in the literature; the closest work reports empirical scaling behaviour and fitted power laws [Tan et al., 2025, Khatri et al., 2025] rather than a quantity to hold fixed. Ghosh et al. [2026] go further, reducing GRPO under a mean-field assumption to a stochastically forced damped oscillator whose stationary fluctuation scales as $1/G$; that is the closest stochastic model of RLVR dynamics we know of, and it differs from ours in summarising the policy by one scalar and in treating the stationary rather than the finite-time spread.

Estimators for RLVR. RLOO [Ahmadian et al., 2024, Williams, 1992], GRPO [Shao et al., 2024], its unnormalised correction [Liu et al., 2025], DAPO [Yu et al., 2025] and sequence-level variants [Zheng et al., 2025] are usually compared by benchmark deltas. Proposition 3 shows that, for binary rewards, the members of this family differ only through a difficulty weight $\lambda(p, G)$, that a p-independent λ is absorbed exactly by the step size, and that standardisation is a change of objective rather than a variance reduction – a sharpening of the argument in Liu et al. [2025]. Recent analyses of why normalisation helps [Ge et al., 2026] and of what outcome-reward objectives cannot simultaneously be [Ding et al., 2026] are complementary: they characterise the estimator, while the question here is how many samples to give it and how far to step.

Allocating rollouts. Several recent methods vary the number of responses per prompt within a batch using heuristics tied to observed pass rates or gradient variance [Nguyen et al., 2026], outside language models the batch size itself has been adapted during training [Park, 2026], and one line of work argues for scaling the response count specifically [Hu et al., 2025]. These are allocation policies; what has been missing is the quantity being optimised. Theorem 3 supplies it, and the answer is a ratio of two measurable traces. This also explains why heuristics keyed to pass rate alone succeed on some corpora and not others: the pass rate determines τ_w and says nothing about τ_b. The two lines of work are largely orthogonal, since the language-model methods distribute responses across prompts at a group size someone else chose, while the question here is what that group size should be; the baseline we compare against is therefore the fixed G of a published recipe rather than an allocation rule.

Deciding when to stop repeating. Stopping a comparison as soon as one arm is convincingly ahead is sequential analysis [Wald, 1945], and its machine-learning form is racing: run every candidate on more data until a confidence bound separates them [Maron and Moore, 1993, Even-Dar et al., 2006], sharpened by using an empirical variance in the bound rather than a range [Audibert et al., 2009], and scaled to hyperparameter search by allocating a budget across candidates [Li et al., 2017]. Section 7 uses the simplest rule of that family, and deliberately so: the contribution is not a better stopping rule but the quantity the rule consumes. Every method in that literature obtains its

variance from the samples it has already drawn, which is sound when a sample is a validation example and unaffordable when a sample is a training run. At one run per arm those bounds are vacuous, and the experiment shows that the resulting floor, not the shape of the bound, is what the comparison actually pays. Supplying the variance from a model of the run's own dynamics is what removes it, and it is orthogonal to which bound is used afterwards.

Note also the difference in what is being allocated. The rollout-allocation methods above distribute responses *within* a batch, across the prompts of one update. The budget at issue here is across independent training runs of the same recipe, which is a level those methods do not address and which is where the reproducibility cost is actually incurred.

Trust regions and KL control. Constraining how far a policy moves is old [Schulman et al., 2015, 2017] and its modern LLM forms clip importance ratios or penalise divergence from a reference. The drift used here is neither: it is the KL between *consecutive* policies, used as a measurement and a control target rather than as a penalty, and its role in this paper is to supply the unit in which a recipe transfers.

Asynchrony and off-policy correction. A parallel literature studies what happens when the sampling policy and the training policy differ, whether through pipeline staleness [Song et al., 2026] or numerical mismatch between rollout and training engines. That work asks how much mismatch a run tolerates. This paper asks a question that arises before any mismatch: given that the sampler and the trainer agree, how should the sampling budget be divided, and how large a step should the result support.

10 Discussion

What this establishes. That the across-seed spread of a reported RLVR score can be forecast from a single training run, to a median factor of 1.10 over 192 settings where the comparison can be made exactly; that the mechanism is filtering rather than accumulation, and that the filter is the shared objective and not an artefact of runs starting nearby; and that the backward pass the forecast needs is cheap because the mean update of a count-based estimator is a gradient field. The practical readings are that run length need not produce linear growth in the spread, since under a stable enough filter later updates saturate rather than accumulate; that batch composition does move it, with square-root returns in the standard deviation; and that response sampling is the larger source at the small and moderate group sizes we measure, with prompt selection taking the majority somewhere between groups of eight and sixteen.

One quantity runs through all of it. Read together, the measurements in this paper keep returning to $\mathbb{E}_x[p(1-p)]$, the mean within-prompt reward variance. Under the score-scale conditions of Corollary 1 it tracks the dominant Bernoulli contribution to the response-level term Σ_{w}, which is a useful observable rather than a determination of Σ_t: it says nothing about the between-prompt term Σ_{b}, the conditional score geometry, or the full covariance. Within that scope it indexes how much seed noise a batch injects. It also indexes the learning signal, so it indexes how much of that noise the following updates filter. It is largest before a policy has learned anything, which is why the per-update kernel tilts towards the start of a run. And when it reaches zero the injection stops and the metric stops resolving the policy, so two of the boundaries below are the same boundary. A practitioner who logs one number from this paper should log that one.

What "filtered" does and does not mean. It would be tidier if the transfer operator were a contraction. It is not: its spectral radius exceeds one throughout a run, only a third to a half of directions contract, and the contracting timescales differ among themselves by more than an order of magnitude. Divergence stays bounded over the horizons anyone trains because expansion is slow relative to run length, a factor of about 1.6 over twenty-five updates, and not because perturbations are destroyed. A run long enough would leave this regime, and we cannot say from here how long that is. The recursion is indifferent to the distinction, which is the argument for carrying the operator rather than a summary of it; the prose is not, which is why we say filtered and not contracted.

On parameter space. S_T lives in parameter space, and a covariance there is not obviously about anything a reader of a paper cares about, since a reparameterisation could inflate or shrink it without changing the policy. That objection is answered by what the forecast predicts rather than by an argument. (11) contracts S_T against ∇M, so what is being tested is a functional of the policy: the standard deviation of a pass rate, in the units the pass rate is reported in. A quantity that were merely a coordinate artefact could not predict that to $1.10\times$ over 192 settings. The same contraction is why policy divergence and metric spread come apart in Section 6.13: they weight the directions of S_T differently, and one of the two weightings can be identically zero.

What the sizing result is not. The stopping rule in (30) is not anytime-valid. Testing after every seed and stopping on the first crossing inflates the error of a fixed-sample test, so its nominal z carries no coverage guarantee, and we make no claim on one. What Section 7 reports is the mistake rate the rule actually realises, measured against an ordering fixed on seeds no policy may draw, which is the quantity a practitioner spending runs cares about. Every policy is graded by the same measurement and separated at a common realised rate, so the comparison does not depend on the boundary being calibrated. Substituting a sequentially valid boundary would move all of these policies together, since the deviation enters each of them the same way, and we would expect it to change the constants and not the ordering. We have not verified that.

Two further limits are structural. Pairs whose ordering is ambiguous over the reserved seeds are excluded, so the benchmark says nothing about near-ties, where the honest answer is to keep training whatever rule is in use. And the decision is over one scalar metric; a comparison made on several metrics at once is a different problem, and a forecast per metric is necessary for it but plainly not sufficient.

What it does not establish. The forecast is validated where the metric, its gradient and the injected covariance are all exact. On a real model each becomes an estimate, and one of the three is biased in a direction we can name. Writing the estimated metric gradient as $\hat{b} = \nabla M + \epsilon$ with ϵ the sampling error of one rollout on the evaluation set,

$$\mathbb{E}\big[\operatorname{Var}(\hat{b}^\top \hat{g})\big] = \operatorname{Var}(\nabla M^\top \hat{g}) + \mathbb{E}\big[\operatorname{Var}(\epsilon^\top \hat{g})\big], \quad (38)$$

so a noisy metric gradient makes the forecast too large, never too small. The second term shrinks with the size of the evaluation rollout and can be estimated by splitting it, which we did not do.

What it establishes it only for runs that barely move. The settings behind that median gain at most 0.036 in pass rate. With the step size raised until runs gain more, and every input still exact, the same forecast is more than 3× off in 44.1% of 354 settings and short by up to 1715× (Section 6.5). That is linear response about one run meeting a run that has saturated, and it reproduces on the exact tier the shortfall first seen on a pretrained model. The estimator that survives those settings linearises the model instead of the run and simulates the seeds (Section 3.5): with no run trained it is within 1.5× in 74.9% of them, about what ten real seeds achieve. It is validated on the enumerable tier only, where a tabular policy and its tangent model span the same family, and whether it holds on a network is the first thing we would test.

And on a real model it underpredicts, by a lot. Section 6.9 runs the whole thing on Qwen2.5-0.5B and the estimator we recommend comes out 25× short, the transported one 16×. Most of those runs, the one the forecast was made from included, learn to answer one constant to every prompt, and which constant a run falls into is most of the spread: a discrete outcome a linearised model has no term for (Section 6.9). What survives the failure is narrower than we first wrote. The share of prompts whose group is not unanimous falls from about a half to under 5% by the fifth update, and a unanimous group carries no count-based advantage at all, so the run stops injecting seed noise there. That is measured directly and does not depend on the forecast. The forecast's own kernel puts 90% of its predicted variance before the same point, which is consistent with it, but the kernel belongs to the model whose magnitude is wrong by 25×, so we do not read it as establishing where the actual seed variance was created. Doing that needs an experiment that randomises one block of updates at a time, which we did not run. assumption that the amplification stays linear. At a step size small enough to keep it linear, the seed term drops below what eight runs can resolve at all.

We would rather report this than a number from a setting chosen to make it work. It also sets the next experiment precisely, though not as directly as we would like: the residual of Section 6.4 is measured between two runs, so it diagnoses a pair after the fact rather than certifying a forecast made from one. Turning it into a check a single run can apply to itself means perturbing a stored state along a direction drawn from its own gradient noise, at a radius matched to one update, and we have not done that.

On the size of the models. The largest measurement here is Qwen2.5-7B with eight seeds over twenty updates. That is enough to say that a 7B model's reported number moves under reseeding by an amount we can measure, and that the KL between its policies, measured per response token on one fixed batch of contexts rather than sampled along each policy's own trajectories, rises over the first ten updates and then stops, with a tail slope of +0.09 against the +1 accumulation requires; it is not enough to say anything about a run of several hundred updates on a reasoning model. We did not run that, and not because it was overlooked: every experiment in this paper was executed on a single laptop, which is enough for exact validation on solvable systems, for populations of small transformers, and for eight seeds at 7B, and is not enough for that. We would rather state the boundary than gesture past it.

Two zeros that mean different things. The most immediately usable thing in this paper may be a warn-

ing rather than a method. A run can report a zero seed spread for two reasons that look identical from outside. The response distribution can collapse, in which case $\Sigma_t = 0$ and no seed uncertainty is generated after that point; or the evaluation can saturate, in which case $\nabla M = 0$ on the evaluated distribution and no seed uncertainty is detected. Section 6.13 exhibits both on real models, the first within a single update at a step size a practitioner might well pick. Neither is evidence that the training procedure is reproducible, and both are cheap to rule out: log the fraction of prompts whose group is not unanimous during training, and $\mathbb{E}_x[\hat{p}(1-\hat{p})]$ on the evaluation set at the end. If either is zero, so is the error bar, and it means nothing. Neither reading needs the true pass rates: it is enough that the measured ones have stopped varying, since that alone is what makes the reported number uninformative.

A correction, and what it moved. We reported the 7B forecast as short by a factor of 38 and read that as the runs having left the regime a local model describes. The forecast behind it carried no adjoint between updates, which the exact tier says is worth a factor of 16 in variance on its own. Carrying Adam's moments backwards moves the same forecast by 8.7, to a shortfall between 3.1 and 17. We were reporting our own missing transport as a property of the runs, and only a reviewer's insistence that the two be separated made us check. The lesson we would pass on is narrow and practical: an ablation that is cheap in the exact tier should be run before its analogue is interpreted on a real model, because there is no other way to tell an implementation gap from a finding.

What the forecast costs, which is the objection we cannot answer. Estimating the injected term from the eight prompts a run trains on gives two forecasts 681× apart, because only about a tenth of groups are not unanimous and eight prompts rarely contain one. Raising that to 32 prompts brings the disagreement between disjoint halves to 5.5× and costs 141 minutes against the 260 that training all 10 seeds costs. A diagnostic consuming half the experiment it replaces is not yet a diagnostic. The correction above changes where the remaining effort belongs rather than removing this objection: the recursion is now the part that works, and what limits the forecast is how well a sparse sample pins down a variance to which most prompts contribute exactly zero. A unanimous group has an identically zero count-based advantage, so an estimator that spends its rollouts finding the live groups and its backward passes only on those is the obvious next thing to build, and we have not built it.

Adam. Lifting the recursion to $z = (\theta, m, v)$ works, at 1.10× at the median and 1.8× at worst, and the optimiser state is most of the problem: a recursion on θ alone is wrong by 24.6×, which is almost exactly what assuming no filtering at all gives. We report this as a correction. An earlier version of this work found the lift failing by 8× wherever the pass rate saturated and read that as Adam's normalisation being a nonlinearity a linear recursion cannot contain. The cause was instead that our mean map squared the mean gradient where the mean of the stochastic map carries $\mathbb{E}[\hat{g}^2]$, discarding a variance term worth six to fifteen times the term kept. It vanishes last where the mean gradient vanishes first, which is what made it look like a property of saturation. Section 6.8 gives the corrected numbers by band.

Limits of the linearisation, and a diagnostic that does not yet transfer. (5) requires runs to stay close enough that the mean gradient field is locally linear over the distance seed noise moves them. Section 6.4 measures that between pairs of runs, and Section 6.11 builds the version a single run can compute about itself. Within the enumerable grid it ranks failures above chance, flagging forecasts wrong by more than 1.3× with an AUROC of 0.78 (0.85 on the replicate grid), but it does not bound the worst case: refusing the fifth of settings with the largest residual leaves the worst error at 1.56×, unchanged, because the worst setting is not among those refused. A claim that it caps the tail, made on a smaller grid, is withdrawn.

Its absolute scale does not transfer to pretrained models, and we say that plainly rather than presenting two correct refusals as validation. Every real-model residual we measure lies between 2 and 160 where the calibration grid spans 0.0044 to 0.0488, and the mixed-support run of Section 6.10, whose forecast is the best in this paper, is refused along with the two worst. Its coverage on real models is therefore zero. The likely reason is visible in the statistic: (29) is a second difference, which on a tabular policy measures curvature and on a pretrained model measures curvature plus an amplified Monte Carlo floor we have not subtracted. A usable version would normalise by that floor, and calibrating it would need more than the four pretrained runs whose answers we already know. We report the present statistic as evidence that local curvature carries information about the tail of forecast error in a controlled setting, and as an unsuccessful first attempt at a cross-scale validity test.

Against drift targeting, in one case. Section 8 recommends specifying a run by a drift target rather than a learning rate, and Section 8 shows that target transferring across batch size where a learning rate

does not. The same technique destroys two of our runs. The mechanism is specific: $\mathbb{E}_x[p(1-p)] \to 0$ as a task is solved, the signal goes with it, and a controller holding drift constant answers by raising the step without bound. Any drift-targeted schedule needs a floor on the signal or a cap on the step; ours had neither. The defensible claim is that behavioural drift is a more invariant coordinate for update magnitude, not that a constant value of it is a good schedule.

A claim we did not make. An earlier draft of this work reported that the outcome of a run is largely settled by its first few batches, on a correlation of 0.88 between early pass rate and final score. Repeating the experiment gave 0.65, and repeating it again gave +0.44; the bootstrap interval at eight runs spans $[-0.19,\ +0.96]$, and at seven billion parameters the same correlation is +0.11. The number was real each time and the claim was not. We record it because the failure is instructive: eight runs make a correlation look like a finding, and only the third run of the same experiment made it obvious that it was not.

What we would do next, in order. Run the simulated seeds of Section 3.5 on the pretrained corpora whose real seeds are already on disk. The test is recorded with its thresholds and needs no training run: one teacher-forced pass per prompt and candidate response at the starting adapters, then the recipe simulated. It can fail in three ways the enumerable tier could not show, since a transformer's log-probabilities are not affine in its parameters, LoRA's are bilinear, and a simulated run can only give responses the base model already gives. Then establish whether stochastic support is causally responsible for the difference between 25× and 2.0, rather than associated with it. Changing the corpus changed the pass-rate distribution, the live share, the score geometry and both noise terms together, so the comparison implicates the measurement regime without isolating it. The experiment that would isolate it holds the model, the optimiser, the budget and the update count fixed and varies only the target live share across several pre-specified bands, asking whether the forecast error moves with it monotonically.

We would not spend the next compute on another pretrained corpus search. One candidate bias is already excluded: the estimation error in ∇M contributes a positive term $\mathbb{E}[\epsilon^\top \Sigma \epsilon]$ to the injected quantity, which is the sign of our remaining overprediction, but cross-fitting two independent metric gradients to remove it moves the mixed-support forecast from 0.0604 to 0.0616 and the ratio not at all. Whatever the residual factor of two is, it is not that, and the curvature residual of 2.7 on the same run points at the linearisation itself.

Beyond that, the forecast should carry its own interval rather than a point value, and the real-model spread should be estimated with a procedure whose coverage is calibrated rather than nominal. Only then is what we have a tool rather than a mechanism with a validated special case and a measured boundary.

How this work was done. The author chose the research questions, decided which experiments to run, judged the results, decided which claims to keep, correct or withdraw, and derived and checked parts of the theory. AI coding agents (Claude Code) wrote most of the code and much of the text, ran the experiment sweeps under that direction, and in a later revision found several of the errors this version corrects. Every number in the tables and in the generated macros is computed from the stored run records released with the code.

References

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, 2021.

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs. *arXiv preprint arXiv:2402.14740*, 2024.

Jean-Yves Audibert, Rémi Munos, and Csaba Szepesvári. Exploration-exploitation tradeoff using variance estimates in multi-armed bandits. *Theoretical Computer Science*, 410(19):1876–1902, 2009.

Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Naz Mohammadi Sepahvand, Edward Raff, Kanika Madan, Vikram Voleti, Samira Ebrahimi Kahou, Vincent Michalski, Tal Arbel, Chris Pal, Gaël Varoquaux, and Pascal Vincent. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems (MLSys)*, 2021.

Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems*, 2018.

Lénaïc Chizat, Edouard Oyallon, and Francis Bach. On Lazy Training in Differentiable Programming. In *Advances in Neural Information Processing Systems*, 2019.

William G. Cochran. *Sampling Techniques*. John Wiley and Sons, 3rd edition, 1977.

Cédric Colas, Olivier Sigaud, and Pierre-Yves Oudeyer. How Many Random Seeds? Statistical Power Analysis in Deep Reinforcement Learning Experiments. *arXiv preprint arXiv:1806.08295*, 2018.

Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, Zhiyuan Liu, Hao Peng, Lei Bai, Wanli Ouyang, Yu Cheng, Bowen Zhou, and Ning Ding. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025.

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanjia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948*, 2025.

Fei Ding, Yongkang Zhang, Yuhao Liao, Zijian Zeng, and Huiming Yang. On the Impossibility of Unbiased and Length-Invariant Policy Optimization with Outcome Rewards. *arXiv preprint arXiv:2607.23364*, 2026.

Eyal Even-Dar, Shie Mannor, and Yishay Mansour. Action elimination and stopping conditions for the multi-armed bandit and reinforcement learning problems. *Journal of Machine Learning Research*, 7: 1079–1105, 2006.

Cheng Ge, Caitlyn Heqi Yin, Hao Liang, and Jiawei Zhang. Why GRPO Needs Normalization: A Local-Curvature Perspective on Adaptive Gradients. *arXiv preprint arXiv:2601.23135*, 2026.

Rajat Ghosh, Datta Nimmaturi, Aryan Singhal, Vaishnavi Bhargava, Henry Wong, Johnu George, and Debojyoti Dutta. Predictable GRPO: A Closed-Form Model of Training Dynamics. *arXiv preprint arXiv:2606.30789*, 2026.

Kelvin Guu, Albert Webson, Ellie Pavlick, Lucas Dixon, Ian Tenney, and Tolga Bolukbasi. Simfluence: Modeling the Influence of Individual Training Examples by Simulating Training Runs. *arXiv preprint arXiv:2303.08114*, 2023.

David Heineman, Valentin Hofmann, Ian Magnusson, Yuling Gu, Noah A. Smith, Hannaneh Hajishirzi, Kyle Lo, and Jesse Dodge. Signal and Noise: A Framework for Reducing Uncertainty in Language Model Evaluation. *arXiv preprint arXiv:2508.13144*, 2025.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep Reinforcement Learning that Matters. *arXiv preprint arXiv:1709.06560*, 2017.

Andreas Hochlehnert, Hardik Bhatnagar, Vishaal Udandarao, Samuel Albanie, Ameya Prabhu, and Matthias Bethge. A Sober Look at Progress in Language Model Reasoning: Pitfalls and Paths to Reproducibility. *arXiv preprint arXiv:2504.07086*, 2025.

Jian Hu, Mingjie Liu, Ximing Lu, Fang Wu, Zaid Harchaoui, Shizhe Diao, Yejin Choi, Pavlo Molchanov,

Jun Yang, Jan Kautz, and Yi Dong. BroRL: Scaling Reinforcement Learning via Broadened Exploration. *arXiv preprint arXiv:2510.01180*, 2025.

Shun ichi Amari. Natural gradient works efficiently in learning. *Neural Computation*, 10(2):251–276, 1998.

Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural Tangent Kernel: Convergence and Generalization in Neural Networks. In *Advances in Neural Information Processing Systems*, 2018.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling Laws for Neural Language Models. *arXiv preprint arXiv:2001.08361*, 2020.

Devvrit Khatri, Lovish Madaan, Rishabh Tiwari, Rachit Bansal, Sai Surya Duvvuri, Manzil Zaheer, Inderjit S. Dhillon, David Brandfonbrener, and Rishabh Agarwal. The Art of Scaling Reinforcement Learning Compute for LLMs. *arXiv preprint arXiv:2510.13786*, 2025.

Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Oyvind Tafjord, Chris Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, and Hannaneh Hajishirzi. Tulu 3: Pushing Frontiers in Open Language Model Post-Training. *arXiv preprint arXiv:2411.15124*, 2024.

Jaehoon Lee, Lechao Xiao, Samuel S. Schoenholz, Yasaman Bahri, Roman Novak, Jascha Sohl-Dickstein, and Jeffrey Pennington. Wide Neural Networks of Any Depth Evolve as Linear Models Under Gradient Descent. In *Advances in Neural Information Processing Systems*, 2019.

Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52, 2017.

Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-Like Training: A Critical Perspective. *arXiv preprint arXiv:2503.20783*, 2025.

Sadhika Malladi, Alexander Wettig, Dingli Yu, Danqi Chen, and Sanjeev Arora. A Kernel-Based View of Language Model Fine-Tuning. In *International Conference on Machine Learning*, 2023.

Stephan Mandt, Matthew D. Hoffman, and David M. Blei. Stochastic Gradient Descent as Approximate Bayesian Inference. *arXiv preprint arXiv:1704.04289*, 2017.

Oded Maron and Andrew W. Moore. Hoeffding races: Accelerating model selection search for classification and function approximation. In *Advances in Neural Information Processing Systems*, 1993.

Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. An Empirical Model of Large-Batch Training. *arXiv preprint arXiv:1812.06162*, 2018.

Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations. *arXiv preprint arXiv:2411.00640*, 2024.

Hieu Trung Nguyen, Bao Nguyen, Wenao Ma, Yuzhi Zhao, Ruifeng She, and Viet Anh Nguyen. Adaptive Rollout Allocation for Online Reinforcement Learning with Verifiable Rewards. *arXiv preprint arXiv:2602.01601*, 2026.

Jongchan Park. Scalable Reinforcement Learning via Adaptive Batch Scaling. *arXiv preprint arXiv:2605.21557*, 2026.

L. S. Pontryagin, V. G. Boltyanskii, R. V. Gamkrelidze, and E. F. Mishchenko. *The Mathematical Theory of Optimal Processes*. Interscience Publishers, 1962.

John Schulman, Sergey Levine, Philipp Moritz, Michael I. Jordan, and Pieter Abbeel. Trust Region Policy Optimization. *arXiv preprint arXiv:1502.05477*, 2015.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Christopher J. Shallue, Jaehoon Lee, Joseph Antognini, Jascha Sohl-Dickstein, Roy Frostig, and George E. Dahl. Measuring the Effects of Data Parallelism on Neural Network Training. *arXiv preprint arXiv:1811.03600*, 2018.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*, 2024.

Jingwei Song, Haofeng Xu, Jie Xiao, Chengke Bao, Jingwei Shi, Pengbin Feng, Yuhang Han, Weixun Wang, Eric Yang, and Tianyu Shi. Scaling Laws for Collapse in Asynchronous GRPO. *arXiv preprint arXiv:2607.01083*, 2026. Version 3; versions 1 and 2 were titled "Staleness-Learning Rate Scaling Laws for Asynchronous RLHF".

Zelin Tan, Hejia Geng, Xiaohang Yu, Mulei Zhang, Guancheng Wan, Yifan Zhou, Qiang He, Xiangyuan Xue, Heng Zhou, Yutao Fan, Zhongzhi Li, Zaibin Zhang, Guibin Zhang, Chen Zhang, Zhenfei Yin, Philip Torr, and Lei Bai. Scaling Behaviors of LLM Reinforcement Learning Post-Training: An Empirical Study in Mathematical Reasoning. *arXiv*

preprint arXiv:2509.25300, 2025.

Abraham Wald. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2): 117–186, 1945.

Tianle Wang, Jiayu Liu, Zhongyuan Wu, Shenghao Jin, Wei Chen, Hao Xu, and Ning Miao. Linear Dynamics in the RLVR Training of Large Language Models. *arXiv preprint arXiv:2601.04537*, 2026.

Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.

Greg Yang and Edward J. Hu. Tensor Programs IV: Feature Learning in Infinite-Width Neural Networks. *International Conference on Machine Learning*, 2021.

Greg Yang, Edward J. Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer. *arXiv preprint arXiv:2203.03466*, 2022.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. *arXiv preprint arXiv:2503.14476*, 2025.

Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group Sequence Policy Optimization. *arXiv preprint arXiv:2507.18071*, 2025.

Hongyi Zhou, Kai Ye, Erhan Xu, Jin Zhu, Ying Yang, Shijin Gong, and Chengchun Shi. Demystifying Group Relative Policy Optimization: Its Policy Gradient is a U-Statistic. *arXiv preprint arXiv:2603.01162*, 2026.

A Proofs of the exact moment results

Proposition 3

Proof. Condition on r_j. The other $G-1$ rewards are i.i.d. Bernoulli(p) and, given r_j, are independent of $\boldsymbol{s}(y_j)$, so $\mathbb{E}[w(k,r_j)\boldsymbol{s}(y_j)] = \sum_r \Pr[r]\, W_r\, u_r$. Summing over j and applying (2) gives $\mathbb{E}[z] = p\,(W_1 - W_0)\,u_1 = \lambda h$. □

Proposition 4

Proof. Split $\mathbb{E}[zz^\top]$ into diagonal and off-diagonal terms. The diagonal contributes $G^{-1}[pQ_1M_1 + (1-p)Q_0M_0]$ with $M_r = S_r + u_r u_r^\top$. For $j \neq l$, condition on (r_j, r_l) and on the count m' among the remaining $G-2$ responses; the two scores are then conditionally independent, giving $\frac{G-1}{G}\sum_{a,b}\Pr[a]\Pr[b]\,C_{ab}\,u_a u_b^\top$, which collapses to $\frac{G-1}{G}p^2(C_{11} - 2C_{10} + C_{00})U$ by (2). Subtracting $\mathbb{E}[z]\mathbb{E}[z]^\top = \lambda^2 p^2 U$ completes the proof. □

B Reproducing the results

Every figure and table is regenerated from stored run records by `analysis/figures.py` and `analysis/tables.py`, and so is every headline number in the prose: they are emitted as macros into `tables/numbers.tex` and referenced rather than typed, so a rerun that moves a number moves it in the text as well. Each run writes a manifest holding the configuration, its hash, the commit the code was at, and the interpreter and platform it ran on.

The exactness tests in `tests/` check every analytic result in Section 4 against brute-force enumeration, and check the batched estimator against the reference implementation term by term. Theorem 2 is additionally checked against the same quantity assembled the expensive way, by building S_T explicitly and forming $\nabla M^\top S_T \nabla M$, and Proposition 1 against the measured asymmetry of $\mathcal{J}$, which is 3×10^{-10} across the estimator family, at the finite-difference floor.

Evaluation randomness, and a bracket instead of a rerun. Recovering a seed term from the spread of an estimated score means subtracting the evaluation's own binomial variance, and that subtraction assumes each run drew its own evaluation. The seven-billion runs were scored before we noticed this, sharing one draw. Re-scoring them costs hours at that size, and is unnecessary: shared evaluation noise is common to every run and so drops out of the variance between them, while independent noise adds to it, which brackets the seed term between the observed spread and the binomially corrected one whatever was actually shared. At 7B those two ends differ by 0.3%, because the binomial term is small against a spread that large. The smaller model was re-run with independent draws, where the correction matters more.

One thing that was not reproducible, and now is. The LoRA adapters are attached by `mlx_lm`, which draws their A matrices from a generator that is not seeded by default. Every process therefore began

from a different parameterisation. The policy was the same either way, because the B matrices start at zero, and every seed within a run shares one initialisation, so the spreads reported here are unaffected: the runs being compared differ only in rollout randomness, as claimed. What was affected is repetition. Rerunning the same configuration gave different numbers, which we noticed when a rerun of one experiment disagreed with itself. The initialisation is now seeded from the run's configuration, and the released tests check that two attachments at one seed agree and that two seeds do not.

What each thing costs. Everything here ran on one Apple M4 laptop with 24 GB of unified memory, and the costs are worth stating because they bound what a reader can repeat. The enumerable-policy experiments are CPU work and finish in minutes; the 192-setting forecast grid, which trains an anchor and 47 further runs in each of them, takes about half an hour. The populations of small transformers take a few hours each. On the pretrained models the arithmetic changes: a single Qwen2.5-0.5B run of twelve updates takes about three minutes and its backward pass about ten, while one Qwen2.5-7B run of twenty updates takes around an hour, so the eight-seed measurement at 7B is most of a day. That last figure is the reason this paper stops where it does.

Using the backward pass elsewhere. Algorithm 1 needs two things from a training run and nothing else: the variance across prompts of the batch gradient projected onto a direction, and a Jacobian-vector product of the mean update field with that direction. The released code states that as a protocol rather than a class hierarchy, so a trainer in another framework implements two methods and reuses the recursion. Two implementations ship with it, one for enumerable policies where every term is exact and one for a pretrained model through MLX. Both are tested against systems whose answer is available in closed form: the protocol against linear Gaussian recursions, and the pretrained-model implementation against a stand-in with the same interface whose mean update field is a fixed linear map, which exercises the finite-difference Hessian-vector product without needing a model to run it on.

One numerical detail is worth recording. The difference step ϵ has a floor set by the precision the weights are held in: below roughly 10^{-4} in float32 the two evaluations cancel into rounding error rather than into a derivative. We use 10^{-3}, which sits above that floor and well below the scale on which the gradient field bends.

A claim index. The released code carries `docs/claims.md`, which lists every claim in this paper against the experiment that produced it and how strongly it is supported: exact, measured with an interval, observed without one, or a negative result. It also lists what we withdrew and what we do not claim. It is written for a reader who distrusts one particular number and wants to find its source without reading the code.

Which experiment produces what.

- `p1_propagation`: the covariance recursion against Monte Carlo (Table 5, Figure 1 right).
- `p2_metric_forecast`: the prospective forecast (Table 2, Figures 1 left and 2).
- `p3_memory_sources`: attribution and the memory horizon (Figure 3, Section 6.1).
- `p4_linearity`: the linearisation residual (Section 6.4).
- `p5_adam_lift`: the lifted recursion (Table 8).
- `p6_spectrum`: the spectrum of the transfer operator (Table 7).
- `s1`, `s3`, `s5`: seed divergence on transformers and on a pretrained model (Figures 6 and 5).
- `s6_scale`: the seven-billion-parameter measurements (Section 6.13).
- `s7_real_forecast`: the backward pass on a pretrained model, and `power.py` the simulation of what that design can resolve (Appendix C).
- `s8_null`: the same run with the reward replaced by a coin (Table 6, Figure 4).
- `a`, `b`, `c`, `d`, `t`: the estimator, allocation and cost-model results of Section 8.

The exact-tier experiments run in minutes on a laptop CPU. The pretrained-model experiments need Apple Silicon and MLX; the seven-billion-parameter runs need about 24 GB of unified memory, and the eight-seed sweep takes several hours.

C What the real-model design can resolve

The seed spread of a score estimated from a finite evaluation is recovered by subtracting the evaluation's own binomial variance from the observed spread across runs. That is a difference of two noisy quantities, and below some true spread it is not distinguishable from zero. Reporting a number from it without knowing where that boundary lies would be reporting noise.

	6 runs		8 runs	
true s.d.	resolved	estimate	resolved	estimate
0.000	50%	0.000	46%	0.000
0.010	58%	0.007	72%	0.009
0.020	83%	0.018	88%	0.018
0.040	99%	0.034	99%	0.039
0.080	100%	0.069	100%	0.076

Table 20: Simulating the design as run, with 48 held-out questions and 16 samples each at a pass rate near 0.35, over a range of true seed spreads. *Resolved* is the fraction of trials in which the subtraction returns something positive; *estimate* is the median of what it returns.

Three readings from Table 20, and the first is a warning about the method rather than about the data. At a true spread of zero the subtraction still returns a positive number in about half of trials, because a difference of two noisy variances is positive half the time. A non-zero reading is therefore not by itself evidence of a seed spread; the interval is.

Second, the design resolves what it needs to. At a true spread of 0.02 the subtraction recovers something in 88% of trials at eight runs and lands at a median of 0.018; at 0.04 it succeeds essentially always. Those bracket the forecast we make.

Third, the bootstrap interval is anti-conservative here: its coverage of the true value runs from 68% to 88% rather than the nominal 95%, which is the percentile bootstrap applied to a variance difference at small n. We report the intervals because they are far more informative than a point estimate, and note that they are narrower than they should be.

D The estimator family as a table

Because the advantage depends only on (k, r), an estimator is a $(G + 1) \times 2$ array, and the moments in (14) are contractions of that array against a binomial distribution. This is what makes the exact treatment cheap: no sampling is involved in evaluating $\lambda(p, G)$, Q_r or C_{ab} for any member of the family, at any group size.

E Noise terms across task families

The corollary relating within-prompt noise to the reward histogram assumes a shared score scale, which holds within a task family and not between families of different answer shape. Table 21 and Figure 17 make the size of that effect explicit.

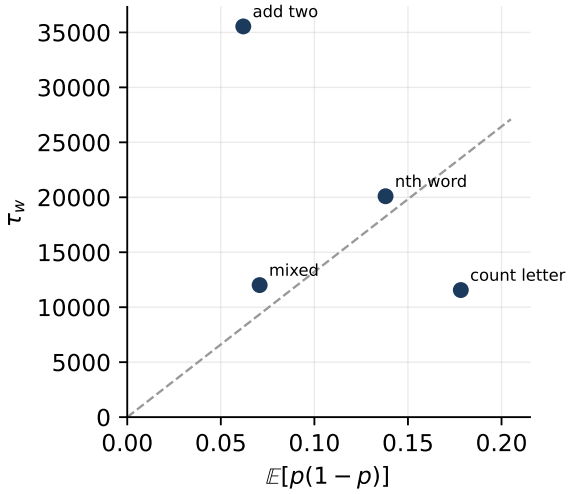


Figure 17: Within-prompt noise against the reward histogram across corpora, with a single fitted scale. The scatter is the score scale differing between task families.

corpus	pass rate	$\mathbb{E}[p(1-p)]$	τ_b	τ_w	$G^\star$	$\tau_w/\mathbb{E}[p(1-p)]$
count letter	0.280	0.178	4.35e+02	1.16e+04	6.15	0.27
nth word	0.326	0.138	4.88e+03	2.01e+04	3.03	0.61
add two	0.546	0.062	1.29e+04	3.55e+04	2.66	2.40
mixed	0.193	0.071	4.31e+03	1.20e+04	2.67	0.71

Table 21: Noise terms measured on a pretrained model. The last column would be constant if the score scale were shared across families; it is not, and it is not meant to be.